Team Note of HCMUS - StellarFlow

Hoang Duy Vu, Do Gia Huy, Nguyen Thanh Nhan

Compiled on October 13, 2026

Contents

1 Misc 1
2 Data Structure 2
3 Trigonometry 4
4 Geometry 4
5 Graph 12
6 String 23
7 Arithmetic 26
8 Numerical 34

1 Misc

1.1 FastIO

```
/*
 Usage: FastIO in; in.init(); in >> n;
*/

struct FastIO{
    string buffer;
    vector<ll> king;
    void init(ll _n = 0){
        getline(cin, buffer, '\0');
        buffer += "\n";
        king.reserve(_n);
        ll cur = 0;
        for(char c: buffer){
            if (c <= '9' && c >= '0'){
                if (cur == -1) cur = 0;
                cur = cur * 10 + (c - '0');
            }
            else{
                if (cur != -1) king.push_back(cur);
                cur = -1;
            }
        }

        reverse(ALL(king));
    }

    template <class T>
    FastIO& operator >> (T &x){
        x = 0;
        if (king.empty()) return *this;
        x = king.back(); king.pop_back();
        return *this;
    }

    bool isEmpty(){return king.empty();}
};
```

1.2 Stress Tester

```
@echo off
title My futile attempt at writing a stress tester.

echo Compiling files, please wait...
g++ -std=c++14 -o test.exe test.cpp
echo Done compiling test generator!

g++ -std=c++14 -o brute.exe brute.cpp
echo Done compiling bruteforce submission!

g++ -std=c++14 -o sol.exe sol.cpp
echo Done compiling (possibly) faulty submission!

for /l %%x in (1, 1, 1000000) do (
  echo Testcase numero: %%x
  test > input.inp
  sol < input.inp > output.out
  brute < input.inp > output.ans
  fc output.out output.ans > log.txt || call :WA

)
echo Accepted!
pause
exit

:WA
echo Wrong Answer
echo Test case:
type input.inp
echo Participant answer:
type output.out
echo Jury answer:
type output.ans

pause
exit
```

1.3 Stress Tester (Ubuntu)

```
/*
    chmod +x run.sh
    ./run.sh
*/

#!/bin/bash
g++ -O3 -std=c++17 gen.cpp -o gen || exit 1
g++ -O3 -std=c++17 brute.cpp -o brute || exit 1
g++ -O3 -std=c++17 sol.cpp -o sol || exit 1

for ((i = 1; ; ++i)); do
    ./gen > input.txt
    ./brute < input.txt > brute_out.txt
    ./sol < input.txt > sol_out.txt

    if ! diff -w brute_out.txt sol_out.txt > /dev/null; then
        echo "============================="
        echo "--> WA o test $i!"
        echo "Input:"
        cat input.txt
        echo "Expected Output (Brute):"
        cat brute_out.txt
        echo "Your Output (Sol):"
        cat sol_out.txt
        echo "============================="
        break
    fi
    echo "Passed test $i"
done
```

1.4 Bench Marker

```
@echo off
title Bench Marker 6000
echo Compiling files, please wait...
g++ -O2 -Wl,--stack=268435456 -std=c++11   -o test.exe
test.cpp
echo Done compiling test generator!
g++ -O2 -Wl,--stack=268435456 -std=c++11   -o sol.exe sol.cpp
echo Done compiling (probably) faulty submission!

for /l %%x in (1, 1, 1000) do (
  echo Test case numero: %%x
  test > input.inp
```

```
  sol < input.inp > output.out
)

pause
exit
```

1.5 Min Plus Convolution

```
// a is convex a[i+1]-a[i] <= a[i+2]-a[i+1]
vector<int> min_plus_convolution(vector<int>& a, vector<int>&
b) {
    int n = sz(a), m = sz(b);
    vector<int> c(n + m - 1, INF);
    auto dc = [&](auto Y, int l, int r, int jl, int jr) {
        if (l > r) return;
        int mid = (l + r) / 2, from = -1, & best = c[mid];
        for (int j = jl; j <= jr; ++j)
            if (int i = mid - j; i >= 0 && i < n)
                if (best > a[i] + b[j])
                    best = a[i] + b[j], from = j;
        Y(Y, l, mid - 1, jl, from);
        Y(Y, mid + 1, r, from, jr);
        };
    return dc(dc, 0, n - 1 + m - 1, 0, m - 1), c;
}
```

1.6 Mathematics

Equations

$$ax^2 + bx + c = 0 \Rightarrow x = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Sums

$$c^a + c^{a+1} + \cdots + c^b = \frac{c^{b+1} - c^a}{c - 1}, c \neq 1$$

$$1 + 2 + 3 + \cdots + n = \frac{n(n+1)}{2}$$

$$1^2 + 2^2 + 3^2 + \cdots + n^2 = \frac{n(2n+1)(n+1)}{6}$$

$$1^3 + 2^3 + 3^3 + \cdots + n^3 = \frac{n^2(n+1)^2}{4}$$

$$1^4 + 2^4 + 3^4 + \cdots + n^4 = \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}$$

Series

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots, (-\infty < x < \infty)$$

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \cdots, (-1 < x \leq 1)$$

$$\sqrt{1+x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{2x^3}{32} - \frac{5x^4}{128} + \cdots, (-1 \leq x \leq 1)$$

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots, (-\infty < x < \infty)$$

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \cdots, (-\infty < x < \infty)$$

Inequalities

$$(a_1^2 + \cdots + a_n^2)(b_1^2 + \cdots + b_n^2) \geq (a_1 b_1 + \cdots + a_n b_n)^2$$

$$(a_1 + \cdots + a_n)^2 \leq n(a_1^2 + \cdots + a_n^2)$$

$$\frac{x_1^2}{a_1} + \cdots + \frac{x_n^2}{a_n} \geq \frac{(x_1 + \cdots + x_n)^2}{a_1 + \cdots + a_n} \quad (a_i > 0)$$

2 Data Structure

2.1 Line Container

```
struct Line {
  mutable ll k, m, p;
  bool operator<(const Line& o) const { return k < o.k; }
  bool operator<(ll x) const { return p < x; }
};

struct LineContainer : multiset<Line, less<>> {
  // (for doubles, use inf = 1/.0, div(a,b) = a/b)
  static const ll inf = LLONG_MAX;
  ll div(ll a, ll b) { // floored division
    return a / b - ((a ^ b) < 0 && a % b); }
  bool isect(iterator x, iterator y) {
    if (y == end()) return x->p = inf, 0;
    if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
    else x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;
  }
  void add(ll k, ll m) {
    auto z = insert({k, m, 0}), y = z++, x = y;
    while (isect(y, z)) z = erase(z);
    if (x != begin() && isect(--x, y)) isect(x, y = erase(y));
    while ((y = x) != begin() && (--x)->p >= y->p)
      isect(x, erase(y));
  }
  ll query(ll x) {
    assert(!empty());
    auto l = *lower_bound(x);
    return l.k * x + l.m;
  }
};
```

2.2 Convex Hull Trick

```
/*
CHT template
*/

const ll INF = 1e18 + 69;

struct CHT{
    #define Node pair<ll, ll>
    vector<Node> a;
    vector<double> bubble;
    bool flag;

    // if flag = 0: if inserted in increasing order, return
    min, otherwise return max
    // flag = 1: the opposite

    CHT(bool _flag){
        flag = _flag;
    }

    double getInter(Node a, Node b){
        double x = (double) (b.second - a.second) / (a.first -
        b.first);
        return x;
    }

    void add(Node x, bool isMin = 1){
        if (a.empty()) {a.push_back(x); return;}
        if (a.back().first == x.first){
            if (isMin) minimize(a[a.size() - 1].second,
            x.second);
            else maximize(a[a.size() - 1].second, x.second);
            return;
        }
        while(a.size() >= 2){
            double x1 = getInter(a.back(), x), x2 =
            bubble.back();
            if (flag){
                if (x1 >= x2) break;
            }
            else{
                if (x1 <= x2) break;
            }
            a.pop_back(); bubble.pop_back();
        }
```

```
        bubble.push_back(getInter(a.back(), x));
        a.push_back(x);
    }

    ll get(ll x){
        if (a.empty()) return INF;
        long idx;
        if (flag) idx = lower_bound(ALL(bubble),x) -
        bubble.begin();
        else idx = lower_bound(ALL(bubble), x,
        greater<double>()) - bubble.begin();
        return a[idx].first * x + a[idx].second;
    }
};
```

2.3 Fast Segment Tree

```
struct SegmentTree{
    int n;
    vector<int> a;

    SegmentTree(int _n){
        n = _n;
        a.resize(n * 2 + 2, -INF);
    }

    void update(int i, int v){
        i += n; if (!maximize(a[i], v)) return;
        while(i > 1){
            i >>= 1;
            a[i] = max(a[i * 2], a[i * 2 + 1]);
        }
    }

    int get(int l, int r){
        l += n; r += n + 1;
        int ans = -INF;
        while(l < r){
            if (l & 1) maximize(ans, a[l++]);
            if (r & 1) maximize(ans, a[--r]);
            l >>= 1; r >>= 1;
        }
        return ans;
    }
};
```

2.4 Lichao Tree

```
/*
Lichao Tree
*/

const ll INF = 1e18;

struct LichaoTree{
    #define Node pair<ll, ll>

    long n;
    bool isMin;
    vector<Node> a;

    LichaoTree(long _n, bool flag = 1){
        n = _n;
        isMin = flag;
        if (flag) a.resize(n * 4 + 4, make_pair(0, INF));
        else a.resize(n * 4 + 4, make_pair(0, -INF));
    }

    ll f(Node t, ll x){return t.first * x + t.second;}

    void update(Node x, long l, long r, long id){
        bool check1 = f(a[id], l) >= f(x, l), check2 =
        f(a[id], r) >= f(x, r);
        bool check3 = f(a[id], l) <= f(x, l), check4 =
        f(a[id], r) <= f(x, r);
        if (check1 && check2){
            if (isMin) a[id] = x;
            return;
        }
        if (check3 && check4){
            if (!isMin) a[id] = x;
            return;
        }
        if (l == r) return;
        long mid = (l + r) >> 1;
        update(x, l, mid, id * 2);
        update(x, mid + 1, r, id * 2 + 1);
    }

    void update(Node x){update(x, 1, n, 1);}

    ll get(long i, long l, long r, long id){
        ll ans1 = f(a[id], i);
        if (l == r) return ans1;
        long mid = (l + r) >> 1;

        ll ans2;
        if (i <= mid) ans2 = get(i, l, mid, id * 2);
        else ans2 = get(i, mid + 1, r, id * 2 + 1);

        if (isMin) return min(ans1, ans2);
        else return max(ans1, ans2);
    }

    ll get(long i){return get(i, 1, n, 1);}
};
```

2.5 Persistent Segment Tree

```
class PersistentSegmentTree {
private:
    int n;
    vector<Node*> roots; // Store roots of different versions

    // Build initial segment tree
    Node* build(int start, int end) {
        if (start == end) return new Node(0); // Initialize
        with zero; customize as needed
        int mid = (start + end) / 2;
        return new Node(0, build(start, mid), build(mid + 1,
        end));
    }

    // Create new version with point update
    Node* update(Node* prev, int start, int end, int idx, int
    val) {
        if (start == end) return new Node(prev->value + val);
        // Update node value
        int mid = (start + end) / 2;
        if (idx <= mid)
            return new Node(prev->value + val,
            update(prev->left, start, mid, idx, val),
            prev->right);
        else
            return new Node(prev->value + val, prev->left,
            update(prev->right, mid + 1, end, idx, val));
    }

    // Query for a range [L, R]
    int query(Node* node, int start, int end, int L, int R) {
        if (start > R || end < L) return 0; // Return 0 for
        sum; adjust for other ops
        if (L <= start && end <= R) return node->value;
        int mid = (start + end) / 2;
        return query(node->left, start, mid, L, R) +
        query(node->right, mid + 1, end, L, R);
    }

public:
    PersistentSegmentTree(int size) : n(size) {
        roots.push_back(build(0, n - 1)); // Initial version
    }

    // Update tree version by creating a new version
    void update(int idx, int val) {
        roots.push_back(update(roots.back(), 0, n - 1, idx,
        val));
    }

    // Query a specific version
    int query(int version, int L, int R) {
        return query(roots[version], 0, n - 1, L, R);
    }
};
```

2.6 Treap

```
/*
Treap template
*/

struct BST{
    struct Node{
        int val, priority;
        Node* child[2];
        Node(int val): val(val), priority(rngesus(0, MASK(30)
        - 1)){
            child[0] = child[1] = nullptr;
        }
    };

    Node* root;

    BST(){
        root = nullptr;
    }

    void split(Node* cur, int x, Node* & L, Node*& R){ //
    split up to a point x
        if (cur == nullptr) {L = R = nullptr;return;}
        if (x < cur -> val){
            split(cur -> child[0], x, L, cur -> child[0]);
            R = cur;
        }
        else{
            split(cur -> child[1], x, cur -> child[1], R);
            L = cur;
        }
    }

    void merge(Node*& cur, Node* L, Node* R){
        if (L == nullptr) cur = R;
        else if (R == nullptr) cur = L;
        else{
            if (R -> priority > L -> priority) {
                cur = R;
                merge(cur -> child[0], L, R -> child[0]);
            }
            else{
                cur = L;
                merge(cur -> child[1], L -> child[1], R);
            }
        }
    }

    void insert(int x){
        Node *L, *R;
        split(root, x, L, R);
        merge(root, L, new Node(x));
        merge(root, root, R);
    }

    void erase(int x){
        Node *L = nullptr, *mid = nullptr, *R = nullptr;
        split(root, x-1, L, mid);
        split(mid, x, mid, R);

        merge(root, L, R);
    }

    bool find(int x){
        Node *id = root;
        while(true){
            if (id == nullptr) return false;
            if (id -> val == x) return true;
            else if (x < id -> val) id = id -> child[0];
            else id = id -> child[1];
        }
    }

    void clear(){
        while(root != nullptr){
            int val = root -> val;
            erase(val);
        }
    }
};
```

3 Trigonometry

Basics. Angles are in radians; $180^\circ = \pi$. For $k \in \mathbb{Z}$,

$$\sin^2 x + \cos^2 x = 1, \qquad 1 + \tan^2 x = \sec^2 x,$$
$$\sin(-x) = -\sin x, \qquad \cos(-x) = \cos x.$$
$$\sin(x + 2k\pi) = \sin x, \quad \cos(x + 2k\pi) = \cos x, \quad \tan(x + k\pi) = \tan x.$$

Addition and multiple angles.

$$\sin(a \pm b) = \sin a \cos b \pm \cos a \sin b,$$
$$\cos(a \pm b) = \cos a \cos b \mp \sin a \sin b,$$
$$\tan(a \pm b) = \frac{\tan a \pm \tan b}{1 \mp \tan a \tan b},$$
$$\sin 2x = 2 \sin x \cos x,$$
$$\cos 2x = \cos^2 x - \sin^2 x = 1 - 2\sin^2 x = 2\cos^2 x - 1,$$
$$\tan 2x = \frac{2\tan x}{1 - \tan^2 x}, \qquad \sin^2 \frac{x}{2} = \frac{1 - \cos x}{2}, \quad \cos^2 \frac{x}{2} = \frac{1 + \cos x}{2}.$$

Products and sums.

$$2 \sin a \cos b = \sin(a + b) + \sin(a - b),$$
$$2 \cos a \cos b = \cos(a + b) + \cos(a - b),$$
$$2 \sin a \sin b = \cos(a - b) - \cos(a + b),$$
$$\sin a + \sin b = 2 \sin \frac{a + b}{2} \cos \frac{a - b}{2},$$
$$\cos a + \cos b = 2 \cos \frac{a + b}{2} \cos \frac{a - b}{2}.$$

Triangle. For sides a, b, c opposite A, B, C and area K,

$$\frac{a}{\sin A} = \frac{b}{\sin B} = \frac{c}{\sin C} = 2R,$$
$$c^2 = a^2 + b^2 - 2ab \cos C, \qquad K = \frac{1}{2} ab \sin C.$$

Use `atan2(y,x)` for the signed direction in $(-\pi, \pi]$; clamp floating values to $[-1, 1]$ before `acos`/`asin`.

4 Geometry

4.1 Formula Sheet

Let a triangle have sides a, b, c, semiperimeter s, area K, inradius r, circumradius R, and median m_a.

$$s = \frac{a + b + c}{2}, \qquad K = \sqrt{s(s - a)(s - b)(s - c)} = rs = \frac{abc}{4R},$$
$$m_a = \frac{1}{2}\sqrt{2b^2 + 2c^2 - a^2}, \qquad h_a = \frac{2K}{a},$$
$$\ell_a = \frac{\sqrt{bc((b + c)^2 - a^2)}}{b + c}, \quad OI^2 = R(R - 2r).$$

Centroid divides each median $2 : 1$. If the angle bisector from A meets BC at D, then $BD : DC = AB : AC$. For a cyclic quadrilateral,

$$AC \cdot BD = AB \cdot CD + BC \cdot DA, \qquad K = \sqrt{(s - a)(s - b)(s - c)(s - d)}.$$

For a circle of radius r and central angle θ (radians),

$$L = r\theta, \qquad A_{\text{sector}} = \frac{r^2\theta}{2}, \qquad A_{\text{segment}} = \frac{r^2}{2}(\theta - \sin\theta).$$

For a regular n-gon of side a,

$$R = \frac{a}{2\sin(\pi/n)}, \quad r = \frac{a}{2\tan(\pi/n)}, \quad K = \frac{na^2}{4\tan(\pi/n)}.$$

Pick's theorem for a lattice polygon: $K = I + B/2 - 1$. Common volumes:

$$V_{\text{sphere}} = \frac{4}{3}\pi r^3, \qquad V_{\text{cone}} = \frac{1}{3}\pi r^2 h,$$
$$V_{\text{cylinder}} = \pi r^2 h, \qquad V_{\text{pyramid}} = \frac{1}{3} Bh.$$

For integer coordinates, use a wide type for cross products. For floating geometry, compare $|x| \le \varepsilon$ and avoid mixing incompatible epsilons.

4.2 Point Struct

```
template<class T> int sgn(T x) { return (x > 0) - (x < 0); }

template<class T>
struct Point {
    T x, y;
    explicit Point(T x = 0, T y = 0) : x(x), y(y) {}

    bool operator<(Point p) const { return tie(x, y) <
    tie(p.x, p.y); }
```

```
    bool operator==(Point p) const { return tie(x, y) ==
    tie(p.x, p.y); }
    bool operator!=(Point p) const { return !(*this == p); }

    Point operator+(Point p) const { return Point(x + p.x, y +
    p.y); }
    Point operator-(Point p) const { return Point(x - p.x, y -
    p.y); }
    Point operator*(T d) const { return Point(x * d, y * d); }
    Point operator/(T d) const { return Point(x / d, y / d); }
    Point operator-() const { return Point(-x, -y); }

    T dot(Point p) const { return x * p.x + y * p.y; }
    T cross(Point p) const { return x * p.y - y * p.x; }
    T cross(Point a, Point b) const { return (a -
    *this).cross(b - *this); }

    T dist2() const { return x * x + y * y; }
    double dist() const { return sqrt((double)dist2()); }
    Point unit() const { return *this / dist(); }
    Point perp() const { return Point(-y, x); }
    Point normal() const { return perp().unit(); }
    double angle() const { return atan2(y, x); }
    Point rotate(double a) const {
        return Point(x * cos(a) - y * sin(a),
                     x * sin(a) + y * cos(a));
    }

    friend istream& operator>>(istream& is, Point& p) { return
    is >> p.x >> p.y; }
    friend ostream& operator<<(ostream& os, const Point& p) {
    return os << p.x << " " << p.y; }
};
```

4.3 Line Distance

Time Complexity: $O(1)$

```
// Signed distance from p to directed line a->b. Positive on
the left.
// a == b is invalid. Products may overflow for large integer
coordinates.
template<class P>
double lineDist(P a, P b, P p) {
    return (double)(b - a).cross(p - a) / (b - a).dist();
}
```

4.4 Segment Distance

Time Complexity: $O(1)$

```
// Distance from p to segment [s,e]. Intended for
Point<double>.
double segDist(Point<double> s, Point<double> e, Point<double>
p) {
    if (s == e) return (p - s).dist();
    double d = (e - s).dist2();
    double t = min(d, max(0.0, (p - s).dot(e - s)));
    return ((p - s) * d - (e - s) * t).dist() / d;
}
```

4.5 Side of Line

Time Complexity: $O(1)$

```
// 1 / 0 / -1 means left / on / right of directed line s->e.
template<class P>
int sideOf(P s, P e, P p) {
    return sgn(s.cross(e, p));
}

template<class P>
int sideOf(P s, P e, P p, double eps) {
    auto area = (e - s).cross(p - s);
    double tolerance = (e - s).dist() * eps;
    return (area > tolerance) - (area < -tolerance);
}
```

4.6 Point on Segment

Time Complexity: $O(1)$

```
// Exact predicate. For Point<double>, prefer segDist(s,e,p)
<= EPS.
template<class P>
bool onSegment(P s, P e, P p) {
    return p.cross(s, e) == 0 && (s - p).dot(e - p) <= 0;
}
```

4.7 Line Intersection

Time Complexity: $O(1)$

```
// Returns {1, point} for a unique intersection, {0, _} for
parallel
// distinct lines, and {-1, _} for coincident lines.
template<class P>
pair<int, P> lineInter(P s1, P e1, P s2, P e2) {
    auto d = (e1 - s1).cross(e2 - s2);
    if (d == 0)
        return {-(s1.cross(e1, s2) == 0), P(0, 0)};
    auto p = s2.cross(e1, e2), q = s2.cross(e2, s1);
    return {1, (s1 * p + e1 * q) / d};
}
```

4.8 Segment Intersection

Time Complexity: $O(1)$

```
// Returns 0, 1, or 2 points. Two points describe the common
segment.
// With integer points, a non-integral unique intersection is
truncated.
template<class P>
vector<P> segInter(P a, P b, P c, P d) {
    auto oa = c.cross(d, a), ob = c.cross(d, b);
    auto oc = a.cross(b, c), od = a.cross(b, d);
    if (sgn(oa) * sgn(ob) < 0 && sgn(oc) * sgn(od) < 0)
        return {(a * ob - b * oa) / (ob - oa)};
    set<P> intersections;
    if (onSegment(c, d, a)) intersections.insert(a);
    if (onSegment(c, d, b)) intersections.insert(b);
    if (onSegment(a, b, c)) intersections.insert(c);
    if (onSegment(a, b, d)) intersections.insert(d);
    return {intersections.begin(), intersections.end()};
}
```

4.9 Line Projection and Reflection

Time Complexity: $O(1)$

```
// Projection of p onto line ab; refl=true reflects p across
the line.
// a == b is invalid. Use floating points for non-integral
answers.
template<class P>
P lineProj(P a, P b, P p, bool refl = false) {
    P v = b - a;
    return p - v.perp() * (1 + refl) * v.cross(p - a) /
    v.dist2();
}
```

4.10 Linear Transformation

Time Complexity: $O(1)$

```
// Maps p0->p1 to q0->q1 by translation, rotation and scaling,
then maps r.
// Requires p0 != p1. Intended for Point<double>.
using P = Point<double>;
P linearTransformation(P p0, P p1, P q0, P q1, P r) {
    P dp = p1 - p0, dq = q1 - q0;
    P num(dp.cross(dq), dp.dot(dq));
    return q0 + P((r - p0).cross(num), (r - p0).dot(num)) /
    dp.dist2();
}
```

4.11 Angle

Usage: Exact integer-angle ordering for rotational sweeps. **Time Complexity:** $O(1)$

```
// Exact integer-angle ordering with an explicit number of
full rotations.
struct Angle {
    int x, y, turns;
    Angle(int x, int y, int turns = 0) : x(x), y(y),
    turns(turns) {}
    Angle operator-(Angle b) const { return {x - b.x, y - b.y,
    turns}; }
    int half() const {
        assert(x || y);
        return y < 0 || (y == 0 && x < 0);
    }
    Angle t90() const { return {-y, x, turns + (half() && x >=
    0)}; }
    Angle t180() const { return {-x, -y, turns + half()}; }
```

```
    Angle t360() const { return {x, y, turns + 1}; }
};

bool operator<(Angle a, Angle b) {
    return make_tuple(a.turns, a.half(), a.y * (ll)b.x) <
           make_tuple(b.turns, b.half(), a.x * (ll)b.y);
}

pair<Angle, Angle> segmentAngles(Angle a, Angle b) {
    if (b < a) swap(a, b);
    return b < a.t180() ? make_pair(a, b) : make_pair(b,
    a.t360());
}

Angle operator+(Angle a, Angle b) {
    Angle r(a.x + b.x, a.y + b.y, a.turns);
    if (a.t180() < r) --r.turns;
    return r.t180() < a ? r.t360() : r;
}

Angle angleDiff(Angle a, Angle b) {
    int turns = b.turns - a.turns;
    a.turns = b.turns;
    return {a.x * b.x + a.y * b.y,
            a.x * b.y - a.y * b.x,
            turns - (b < a)};
}
```

4.12 Circle Tangents

Time Complexity: $O(1)$

```
// External tangents of two circles; negate r2 for internal
tangents.
// Returns 0, 1, or 2 pairs of tangency points.
template<class P>
vector<pair<P, P>> tangents(P c1, double r1, P c2, double r2)
{
    P d = c2 - c1;
    double dr = r1 - r2, d2 = d.dist2(), h2 = d2 - dr * dr;
    if (d2 == 0 || h2 < 0) return {};
    vector<pair<P, P>> out;
    for (double sign : {-1.0, 1.0}) {
        P v = (d * dr + d.perp() * sqrt(max(0.0, h2)) * sign)
        / d2;
        out.push_back({c1 + v * r1, c2 + v * r2});
    }
    if (h2 == 0) out.pop_back();
    return out;
}
```

4.13 Circumcircle

Time Complexity: $O(1)$

```
using P = Point<double>;

// A, B, C must be non-collinear.
double ccRadius(P A, P B, P C) {
    return (B - A).dist() * (C - B).dist() * (A - C).dist() /
           abs((B - A).cross(C - A)) / 2;
}

P ccCenter(P A, P B, P C) {
    P b = C - A, c = B - A;
    return A + (b * c.dist2() - c * b.dist2()).perp() /
               b.cross(c) / 2;
}
```

4.14 Polygon Area

Time Complexity: $O(n)$

```
// Twice the signed area: positive for CCW, negative for CW.
template<class T>
T polygonArea2(const vector<Point<T>>& polygon) {
    if (polygon.empty()) return 0;
    T area = polygon.back().cross(polygon[0]);
    for (int i = 0; i + 1 < (int)polygon.size(); ++i)
        area += polygon[i].cross(polygon[i + 1]);
    return area;
}
```

4.15 Polygon Center

Time Complexity: $O(n)$

```
// Center of mass of a non-degenerate polygon. Orientation may
be CW or CCW.
using P = Point<double>;
P polygonCenter(const vector<P>& polygon) {
    P result(0, 0);
    double area2 = 0;
    for (int i = 0, j = (int)polygon.size() - 1;
         i < (int)polygon.size(); j = i++) {
        double cross = polygon[j].cross(polygon[i]);
        result = result + (polygon[i] + polygon[j]) * cross;
        area2 += cross;
    }
    assert(area2 != 0);
    return result / area2 / 3;
}
```

4.16 Polygon Cut

Time Complexity: $O(n)$

```
// Removes everything strictly left of directed line s->e.
// Boundary points are kept. Intended for Point<double>.
using P = Point<double>;
vector<P> polygonCut(const vector<P>& polygon, P s, P e) {
    vector<P> result;
    for (int i = 0; i < (int)polygon.size(); ++i) {
        P current = polygon[i];
        P previous = i ? polygon[i - 1] : polygon.back();
        double a = s.cross(e, current), b = s.cross(e,
        previous);
        if ((a < 0) != (b < 0))
            result.push_back(current + (previous - current) *
            (a / (a - b)));
        if (a < 0) result.push_back(current);
    }
    return result;
}
```

4.17 Polygon Union Area

Usage: Input polygons must be simple and CCW. **Time Complexity:** $O(N^2)$

```
// Area of the union of CCW simple polygons. O(N^2) total
vertices.
using P = Point<double>;
double edgeRatio(P a, P b) { return sgn(b.x) ? a.x / b.x : a.y
/ b.y; }

double polygonUnionArea(vector<vector<P>>& polygons) {
    double answer = 0;
    for (int i = 0; i < (int)polygons.size(); ++i) {
        for (int v = 0; v < (int)polygons[i].size(); ++v) {
            P A = polygons[i][v];
            P B = polygons[i][(v + 1) % polygons[i].size()];
            vector<pair<double, int>> events = {{0, 0}, {1,
            0}};
            for (int j = 0; j < (int)polygons.size(); ++j) if
            (i != j) {
                for (int u = 0; u < (int)polygons[j].size();
                ++u) {
                    P C = polygons[j][u];
                    P D = polygons[j][(u + 1) %
                    polygons[j].size()];
                    int sideC = sideOf(A, B, C), sideD =
                    sideOf(A, B, D);
                    if (sideC != sideD) {
                        double sideA = C.cross(D, A), sideB =
                        C.cross(D, B);
                        if (min(sideC, sideD) < 0)
                            events.emplace_back(sideA / (sideA
                            - sideB),
                                                sgn(sideC -
                                                sideD));
                    } else if (!sideC && !sideD && j < i &&
                               sgn((B - A).dot(D - C)) > 0) {
                        events.emplace_back(edgeRatio(C - A, B
                        - A), 1);
                        events.emplace_back(edgeRatio(D - A, B
                        - A), -1);
                    }
```

```
                }
            }
            sort(events.begin(), events.end());
            for (auto& event : events)
                event.first = clamp(event.first, 0.0, 1.0);
            double uncovered = 0;
            int cover = events[0].second;
            for (int j = 1; j < (int)events.size(); ++j) {
                if (!cover) uncovered += events[j].first -
                events[j - 1].first;
                cover += events[j].second;
            }
            answer += A.cross(B) * uncovered;
        }
    }
    return answer / 2;
}
```

4.18 Convex Hull

Time Complexity: $O(n \log n)$

```
// Strict convex hull in CCW order; removes collinear points
on edges.
// O(n log n). Duplicate input points are allowed.
using P = Point<ll>;
vector<P> convexHull(vector<P> points) {
    if (points.size() <= 1) return points;
    sort(points.begin(), points.end());
    points.erase(unique(points.begin(), points.end()),
    points.end());
    if (points.size() <= 1) return points;

    vector<P> hull(points.size() + 1);
    int start = 0, top = 0;
    for (int pass = 0; pass < 2; ++pass) {
        for (P p : points) {
            while (top >= start + 2 &&
                   hull[top - 2].cross(hull[top - 1], p) <= 0)
                --top;
            hull[top++] = p;
        }
        start = --top;
        reverse(points.begin(), points.end());
    }
    hull.resize(top);
    return hull;
}
```

4.19 Point in Convex Hull

Time Complexity: $O(\log n)$

```
// Point in strict CCW convex hull without collinear vertices.
O(log n).
using P = Point<ll>;
bool inHull(const vector<P>& hull, P point, bool strict =
true) {
    int n = (int)hull.size(), a = 1, b = n - 1, boundary =
    !strict;
    if (n == 0) return false;
    if (n < 3) return boundary && onSegment(hull[0],
    hull.back(), point);
    if (sideOf(hull[0], hull[a], hull[b]) > 0) swap(a, b);
    if (sideOf(hull[0], hull[a], point) >= boundary ||
        sideOf(hull[0], hull[b], point) <= -boundary)
        return false;
    while (abs(a - b) > 1) {
        int middle = (a + b) / 2;
        if (sideOf(hull[0], hull[middle], point) > 0) b =
        middle;
        else a = middle;
    }
    return sgn(hull[a].cross(hull[b], point)) < boundary;
}
```

4.20 Line–Convex Hull Intersection

Time Complexity: $O(\log n)$

```
// Convex polygon must be CCW with no collinear consecutive
vertices.
// Returns crossed side indices; {-1,-1} means disjoint. O(log
n).
template<class P>
int extremeVertex(const vector<P>& polygon, P direction) {
    int n = (int)polygon.size();
    auto compare = [&](int i, int j) {
        return sgn(direction.perp().cross(
            polygon[i % n] - polygon[j % n]));
    };
    auto extreme = [&](int i) {
        return compare(i + 1, i) >= 0 && compare(i, (i - 1 +
        n) % n) < 0;
    };
    int low = 0, high = n;
    if (extreme(0)) return 0;
    while (low + 1 < high) {
        int middle = (low + high) / 2;
        if (extreme(middle)) return middle;
        int leftSlope = compare(low + 1, low);
        int middleSlope = compare(middle + 1, middle);
        if (leftSlope < middleSlope ||
            (leftSlope == middleSlope && leftSlope ==
            compare(low, middle)))
            high = middle;
        else
            low = middle;
    }
    return low;
}

template<class P>
array<int, 2> lineHull(P a, P b, const vector<P>& polygon) {
    int n = (int)polygon.size();
    assert(n >= 3 && a != b);
    auto compareLine = [&](int i) { return
    sgn(a.cross(polygon[i], b)); };
    int endA = extremeVertex(polygon, (a - b).perp());
    int endB = extremeVertex(polygon, (b - a).perp());
    if (compareLine(endA) < 0 || compareLine(endB) > 0) return
    {-1, -1};
    array<int, 2> result;
    for (int pass = 0; pass < 2; ++pass) {
        int low = endB, high = endA;
        while ((low + 1) % n != high) {
            int middle = ((low + high + (low < high ? 0 : n))
            / 2) % n;
            if (compareLine(middle) == compareLine(endB)) low
            = middle;
            else high = middle;
        }
        result[pass] = (low + !compareLine(high)) % n;
        swap(endA, endB);
    }
    if (result[0] == result[1]) return {result[0], -1};
    if (!compareLine(result[0]) && !compareLine(result[1])) {
        switch ((result[0] - result[1] + n + 1) % n) {
            case 0: return {result[0], result[0]};
            case 2: return {result[1], result[1]};
        }
    }
    return result;
}
```

4.21 Closest Pair

Time Complexity: $O(n \log n)$

```
// Closest pair of integer points in O(n log n). Requires at
least 2 points.
using P = Point<ll>;
pair<P, P> closestPair(vector<P> points) {
    assert(points.size() > 1);
    set<P> active;
    sort(points.begin(), points.end(), [](P a, P b) {
        return tie(a.y, a.x) < tie(b.y, b.x);
    });
    pair<ll, pair<P, P>> best{LLONG_MAX, {P(), P()}};
    int left = 0;
    for (P point : points) {
        ll width = 1 + (ll)sqrt((long double)best.first);
        P delta(width, 0);
        while (left < (int)points.size() &&
               points[left].y <= point.y - width)
            active.erase(points[left++]);
        auto low = active.lower_bound(point - delta);
        auto high = active.upper_bound(point + delta);
```

```
        for (; low != high; ++low)
            best = min(best, {(*low - point).dist2(), {*low,
            point}});
        active.insert(point);
    }
    return best.second;
}
```

4.22 KD-tree

Usage: Static nearest-neighbor queries. **Time Complexity:** Build $O(n \log n)$, query average $O(\log n)$

```
// Static 2D KD-tree for nearest-neighbor queries on integer
points.
using KDValue = long long;
using KDPoint = Point<KDValue>;

struct KDNode {
    KDPoint point;
    KDValue x0 = LLONG_MAX, x1 = LLONG_MIN;
    KDValue y0 = LLONG_MAX, y1 = LLONG_MIN;
    unique_ptr<KDNode> first, second;

    explicit KDNode(vector<KDPoint> points) : point(points[0])
    {
        for (KDPoint p : points) {
            x0 = min(x0, p.x); x1 = max(x1, p.x);
            y0 = min(y0, p.y); y1 = max(y1, p.y);
        }
        if (points.size() <= 1) return;
        bool splitX = x1 - x0 >= y1 - y0;
        sort(points.begin(), points.end(), [&](KDPoint a,
        KDPoint b) {
            return splitX ? tie(a.x, a.y) < tie(b.x, b.y)
                          : tie(a.y, a.x) < tie(b.y, b.x);
        });
        int half = (int)points.size() / 2;
        first =
        make_unique<KDNode>(vector<KDPoint>(points.begin(),
        points.begin() + half));
        second =
        make_unique<KDNode>(vector<KDPoint>(points.begin() +
        half, points.end()));
    }

    KDValue boxDistance(KDPoint p) const {
        KDValue x = clamp(p.x, x0, x1), y = clamp(p.y, y0,
        y1);
        return (KDPoint(x, y) - p).dist2();
    }
};

struct KDTree {
    unique_ptr<KDNode> root;
    explicit KDTree(const vector<KDPoint>& points) {
        assert(!points.empty());
        root = make_unique<KDNode>(points);
    }

    pair<KDValue, KDPoint> search(const KDNode* node, KDPoint
    query) const {
        if (!node->first)
            return {(query - node->point).dist2(),
            node->point};
        const KDNode *first = node->first.get(), *second =
        node->second.get();
        KDValue firstBound = first->boxDistance(query);
        KDValue secondBound = second->boxDistance(query);
        if (firstBound > secondBound) {
            swap(firstBound, secondBound);
            swap(first, second);
        }
        auto best = search(first, query);
        if (secondBound < best.first) best = min(best,
        search(second, query));
        return best;
    }

    pair<KDValue, KDPoint> nearest(KDPoint query) const {
        return search(root.get(), query);
    }
};
```

4.23 Delaunay Triangulation

Usage: Via lifting to a 3D hull; excludes degenerate inputs. **Time Complexity:** $O(n^2)$

```
// O(n^2) Delaunay via lifting to a 3D hull.
// Undefined for 3 collinear or 4 cocircular input points.
template<class P, class TriangleCallback>
void delaunay(vector<P>& points, TriangleCallback addTriangle)
{
    if (points.size() < 3) return;
    if (points.size() == 3) {
        int reversed = points[0].cross(points[1], points[2]) <
        0;
        addTriangle(0, 1 + reversed, 2 - reversed);
        return;
    }
    vector<P3> lifted;
    for (P point : points)
        lifted.emplace_back(point.x, point.y, point.dist2());
    for (auto face : hull3d(lifted)) {
        P3 normal = (lifted[face.b] - lifted[face.a]).cross(
                    lifted[face.c] - lifted[face.a]);
        if (normal.dot(P3(0, 0, 1)) < 0)
            addTriangle(face.a, face.c, face.b);
    }
}
```

4.24 Fast Delaunay Triangulation

Usage: Distinct integer points; collinear sets return no triangles. **Time Complexity:** $O(n \log n)$

```
// O(n log n) Delaunay triangulation for distinct integer
points.
// Collinear sets return no triangles. Coordinates should fit
__int128 products.
using DPoint = Point<ll>;
using WideInt = __int128_t;
struct DelaunayQuad;
using DQuad = DelaunayQuad*;

const DPoint DELAUNAY_NONE(LLONG_MAX, LLONG_MAX);
struct DelaunayQuad {
    DQuad rot, nextEdge;
    DPoint point = DELAUNAY_NONE;
    bool marked = false;
    DPoint& destination() { return reverse()->point; }
    DQuad& reverse() { return rot->rot; }
    DQuad previous() { return rot->nextEdge->rot; }
    DQuad next() { return reverse()->previous(); }
};

DQuad delaunayFreeList = nullptr;

bool inCircumcircle(DPoint p, DPoint a, DPoint b, DPoint c) {
    WideInt p2 = p.dist2();
    WideInt A = (WideInt)a.dist2() - p2;
    WideInt B = (WideInt)b.dist2() - p2;
    WideInt C = (WideInt)c.dist2() - p2;
    return (WideInt)p.cross(a, b) * C +
           (WideInt)p.cross(b, c) * A +
           (WideInt)p.cross(c, a) * B > 0;
}

DQuad makeDelaunayEdge(DPoint origin, DPoint destination) {
    DQuad edge;
    if (delaunayFreeList) {
        edge = delaunayFreeList;
        delaunayFreeList = edge->nextEdge;
    } else {
        DQuad q0 = new DelaunayQuad{};
        DQuad q1 = new DelaunayQuad{};
        DQuad q2 = new DelaunayQuad{};
        DQuad q3 = new DelaunayQuad{};
        q0->rot = q1; q1->rot = q2; q2->rot = q3; q3->rot =
        q0;
        edge = q0;
    }
    for (int i = 0; i < 4; ++i) {
        edge = edge->rot;
        edge->point = DELAUNAY_NONE;
        edge->marked = false;
        edge->nextEdge = (i & 1) ? edge : edge->reverse();
```

```
    }
    edge->point = origin;
    edge->destination() = destination;
    return edge;
}

void spliceDelaunay(DQuad a, DQuad b) {
    swap(a->nextEdge->rot->nextEdge,
    b->nextEdge->rot->nextEdge);
    swap(a->nextEdge, b->nextEdge);
}

DQuad connectDelaunay(DQuad a, DQuad b) {
    DQuad edge = makeDelaunayEdge(a->destination(), b->point);
    spliceDelaunay(edge, a->next());
    spliceDelaunay(edge->reverse(), b);
    return edge;
}

pair<DQuad, DQuad> delaunayRec(const vector<DPoint>& points) {
    int n = (int)points.size();
    if (n <= 3) {
        DQuad a = makeDelaunayEdge(points[0], points[1]);
        DQuad b = makeDelaunayEdge(points[1], points.back());
        if (n == 2) return {a, a->reverse()};
        spliceDelaunay(a->reverse(), b);
        auto side = points[0].cross(points[1], points[2]);
        DQuad c = side ? connectDelaunay(b, a) : nullptr;
        return {side < 0 ? c->reverse() : a,
                side < 0 ? c : b->reverse()};
    }

    int half = n / 2;
    DQuad leftInner, rightInner, leftOuter, rightOuter;
    tie(leftOuter, leftInner) = delaunayRec(
        vector<DPoint>(points.begin(), points.begin() +
        half));
    tie(rightInner, rightOuter) = delaunayRec(
        vector<DPoint>(points.begin() + half, points.end()));
    while ((rightInner->point.cross(leftInner->destination(),
    leftInner->point) < 0 &&
            (leftInner = leftInner->next())) ||
           (leftInner->point.cross(rightInner->destination(),
           rightInner->point) > 0 &&
            (rightInner = rightInner->reverse()->nextEdge)))
            {}

    DQuad base = connectDelaunay(rightInner->reverse(),
    leftInner);
    if (leftInner->point == leftOuter->point) leftOuter =
    base->reverse();
    if (rightInner->point == rightOuter->point) rightOuter =
    base;

    auto valid = [&](DQuad edge) {
        return edge->destination().cross(base->destination(),
        base->point) > 0;
    };
    auto removeBad = [&](DQuad initial, bool followNextEdge) {
        DQuad edge = followNextEdge ? initial->nextEdge :
        initial->previous();
        if (valid(edge)) {
            auto advance = [&](DQuad q) {
                return followNextEdge ? q->nextEdge :
                q->previous();
            };
            while
            (inCircumcircle(advance(edge)->destination(),
                            base->destination(),
                            base->point,
                            edge->destination())) {
                DQuad next = advance(edge);
                spliceDelaunay(edge, edge->previous());
                spliceDelaunay(edge->reverse(),
                edge->reverse()->previous());
                edge->nextEdge = delaunayFreeList;
                delaunayFreeList = edge;
                edge = next;
            }
        }
        return edge;
    };

    while (true) {
        DQuad leftCandidate = removeBad(base->reverse(),
        true);
        DQuad rightCandidate = removeBad(base, false);
        if (!valid(leftCandidate) && !valid(rightCandidate))
        break;
        if (!valid(leftCandidate) ||
            (valid(rightCandidate) &&
             inCircumcircle(rightCandidate->destination(),
             base->destination(),
                            base->point,
                            leftCandidate->destination())))
            base = connectDelaunay(rightCandidate,
            base->reverse());
        else
            base = connectDelaunay(base->reverse(),
            leftCandidate->reverse());
    }
    return {leftOuter, rightOuter};
}

// Flat output: every consecutive triple is one CCW triangle.
vector<DPoint> fastDelaunay(vector<DPoint> points) {
    sort(points.begin(), points.end());
    assert(unique(points.begin(), points.end()) ==
    points.end());
    if (points.size() < 2) return {};
    DQuad edge = delaunayRec(points).first;
    while
    (edge->nextEdge->destination().cross(edge->destination(),
    edge->point) < 0)
        edge = edge->nextEdge;

    vector<DQuad> queue;
    vector<DPoint> result;
    auto addFace = [&](DQuad start) {
        DQuad current = start;
        do {
            current->marked = true;
            result.push_back(current->point);
            queue.push_back(current->reverse());
            current = current->next();
        } while (current != start);
    };
    addFace(edge);
    result.clear(); // discard the outer face
    for (int i = 0; i < (int)queue.size(); ++i)
        if (!queue[i]->marked) addFace(queue[i]);
    return result;
}
```

4.25 Point 3D

```
template<class T>
struct Point3D {
    using P = Point3D;
    T x, y, z;
    explicit Point3D(T x = 0, T y = 0, T z = 0) : x(x), y(y),
    z(z) {}
    bool operator<(P p) const { return tie(x, y, z) < tie(p.x,
    p.y, p.z); }
    bool operator==(P p) const { return tie(x, y, z) ==
    tie(p.x, p.y, p.z); }
    P operator+(P p) const { return P(x + p.x, y + p.y, z +
    p.z); }
    P operator-(P p) const { return P(x - p.x, y - p.y, z -
    p.z); }
    P operator*(T d) const { return P(x * d, y * d, z * d); }
    P operator/(T d) const { return P(x / d, y / d, z / d); }
    T dot(P p) const { return x * p.x + y * p.y + z * p.z; }
    P cross(P p) const {
        return P(y * p.z - z * p.y,
                 z * p.x - x * p.z,
                 x * p.y - y * p.x);
    }
    T dist2() const { return x * x + y * y + z * z; }
    double dist() const { return sqrt((double)dist2()); }
    double phi() const { return atan2(y, x); }
    double theta() const { return atan2(hypot((double)x,
    (double)y), z); }
```

```
    P unit() const { return *this / dist(); }
    P normal(P p) const { return cross(p).unit(); }
    P rotate(double angle, P axis) const {
        double sine = sin(angle), cosine = cos(angle);
        P unitAxis = axis.unit();
        return unitAxis * dot(unitAxis) * (1 - cosine) +
               (*this) * cosine - cross(unitAxis) * sine;
    }
};
```

4.26 3D Convex Hull

Usage: At least four points; no four coplanar. **Time Complexity:** $O(n^2)$

```
// O(n^2). Requires >=4 points and no four coplanar points.
using P3 = Point3D<double>;

struct HullEdge {
    int first = -1, second = -1;
    void insert(int x) { (first == -1 ? first : second) = x; }
    void remove(int x) { (first == x ? first : second) = -1; }
    int count() const { return (first != -1) + (second != -1); }
    }
};

struct Face3D { P3 normal; int a, b, c; };

vector<Face3D> hull3d(const vector<P3>& points) {
    int n = (int)points.size();
    assert(n >= 4);
    vector<vector<HullEdge>> edges(n, vector<HullEdge>(n));
    vector<Face3D> faces;

    auto makeFace = [&](int i, int j, int k, int inside) {
        P3 normal = (points[j] - points[i]).cross(points[k] -
        points[i]);
        if (normal.dot(points[inside]) >
        normal.dot(points[i])) normal = normal * -1;
        Face3D face{normal, i, j, k};
        edges[i][j].insert(k); edges[i][k].insert(j);
        edges[j][k].insert(i);
        faces.push_back(face);
    };

    for (int i = 0; i < 4; ++i)
        for (int j = i + 1; j < 4; ++j)
            for (int k = j + 1; k < 4; ++k)
                makeFace(i, j, k, 6 - i - j - k);

    for (int i = 4; i < n; ++i) {
        for (int j = 0; j < (int)faces.size(); ++j) {
            Face3D face = faces[j];
            if (face.normal.dot(points[i]) <=
            face.normal.dot(points[face.a])) continue;
            edges[face.a][face.b].remove(face.c);
            edges[face.a][face.c].remove(face.b);
            edges[face.b][face.c].remove(face.a);
            faces[j--] = faces.back();
            faces.pop_back();
        }
        int oldFaces = (int)faces.size();
        for (int j = 0; j < oldFaces; ++j) {
            Face3D face = faces[j];
            if (edges[face.a][face.b].count() != 2)
                makeFace(face.a, face.b, i, face.c);
            if (edges[face.a][face.c].count() != 2)
                makeFace(face.a, face.c, i, face.b);
            if (edges[face.b][face.c].count() != 2)
                makeFace(face.b, face.c, i, face.a);
        }
    }
    for (Face3D& face : faces) {
        P3 geometric = (points[face.b] -
        points[face.a]).cross(
                       points[face.c] - points[face.a]);
        if (geometric.dot(face.normal) <= 0) swap(face.b,
        face.c);
    }
    return faces;
}
```

4.27 Polyhedron Volume

Usage: Triangular faces must point outwards. **Time Complexity:** $O(F)$

```
// Signed volume; triangular faces must point outwards.
template<class Points, class Faces>
double signedPolyVolume(const Points& points, const Faces&
faces) {
    double volume = 0;
    for (auto face : faces) {
        auto normal = points[face.a].cross(points[face.b]);
        volume += normal.dot(points[face.c]);
    }
    return volume / 6;
}
```

4.28 Spherical Distance

Time Complexity: $O(1)$

```
// Angles are radians: longitude phi and zenith angle theta
(0=north pole).
double sphericalDistance(double phi1, double theta1,
                         double phi2, double theta2, double
                         radius) {
    double dx = sin(theta2) * cos(phi2) - sin(theta1) *
    cos(phi1);
    double dy = sin(theta2) * sin(phi2) - sin(theta1) *
    sin(phi1);
    double dz = cos(theta2) - cos(theta1);
    double chord = sqrt(dx * dx + dy * dy + dz * dz);
    return radius * 2 * asin(clamp(chord / 2, 0.0, 1.0));
}
```

4.29 Circles Intersect

```
/*
 * Usage :
 * Computes the pair of points at which two circles intersect.
 -> out
 * Returns false in case of no intersection.
 */

typedef Point<double> P;
bool circleInter(P a,P b,double r1,double r2,pair<P, P>* out)
{
  if (a == b) { assert(r1 != r2); return false; }
  P vec = b - a;
  double d2 = vec.dist2(), sum = r1+r2, dif = r1-r2,
         p = (d2 + r1*r1 - r2*r2)/(d2*2), h2 = r1*r1 - p*p*d2;
  if (sum*sum < d2 || dif*dif > d2) return false;
  P mid = a + vec*p, per = vec.perp() * sqrt(fmax(0, h2) /
  d2);
  *out = {mid + per, mid - per};
  return true;
}
```

4.30 Circle vs CCW Polygon Intersection

```
/*
 * Usage :
 * Returns the area of the intersection of a circle with a CCW
 polygon.
 * Time complexity : O(n)
*/

typedef Point<double> P;
double circlePoly(P c, double r, vector<P> ps) {
  auto angle = [](P p, P q) { return atan2(p.cross(q),
  p.dot(q)); };
  auto tri = [&](P p, P q) {
    auto r2 = r * r / 2;
    P d = q - p;
    auto a = d.dot(p)/d.dist2(), b =
    (p.dist2()-r*r)/d.dist2();
    auto det = a * a - b;
    if (det <= 0) return angle(p, q) * r2;
    auto s = max(0., -a-sqrt(det)), t = min(1., -a+sqrt(det));
    if (t < 0 || 1 <= s) return angle(p, q) * r2;
    P u = p + d * s, v = p + d * t;
    return angle(p,u) * r2 + u.cross(v)/2 + angle(v,q) * r2;
  };
  auto sum = 0.0;
```

```
  for (int i = 0; i < (int)ps.size(); ++i)
    sum += tri(ps[i] - c, ps[(i + 1) % ps.size()] - c);
  return sum;
}
```

4.31 Circle vs Line Intersect

```
/*
 * Usage :
 * Finds the intersection between a circle and a line.
 * Returns a vector of either 0, 1, or 2 intersection points.
 * P is intended to be Point<double>.
 */

template<class Point>
vector<Point> circleLine(Point c, double r, Point a, Point b)
{
  Point ab = b - a, p = a + ab * (c-a).dot(ab) / ab.dist2();
  double s = a.cross(b, c), h2 = r*r - s*s / ab.dist2();
  if (h2 < 0) return {};
  if (h2 == 0) return {p};
  Point h = ab.unit() * sqrt(h2);
  return {p - h, p + h};
}
```

4.32 Hull Diameter

```
/*
 * Usage :
 * Returns the two points with max distance on a convex hull
 (CCW, no duplicate/collinear points).
 * Time complexity : O(n)
*/

typedef Point<ll> P;
array<P, 2> hullDiameter(vector<P> S) {
  int n = (int)S.size(), j = n < 2 ? 0 : 1;
  assert(n > 0);
  pair<ll, array<P, 2>> res({0, {S[0], S[0]}});
  for (int i = 0; i < j; ++i)
    for (;; j = (j + 1) % n) {
      res = max(res, {(S[i] - S[j]).dist2(), {S[i], S[j]}});
      if ((S[(j + 1) % n] - S[j]).cross(S[i + 1] - S[i]) >= 0)
        break;
    }
  return res.second;
}
```

4.33 Manhattan Minimum Spanning Tree

Time Complexity:

```
struct Path{
    int u, v; ll w;
    Path(){}
    Path(int _u, int _v, ll _w){u = _u, v  =_v, w = _w;}
};

typedef Point<ll> P;
const ll INF = 1e18;
namespace ManhattanSpanningTreeSolver {
    vector<Path> solve(vector<P> ps) {
        vector<int> id(ps.size());
        iota(id.begin(), id.end(), 0);
        vector<Path> edges;
        for(int k = 0; k < 4; ++k) {
            sort(id.begin(), id.end(), [&](int i, int j) {
                return (ps[i]-ps[j]).x < (ps[j]-ps[i]).y;});
            map<ll, int> sweep;
            for (int i : id) {
                for (auto it = sweep.lower_bound(-ps[i].y);
                          it != sweep.end();
                          sweep.erase(it++)) {
                    int j = it->second;
                    P d = ps[i] - ps[j];
                    if (d.y > d.x) break;
                    edges.push_back(Path(i, j, d.y + d.x));
                }
                sweep[-ps[i].y] = i;
            }
            for (P& p : ps) if (k & 1) p.x = -p.x; else
            swap(p.x, p.y);
        }
        return edges;
    }
};
```

4.34 Point in Poly

```
/*
 * Usage :
 * Returns true if p lies within the polygon. If strict is
 true,
 * it returns false for points on the boundary.
 * The algorithm uses products in intermediate steps so watch
 out for overflow.
 * Time complexity : O(n)
*/

template<class P>
bool inPolygon(const vector<P>& polygon, P point, bool strict
= true) {
    int inside = 0, n = (int)polygon.size();
    for (int i = 0; i < n; ++i) {
        P next = polygon[(i + 1) % n];
        if (onSegment(polygon[i], next, point)) return
        !strict;
        inside ^= ((point.y < polygon[i].y) - (point.y <
        next.y)) *
                  point.cross(polygon[i], next) > 0;
    }
    return inside;
}
```

4.35 Smallest Enclosing Circle

```
using P = Point<double>;

// Expected O(n). Empty input returns center (0,0), radius 0.
pair<P, double> minimumEnclosingCircle(vector<P> points) {
    if (points.empty()) return {P(0, 0), 0};
    shuffle(points.begin(), points.end(), rng);
    P center = points[0];
    double radius = 0, tolerance = 1 + 1e-8;
    for (int i = 0; i < (int)points.size(); ++i) {
        if ((center - points[i]).dist() <= radius * tolerance)
        continue;
        center = points[i];
        radius = 0;
        for (int j = 0; j < i; ++j) {
            if ((center - points[j]).dist() <= radius *
            tolerance) continue;
            center = (points[i] + points[j]) / 2;
            radius = (center - points[i]).dist();
            for (int k = 0; k < j; ++k) {
                if ((center - points[k]).dist() <= radius *
                tolerance) continue;
                center = ccCenter(points[i], points[j],
                points[k]);
                radius = (center - points[i]).dist();
            }
        }
    }
    return {center, radius};
}
```

4.36 Minkowski Sum

```
/*
    Usage: minkowskiSum(a: convex polygon, b: convex polygon)
*/

template<class Point>
vector<Point> minkowskiSum(vector<Point> a, vector<Point> b) {
    rotate(begin(a), min_element(begin(a), end(a)), end(a));
    rotate(begin(b), min_element(begin(b), end(b)), end(b));
    int n = a.size(), m = b.size();
    vector<Point> h(n + m + 1); h[0] = a[0] + b[0];
    int t = 1;
    for (int i = 0, j = 0; i < n || j < m; ) {
        if (i == n) j++;
        else if (j == m) i++;
        else {
```

```
            Point pa = a[(i + 1) % n] - a[i], pb = b[(j + 1) %
            m] - b[j];
            auto cr = pa.cross(pb);
            if (cr >= 0) i++;
            if (cr <= 0) j++;
        }
        h[t++] = (a[i % n] + b[j % m]);
    }
    return {h.begin(), h.begin() + t - (t >= 2 && h[0] == h[t
    - 1])};
}
```

5 Graph

5.1 Bellman–Ford

Time Complexity: $O(VE)$

```
struct BFEdge { int from, to; ll weight; };

// INF: unreachable, -INF: reachable from a negative cycle.
O(VE).
pair<vector<ll>, vector<int>> bellmanFord(
        int n, const vector<BFEdge>& edges, int source) {
    const ll INF = (1LL << 62);
    vector<ll> distance(n, INF);
    vector<int> parent(n, -1), bad(n);
    distance[source] = 0;
    for (int iteration = 0; iteration < n; ++iteration) {
        bool changed = false;
        for (auto edge : edges) {
            if (distance[edge.from] == INF) continue;
            ll candidate = max(-INF, distance[edge.from] +
            edge.weight);
            if (candidate < distance[edge.to]) {
                distance[edge.to] = candidate;
                parent[edge.to] = edge.from;
                changed = true;
                if (iteration == n - 1) bad[edge.to] = true;
            }
        }
        if (!changed) break;
    }
    for (int iteration = 0; iteration < n; ++iteration)
        for (auto edge : edges)
            if (bad[edge.from]) bad[edge.to] = true;
    for (int vertex = 0; vertex < n; ++vertex)
        if (bad[vertex]) distance[vertex] = -INF;
    return {distance, parent};
}
```

5.2 Floyd–Warshall

Time Complexity: $O(V^3)$

```
// matrix[i][j] = INF when there is no edge. O(n^3).
// Output -INF means that arbitrarily small walks exist.
void floydWarshall(vector<vector<ll>>& matrix) {
    const ll INF = (1LL << 62);
    int n = (int)matrix.size();
    for (int i = 0; i < n; ++i) matrix[i][i] =
    min(matrix[i][i], 0LL);
    for (int k = 0; k < n; ++k)
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j)
                if (matrix[i][k] != INF && matrix[k][j] !=
                INF)
                    matrix[i][j] = min(matrix[i][j],
                        max(-INF, matrix[i][k] +
                        matrix[k][j]));
    for (int k = 0; k < n; ++k) if (matrix[k][k] < 0)
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j)
                if (matrix[i][k] != INF && matrix[k][j] !=
                INF)
                    matrix[i][j] = -INF;
}
```

5.3 Topological Sort

Time Complexity: $O(V + E)$

```
// Returns fewer than n vertices iff the directed graph has a
cycle. O(V+E).
vector<int> topologicalSort(const vector<vector<int>>& graph)
{
    int n = (int)graph.size();
    vector<int> degree(n), order;
    for (const auto& edges : graph)
        for (int to : edges) ++degree[to];
    for (int vertex = 0; vertex < n; ++vertex)
        if (!degree[vertex]) order.push_back(vertex);
    for (int at = 0; at < (int)order.size(); ++at)
        for (int to : graph[order[at]])
            if (!--degree[to]) order.push_back(to);
    return order;
}
```

5.4 Dinic Max Flow

Time Complexity: $O(V^2E)$

```
struct Dinic {
    struct Edge {
        int to, reverse;
        ll capacity, original;
        ll flow() const { return original - capacity; }
    };
    vector<vector<Edge>> graph;
    vector<int> level, nextEdge;

    Dinic(int n) : graph(n), level(n), nextEdge(n) {}

    void addEdge(int from, int to, ll capacity, ll
    reverseCapacity = 0) {
        graph[from].push_back(
            {to, (int)graph[to].size(), capacity, capacity});
        graph[to].push_back(
            {from, (int)graph[from].size() - 1,
             reverseCapacity, reverseCapacity});
    }

    bool bfs(int source, int sink) {
        fill(level.begin(), level.end(), -1);
        queue<int> queue;
        level[source] = 0;
        queue.push(source);
        while (!queue.empty()) {
            int vertex = queue.front();
            queue.pop();
            for (const Edge& edge : graph[vertex])
                if (edge.capacity > 0 && level[edge.to] == -1)
                {
                    level[edge.to] = level[vertex] + 1;
                    queue.push(edge.to);
                }
        }
        return level[sink] != -1;
    }

    ll dfs(int vertex, int sink, ll pushed) {
        if (vertex == sink || !pushed) return pushed;
        for (int& i = nextEdge[vertex]; i <
        (int)graph[vertex].size(); ++i) {
            Edge& edge = graph[vertex][i];
            if (edge.capacity > 0 && level[edge.to] ==
            level[vertex] + 1) {
                ll amount = dfs(edge.to, sink, min(pushed,
                edge.capacity));
                if (amount) {
                    edge.capacity -= amount;
                    graph[edge.to][edge.reverse].capacity +=
                    amount;
                    return amount;
                }
            }
        }
        return 0;
    }

    ll maxFlow(int source, int sink, ll limit = LLONG_MAX) {
        ll flow = 0;
        while (flow < limit && bfs(source, sink)) {
            fill(nextEdge.begin(), nextEdge.end(), 0);
            while (flow < limit) {
```

```
            ll pushed = dfs(source, sink, limit - flow);
            if (!pushed) break;
            flow += pushed;
        }
    }
    return flow;
}

void reset() {
    for (auto& edges : graph)
        for (Edge& edge : edges) edge.capacity =
        edge.original;
}
};
```

5.5 Push–Relabel Max Flow

Time Complexity: $O(V^2\sqrt{E})$

```
struct PushRelabel {
    struct Edge { int to, reverse; ll flow, capacity; };
    vector<vector<Edge>> graph;
    vector<ll> excess;
    vector<Edge*> current;
    vector<vector<int>> active;
    vector<int> height;

    PushRelabel(int n)
        : graph(n), excess(n), current(n), active(2 * n),
        height(n) {}

    void addEdge(int from, int to, ll capacity, ll
    reverseCapacity = 0) {
        if (from == to) return;
        graph[from].push_back({to, (int)graph[to].size(), 0,
        capacity});
        graph[to].push_back({from, (int)graph[from].size() -
        1, 0,
                             reverseCapacity});
    }

    void addFlow(Edge& edge, ll amount) {
        Edge& reverse = graph[edge.to][edge.reverse];
        if (!excess[edge.to] && amount)
            active[height[edge.to]].push_back(edge.to);
        edge.flow += amount;
        edge.capacity -= amount;
        excess[edge.to] += amount;
        reverse.flow -= amount;
        reverse.capacity += amount;
        excess[reverse.to] -= amount;
    }

    ll maxFlow(int source, int sink) {
        int n = (int)graph.size();
        height[source] = n;
        excess[sink] = 1;
        vector<int> count(2 * n);
        count[0] = n - 1;
        for (int i = 0; i < n; ++i) current[i] =
        graph[i].data();
        for (Edge& edge : graph[source]) addFlow(edge,
        edge.capacity);
        for (int highest = 0;;) {
            while (active[highest].empty())
                if (!highest--) return -excess[source];
            int vertex = active[highest].back();
            active[highest].pop_back();
            while (excess[vertex] > 0) {
                if (current[vertex] == graph[vertex].data() +
                graph[vertex].size()) {
                    height[vertex] = INT_MAX;
                    for (Edge& edge : graph[vertex])
                        if (edge.capacity && height[vertex] >
                        height[edge.to] + 1)
                            height[vertex] = height[edge.to] +
                            1,
                            current[vertex] = &edge;
                    ++count[height[vertex]];
                    if (!--count[highest] && highest < n)
                        for (int i = 0; i < n; ++i)
                            if (highest < height[i] &&
                            height[i] < n)
                                --count[height[i]], height[i]
                                = n + 1;
                    highest = height[vertex];
                } else if (current[vertex]->capacity
                        && height[vertex] ==
                        height[current[vertex]->to] + 1) {
                    addFlow(*current[vertex],
                            min(excess[vertex],
                            current[vertex]->capacity));
                } else {
                    ++current[vertex];
                }
            }
        }
    }

    bool sourceSideOfMinCut(int vertex) const {
        return height[vertex] >= (int)graph.size();
    }
};
```

5.6 Edmonds–Karp

Time Complexity: $O(VE^2)$

```
// Residual capacities are modified. O(VE^2).
template<class T>
T edmondsKarp(vector<unordered_map<int, T>>& graph, int
source, int sink) {
    assert(source != sink);
    int n = (int)graph.size();
    T flow = 0;
    vector<int> parent(n), queue(n);
    while (true) {
        fill(parent.begin(), parent.end(), -1);
        parent[source] = source;
        int head = 0, tail = 1;
        queue[0] = source;
        while (head < tail && parent[sink] == -1) {
            int vertex = queue[head++];
            for (auto edge : graph[vertex])
                if (edge.second > 0 && parent[edge.first] ==
                -1) {
                    parent[edge.first] = vertex;
                    queue[tail++] = edge.first;
                }
        }
        if (parent[sink] == -1) return flow;
        T add = numeric_limits<T>::max();
        for (int vertex = sink; vertex != source; vertex =
        parent[vertex])
            add = min(add, graph[parent[vertex]][vertex]);
        flow += add;
        for (int vertex = sink; vertex != source; vertex =
        parent[vertex]) {
            int previous = parent[vertex];
            if ((graph[previous][vertex] -= add) == 0)
                graph[previous].erase(vertex);
            graph[vertex][previous] += add;
        }
    }
}
```

5.7 Minimum Cut

Usage: Run max flow first; returns the source side. **Time Complexity:** $O(V + E)$

```
// After max-flow, this returns vertices reachable from source
through
// positive residual-capacity edges. Requires Dinic from Max
Flow.
vector<int> minCutSide(const Dinic& flow, int source) {
    vector<int> seen(flow.graph.size()), stack = {source},
    result;
    seen[source] = true;
    while (!stack.empty()) {
        int vertex = stack.back();
        stack.pop_back();
        result.push_back(vertex);
        for (const auto& edge : flow.graph[vertex])
            if (edge.capacity > 0 && !seen[edge.to]) {
                seen[edge.to] = true;
                stack.push_back(edge.to);
```

```
            }
        }
    }
    return result;
}
```

5.8 Global Minimum Cut

Time Complexity: $O(V^3)$

```
// Stoer-Wagner for an undirected weighted adjacency matrix.
O(V^3).
// Returns {cut weight, one side of the cut}.
pair<ll, vector<int>> globalMinCut(vector<vector<ll>> matrix)
{
    int n = (int)matrix.size();
    if (!n) return {0, {}};
    pair<ll, vector<int>> best = {LLONG_MAX, {}};
    vector<vector<int>> groups(n);
    for (int i = 0; i < n; ++i) groups[i] = {i};
    vector<int> alive(n, true);
    for (int phase = 1; phase < n; ++phase) {
        vector<ll> weight(n);
        vector<int> used(n);
        int previous = -1, selected = -1;
        for (int iteration = 0; iteration < n - phase + 1;
        ++iteration) {
            selected = -1;
            for (int vertex = 0; vertex < n; ++vertex)
                if (alive[vertex] && !used[vertex]
                        && (selected == -1 || weight[vertex] >
                        weight[selected]))
                    selected = vertex;
            if (iteration == n - phase) {
                if (weight[selected] < best.first)
                    best = {weight[selected],
                    groups[selected]};
                if (previous == -1) break;
                for (int vertex = 0; vertex < n; ++vertex) {
                    matrix[previous][vertex] +=
                    matrix[selected][vertex];
                    matrix[vertex][previous] =
                    matrix[previous][vertex];
                }
                auto& destination = groups[previous];
                destination.insert(destination.end(),
                    groups[selected].begin(),
                    groups[selected].end());
                alive[selected] = false;
            } else {
                used[selected] = true;
                for (int vertex = 0; vertex < n; ++vertex)
                    if (alive[vertex] && !used[vertex])
                        weight[vertex] +=
                        matrix[selected][vertex];
                previous = selected;
            }
        }
    }
    return best;
}
```

5.9 Gomory–Hu Tree

Time Complexity: $O(V)$ max-flow computations

```
using GomoryHuEdge = array<ll, 3>; // {u,v,capacity}

// Requires PushRelabel. Undirected non-negative capacities.
O(V) max-flows.
vector<GomoryHuEdge> gomoryHu(int n, const
vector<GomoryHuEdge>& edges) {
    vector<GomoryHuEdge> tree;
    vector<int> parent(n);
    for (int vertex = 1; vertex < n; ++vertex) {
        PushRelabel flow(n);
        for (auto edge : edges)
            flow.addEdge((int)edge[0], (int)edge[1], edge[2],
            edge[2]);
        ll cut = flow.maxFlow(vertex, parent[vertex]);
        tree.push_back({vertex, parent[vertex], cut});
        for (int next = vertex + 1; next < n; ++next)
            if (parent[next] == parent[vertex]
                    && flow.sourceSideOfMinCut(next))
                parent[next] = vertex;
    }
    return tree;
}
```

5.10 Min-Cost Max-Flow

Time Complexity: $O(FE \log V + VE)$

```
struct MinCostMaxFlow {
    struct Edge { int to, reverse; ll capacity, cost, flow; };
    vector<vector<Edge>> graph;

    MinCostMaxFlow(int n) : graph(n) {}

    void addEdge(int from, int to, ll capacity, ll cost) {
        graph[from].push_back({to, (int)graph[to].size(),
        capacity, cost, 0});
        graph[to].push_back(
            {from, (int)graph[from].size() - 1, 0, -cost, 0});
    }

    // Assumes no reachable negative-cost cycle with residual
    capacity.
    // Returns as much flow as possible up to limit, and its
    minimum cost.
    pair<ll, ll> minCostFlow(int source, int sink, ll limit =
    LLONG_MAX) {
        int n = (int)graph.size();
        const ll INF = (1LL << 62);
        vector<ll> potential(n, INF);
        potential[source] = 0;
        for (int iteration = 0; iteration < n - 1;
        ++iteration) {
            bool changed = false;
            for (int vertex = 0; vertex < n; ++vertex)
                if (potential[vertex] != INF)
                    for (const Edge& edge : graph[vertex])
                        if (edge.capacity > edge.flow
                                && potential[edge.to] >
                                   potential[vertex] +
                                   edge.cost) {
                            potential[edge.to] = max(-INF,
                                potential[vertex] +
                                edge.cost);
                            changed = true;
                        }
            if (!changed) break;
        }
        for (ll& value : potential) if (value == INF) value =
        0;

        ll totalFlow = 0, totalCost = 0;
        vector<ll> distance(n);
        vector<int> parentVertex(n), parentEdge(n);
        while (totalFlow < limit) {
            fill(distance.begin(), distance.end(), INF);
            distance[source] = 0;
            priority_queue<pair<ll, int>,
                vector<pair<ll, int>>, greater<pair<ll, int>>>
                queue;
            queue.push({0, source});
            while (!queue.empty()) {
                ll currentDistance = queue.top().first;
                int vertex = queue.top().second;
                queue.pop();
                if (currentDistance != distance[vertex])
                continue;
                for (int i = 0; i < (int)graph[vertex].size();
                ++i) {
                    Edge& edge = graph[vertex][i];
                    if (edge.flow == edge.capacity) continue;
                    ll candidate = currentDistance + edge.cost
                                + potential[vertex] -
                                potential[edge.to];
                    if (candidate < distance[edge.to]) {
                        distance[edge.to] = candidate;
                        parentVertex[edge.to] = vertex;
                        parentEdge[edge.to] = i;
                        queue.push({candidate, edge.to});
                    }
                }
            }
            if (distance[sink] == INF) break;
```

```
            for (int vertex = 0; vertex < n; ++vertex)
                if (distance[vertex] != INF) potential[vertex]
                += distance[vertex];
            ll add = limit - totalFlow;
            for (int vertex = sink; vertex != source;
                    vertex = parentVertex[vertex]) {
                int previous = parentVertex[vertex];
                Edge& edge =
                graph[previous][parentEdge[vertex]];
                add = min(add, edge.capacity - edge.flow);
            }
            for (int vertex = sink; vertex != source;
                    vertex = parentVertex[vertex]) {
                int previous = parentVertex[vertex];
                Edge& edge =
                graph[previous][parentEdge[vertex]];
                edge.flow += add;
                graph[edge.to][edge.reverse].flow -= add;
                totalCost += add * edge.cost;
            }
            totalFlow += add;
        }
        return {totalFlow, totalCost};
    }
};
```

5.11 Flow With Lower Bounds

Time Complexity: $O(V^2E)$

```
struct FlowWithDemand {
    struct BoundedEdge { int from, to; ll lower, upper; };
    int n, source, sink;
    vector<BoundedEdge> edges;

    FlowWithDemand(int n, int source, int sink)
        : n(n), source(source), sink(sink) {}

    void addEdge(int from, int to, ll lower, ll upper) {
        assert(0 <= lower && lower <= upper);
        edges.push_back({from, to, lower, upper});
    }

    // Whether a feasible s-t flow exists with at most
    backCapacity total flow.
    // Use LLONG_MAX/4 for an unrestricted feasible
    flow/circulation.
    bool feasible(ll backCapacity = LLONG_MAX / 4) const {
        int superSource = n, superSink = n + 1;
        Dinic flow(n + 2);
        vector<ll> balance(n);
        for (auto edge : edges) {
            flow.addEdge(edge.from, edge.to, edge.upper -
            edge.lower);
            balance[edge.from] -= edge.lower;
            balance[edge.to] += edge.lower;
        }
        flow.addEdge(sink, source, backCapacity);
        ll required = 0;
        for (int vertex = 0; vertex < n; ++vertex) {
            if (balance[vertex] > 0) {
                flow.addEdge(superSource, vertex,
                balance[vertex]);
                required += balance[vertex];
            } else if (balance[vertex] < 0) {
                flow.addEdge(vertex, superSink,
                -balance[vertex]);
            }
        }
        return flow.maxFlow(superSource, superSink) ==
        required;
    }
};
```

5.12 Hopcroft–Karp

Time Complexity: $O(E\sqrt{V})$

```
struct HopcroftKarp {
    int leftSize, rightSize;
    vector<vector<int>> graph;
    vector<int> leftMatch, rightMatch, distance;

    HopcroftKarp(int leftSize, int rightSize)
        : leftSize(leftSize), rightSize(rightSize),
        graph(leftSize),
          leftMatch(leftSize, -1), rightMatch(rightSize, -1),
          distance(leftSize) {}

    void addEdge(int left, int right) {
    graph[left].push_back(right); }

    bool bfs() {
        queue<int> queue;
        fill(distance.begin(), distance.end(), -1);
        for (int left = 0; left < leftSize; ++left)
            if (leftMatch[left] == -1)
                distance[left] = 0, queue.push(left);
        bool found = false;
        while (!queue.empty()) {
            int left = queue.front();
            queue.pop();
            for (int right : graph[left]) {
                int next = rightMatch[right];
                if (next == -1) found = true;
                else if (distance[next] == -1)
                    distance[next] = distance[left] + 1,
                    queue.push(next);
            }
        }
        return found;
    }

    bool dfs(int left) {
        for (int right : graph[left]) {
            int next = rightMatch[right];
            if (next == -1
                    || (distance[next] == distance[left] + 1
                    && dfs(next))) {
                leftMatch[left] = right;
                rightMatch[right] = left;
                return true;
            }
        }
        distance[left] = -1;
        return false;
    }

    int maxMatching() {
        int result = 0;
        while (bfs())
            for (int left = 0; left < leftSize; ++left)
                if (leftMatch[left] == -1 && dfs(left))
                ++result;
        return result;
    }
};
```

5.13 DFS Bipartite Matching

Time Complexity: $O(VE)$

```
// Randomized phase-based Kuhn for a bipartite graph.
// graph[u] contains right-side neighbors of left vertex u.
// Each phase visits every left vertex at most once; shuffling
avoids bad orders.
// Returns a maximum matching in rightToLeft. Worst case:
O(VE).
int dfsMatching(vector<vector<int>> graph, vector<int>&
rightToLeft) {
    int n = (int)graph.size(), m = (int)rightToLeft.size();
    vector<int> leftToRight(n, -1), seen(n), order(n);
    iota(order.begin(), order.end(), 0);
    fill(rightToLeft.begin(), rightToLeft.end(), -1);

    unsigned seed = (unsigned)chrono::steady_clock::now()
        .time_since_epoch().count();
    static mt19937 rng(seed);
    shuffle(order.begin(), order.end(), rng);
    for (auto& edges : graph) shuffle(edges.begin(),
    edges.end(), rng);

    int matching = 0, timer = 0;
    function<bool(int)> augment = [&](int u) {
        if (seen[u] == timer) return false;
        seen[u] = timer;
        for (int v : graph[u]) {
```

```
            assert(0 <= v && v < m);
            if (rightToLeft[v] == -1 ||
            augment(rightToLeft[v])) {
                leftToRight[u] = v;
                rightToLeft[v] = u;
                return true;
            }
        }
        return false;
    };

    bool improved = true;
    while (improved) {
        improved = false;
        ++timer;
        for (int u : order) {
            if (leftToRight[u] == -1 && augment(u)) {
                ++matching;
                improved = true;
            }
        }
    }
    return matching;
}
```

5.14 Bipartite Minimum Vertex Cover

Time Complexity: $O(VE)$

```
// Bipartite minimum vertex cover. Returned ids are L: [0,n),
R: [n,n+m).
// Requires dfsMatching() above. O(VE).
vector<int> minimumVertexCover(const vector<vector<int>>&
graph, int rightSize) {
    int leftSize = (int)graph.size();
    vector<int> rightToLeft(rightSize, -1);
    int matchingSize = dfsMatching(graph, rightToLeft);
    vector<int> reachableLeft(leftSize, true),
    reachableRight(rightSize);
    for (int left : rightToLeft)
        if (left != -1) reachableLeft[left] = false;
    vector<int> stack;
    for (int left = 0; left < leftSize; ++left)
        if (reachableLeft[left]) stack.push_back(left);
    while (!stack.empty()) {
        int left = stack.back();
        stack.pop_back();
        reachableLeft[left] = true;
        for (int right : graph[left])
            if (!reachableRight[right] && rightToLeft[right]
            != -1) {
                reachableRight[right] = true;
                stack.push_back(rightToLeft[right]);
            }
    }
    vector<int> cover;
    for (int left = 0; left < leftSize; ++left)
        if (!reachableLeft[left]) cover.push_back(left);
    for (int right = 0; right < rightSize; ++right)
        if (reachableRight[right]) cover.push_back(leftSize +
        right);
    assert((int)cover.size() == matchingSize);
    return cover;
}
```

5.15 Weighted Bipartite Matching

Time Complexity: $O(N^2M)$

```
// Minimum-cost matching of every row to a distinct column;
rows <= columns.
// Negate costs for maximum weight. O(rows^2 * columns).
template<class T>
pair<T, vector<int>> weightedMatching(const vector<vector<T>>&
costMatrix) {
    int rows = (int)costMatrix.size();
    int columns = rows ? (int)costMatrix[0].size() : 0;
    assert(rows <= columns);
    vector<T> distance(columns), potential(columns);
    vector<int> match(rows), reverse(columns, -1),
    order(columns), previous(columns);
    T total = 0;
    for (int row = rows - 1; row >= 0; --row) {
        for (int column = 0; column < columns; ++column) {
            distance[column] = costMatrix[row][column];
            order[column] = column;
            previous[column] = row;
        }
        int used = 0, freeColumn = -1, matchedRow = -1;
        T delta = 0;
        while (true) {
            for (int j = used; j < columns; ++j) {
                int column = order[j];
                T reduced = distance[column] -
                potential[column];
                if (j == used || delta > reduced) {
                    delta = reduced;
                    swap(order[used], order[j]);
                }
            }
            freeColumn = order[used++];
            matchedRow = reverse[freeColumn];
            if (matchedRow == -1) break;
            for (int column = 0; column < columns; ++column) {
                T candidate = costMatrix[matchedRow][column]
                            -
                            costMatrix[matchedRow][freeColumn]
                            + distance[freeColumn];
                if (candidate < distance[column]) {
                    distance[column] = candidate;
                    previous[column] = matchedRow;
                }
            }
        }
        total += distance[freeColumn];
        while (used--)
            potential[order[used]] = distance[order[used]] -
            delta;
        for (; matchedRow != row; swap(freeColumn,
        match[matchedRow]))
            matchedRow = reverse[freeColumn] =
            previous[freeColumn];
    }
    return {total, match};
}
```

5.16 General Matching (Blossom)

Time Complexity: $O(V^3)$

```
// Edmonds' blossom algorithm for an undirected graph. O(n^3).
struct Blossom {
    int n;
    vector<vector<int>> graph;
    vector<int> match, parent, base, queue;
    vector<bool> used, inBlossom;

    Blossom(int n) : n(n), graph(n), match(n, -1), parent(n),
        base(n), queue(n), used(n), inBlossom(n) {}

    void addEdge(int a, int b) {
        assert(a != b);
        graph[a].push_back(b);
        graph[b].push_back(a);
    }

    int lca(int a, int b) {
        vector<bool> seen(n);
        while (true) {
            a = base[a];
            seen[a] = true;
            if (match[a] == -1) break;
            a = parent[match[a]];
        }
        while (true) {
            b = base[b];
            if (seen[b]) return b;
            b = parent[match[b]];
        }
    }

    void markPath(int vertex, int root, int child) {
        while (base[vertex] != root) {
            inBlossom[base[vertex]] =
            inBlossom[base[match[vertex]]] = true;
            parent[vertex] = child;
            child = match[vertex];
```

```
            vertex = parent[match[vertex]];
        }
    }

    int findPath(int root) {
        fill(used.begin(), used.end(), false);
        fill(parent.begin(), parent.end(), -1);
        iota(base.begin(), base.end(), 0);
        int head = 0, tail = 0;
        queue[tail++] = root;
        used[root] = true;
        while (head < tail) {
            int vertex = queue[head++];
            for (int to : graph[vertex]) {
                if (base[vertex] == base[to] || match[vertex]
                == to) continue;
                if (to == root || (match[to] != -1
                        && parent[match[to]] != -1)) {
                    int rootBase = lca(vertex, to);
                    fill(inBlossom.begin(), inBlossom.end(),
                    false);
                    markPath(vertex, rootBase, to);
                    markPath(to, rootBase, vertex);
                    for (int i = 0; i < n; ++i)
                        if (inBlossom[base[i]]) {
                            base[i] = rootBase;
                            if (!used[i]) used[i] = true,
                            queue[tail++] = i;
                        }
                } else if (parent[to] == -1) {
                    parent[to] = vertex;
                    if (match[to] == -1) return to;
                    to = match[to];
                    used[to] = true;
                    queue[tail++] = to;
                }
            }
        }
        return -1;
    }

    int maxMatching() {
        int result = 0;
        for (int vertex = 0; vertex < n; ++vertex) if
        (match[vertex] == -1) {
            int endpoint = findPath(vertex);
            if (endpoint == -1) continue;
            ++result;
            while (endpoint != -1) {
                int previous = parent[endpoint];
                int next = previous == -1 ? -1 :
                match[previous];
                if (previous != -1)
                    match[endpoint] = previous,
                    match[previous] = endpoint;
                endpoint = next;
            }
        }
        return result;
    }
};
```

5.17 Strongly Connected Components

Time Complexity: $O(V + E)$

```
// Tarjan SCC. component ids are in reverse topological order.
O(V+E).
struct SCC {
    vector<vector<int>> graph;
    vector<int> discovery, low, component, stack;
    vector<bool> onStack;
    int timer = 0, componentCount = 0;

    SCC(const vector<vector<int>>& graph)
        : graph(graph), discovery(graph.size(), -1),
        low(graph.size()),
          component(graph.size(), -1), onStack(graph.size()) {
        for (int vertex = 0; vertex < (int)graph.size();
        ++vertex)
            if (discovery[vertex] == -1) dfs(vertex);
    }

    void dfs(int vertex) {
        discovery[vertex] = low[vertex] = timer++;
        stack.push_back(vertex);
        onStack[vertex] = true;
        for (int to : graph[vertex]) {
            if (discovery[to] == -1) {
                dfs(to);
                low[vertex] = min(low[vertex], low[to]);
            } else if (onStack[to]) {
                low[vertex] = min(low[vertex], discovery[to]);
            }
        }
        if (low[vertex] != discovery[vertex]) return;
        while (true) {
            int top = stack.back();
            stack.pop_back();
            onStack[top] = false;
            component[top] = componentCount;
            if (top == vertex) break;
        }
        ++componentCount;
    }
};
```

5.18 Block-Cut Tree

Time Complexity: $O(V + E)$

```
// Vertex-biconnected components and their block-cut forest.
O(V+E).
struct BlockCutTree {
    int n, timer = 0;
    vector<vector<pair<int, int>>> graph;
    vector<int> tin, low, edgeStack, articulation, position;
    vector<pair<int, int>> edges;
    vector<vector<int>> blocks, tree;

    BlockCutTree(int n) : n(n), graph(n), tin(n, -1), low(n),
                          articulation(n), position(n, -1) {}

    void addEdge(int a, int b) {
        int id = (int)edges.size();
        edges.push_back({a, b});
        graph[a].push_back({b, id});
        graph[b].push_back({a, id});
    }

    void dfs(int vertex, int parentEdge) {
        tin[vertex] = low[vertex] = timer++;
        int children = 0;
        for (auto item : graph[vertex]) {
            int to = item.first, edge = item.second;
            if (edge == parentEdge) continue;
            if (tin[to] == -1) {
                edgeStack.push_back(edge);
                ++children;
                dfs(to, edge);
                low[vertex] = min(low[vertex], low[to]);
                if (low[to] >= tin[vertex]) {
                    if (parentEdge != -1 || children > 1)
                        articulation[vertex] = true;
                    vector<int> block;
                    while (true) {
                        int id = edgeStack.back();
                        edgeStack.pop_back();
                        block.push_back(edges[id].first);
                        block.push_back(edges[id].second);
                        if (id == edge) break;
                    }
                    sort(block.begin(), block.end());
                    block.erase(unique(block.begin(),
                    block.end()), block.end());
                    blocks.push_back(move(block));
                }
            } else if (tin[to] < tin[vertex]) {
                edgeStack.push_back(edge);
                low[vertex] = min(low[vertex], tin[to]);
            }
        }
        if (parentEdge == -1 && children <= 1)
        articulation[vertex] = false;
    }
```

```
    // Articulation nodes come first, followed by component
    nodes.
    void build() {
        for (int vertex = 0; vertex < n; ++vertex) if
        (tin[vertex] == -1) {
            if (graph[vertex].empty())
            blocks.push_back({vertex});
            else dfs(vertex, -1);
        }
        int nodes = 0;
        for (int vertex = 0; vertex < n; ++vertex)
            if (articulation[vertex]) position[vertex] =
            nodes++;
        int blockStart = nodes;
        nodes += blocks.size();
        tree.assign(nodes, {});
        for (int block = 0; block < (int)blocks.size();
        ++block)
            for (int vertex : blocks[block]) {
                int blockNode = blockStart + block;
                if (articulation[vertex]) {
                    int jointNode = position[vertex];
                    tree[jointNode].push_back(blockNode);
                    tree[blockNode].push_back(jointNode);
                } else {
                    position[vertex] = blockNode;
                }
            }
    }
};
```

5.19 Two Sat

Time Complexity: $O(V + E)$

```
/*
Usage:
 - TwoSAT(n): initialize a graph of size n
 - addDisjunction(u, v): u and v = true
 - solve(): check if it's correct
*/

struct TwoSAT {
    int n, time = 0, sccCount = 0;
    vector<vector<int>> adj;
    vector<int> low, ids, scc;
    vector<bool> onStack;
    vector<int> assignment, st;

    TwoSAT(int n) : n(n) {
        adj.resize(2 * n);
        low.assign(2 * n, -1);
        ids.assign(2 * n, -1);
        scc.assign(2 * n, -1);
        onStack.assign(2 * n, false);
        assignment.resize(n);
        st.reserve(2 * n);
    }

    inline void addDisjunction(int u, int v) {
        adj[u ^ 1].push_back(v);
        adj[v ^ 1].push_back(u);
    }

    void tarjanDFS(int v) {
        ids[v] = low[v] = time++;
        st.push_back(v);
        onStack[v] = true;

        for (int u : adj[v]) {
            if (ids[u] == -1)
                tarjanDFS(u);
            if (onStack[u])
                low[v] = min(low[v], low[u]);
        }

        if (ids[v] == low[v]) {
            while (true) {
                int u = st.back();
                st.pop_back();
                onStack[u] = false;
                scc[u] = sccCount;
                if (u == v) break;
            }
            ++sccCount;
        }
    }

    bool solve() {
        for (int i = 0; i < 2 * n; ++i)
            if (ids[i] == -1)
                tarjanDFS(i);
         for (int i = 0; i < n; ++i) {
            if (scc[2 * i] == scc[2 * i + 1])
                return false;
            assignment[i] = (scc[2 * i] < scc[2 * i + 1]);
        }
        return true;
    }
};
```

5.20 Euler Walk

Time Complexity: $O(V + E)$

```
// Adjacency entries are {destination, global edge id}; both
directions of an
// undirected edge share an id. Empty means no trail from
source. O(V+E).
vector<int> eulerWalk(const vector<vector<pair<int, int>>>&
graph,
        int edgeCount, int source = 0) {
    int n = (int)graph.size();
    vector<int> balance(n), iterator(n), used(edgeCount),
    result;
    vector<int> stack = {source};
    ++balance[source];
    while (!stack.empty()) {
        int vertex = stack.back();
        int& index = iterator[vertex];
        while (index < (int)graph[vertex].size()
                && used[graph[vertex][index].second]) ++index;
        if (index == (int)graph[vertex].size()) {
            result.push_back(vertex);
            stack.pop_back();
        } else {
            int to = graph[vertex][index].first;
            int edge = graph[vertex][index++].second;
            if (!used[edge]) {
                --balance[vertex];
                ++balance[to];
                used[edge] = true;
                stack.push_back(to);
            }
        }
    }
    for (int value : balance)
        if (value < 0) return {};
    if ((int)result.size() != edgeCount + 1) return {};
    reverse(result.begin(), result.end());
    return result;
}
```

5.21 Edge Coloring

Time Complexity: $O(VE)$

```
// (Delta+1)-colors a simple undirected graph. O(VE).
// Returns the color of each input edge, numbered from 0.
vector<int> edgeColoring(int n, const vector<pair<int, int>>&
edges) {
    vector<int> colorsAtVertex(n + 1), result(edges.size()),
    fan(n), freeColor(n);
    for (auto edge : edges)
        ++colorsAtVertex[edge.first],
        ++colorsAtVertex[edge.second];
    int colorCount = *max_element(colorsAtVertex.begin(),
    colorsAtVertex.end()) + 1;
    vector<vector<int>> adjacent(n, vector<int>(colorCount,
    -1));
    for (auto edge : edges) {
        int u = edge.first, v = edge.second;
        fan[0] = v;
        vector<int> location(colorCount);
        int at = u, end = u, color = freeColor[u], index = 0,
        i = 0;
        int d;
```

```
        while ((d = freeColor[v]), !location[d]
                && (v = adjacent[u][d]) != -1) {
            location[d] = ++index;
            colorsAtVertex[index] = d;
            fan[index] = v;
        }
        colorsAtVertex[location[d]] = color;
        for (int current = d; at != -1;
                current ^= color ^ d, at =
                adjacent[at][current])
            swap(adjacent[at][current],
                 adjacent[end = at][current ^ color ^ d]);
        while (adjacent[fan[i]][d] != -1) {
            int left = fan[i], right = fan[++i], other =
            colorsAtVertex[i];
            adjacent[u][other] = left;
            adjacent[left][other] = u;
            adjacent[right][other] = -1;
            freeColor[right] = other;
        }
        adjacent[u][d] = fan[i];
        adjacent[fan[i]][d] = u;
        for (int vertex : {fan[0], u, end}) {
            freeColor[vertex] = 0;
            while (adjacent[vertex][freeColor[vertex]] != -1)
                ++freeColor[vertex];
        }
    }
    for (int i = 0; i < (int)edges.size(); ++i) {
        int u = edges[i].first, v = edges[i].second;
        while (adjacent[u][result[i]] != v) ++result[i];
    }
    return result;
}
```

5.22 Enumerate Maximal Cliques

Time Complexity: $O(3^{V/3})$

```
using CliqueMask = bitset<128>;

template<class F>
void maximalCliqueDfs(const vector<CliqueMask>& adjacency, F&
callback,
        CliqueMask candidates, CliqueMask excluded, CliqueMask
        current) {
    if (!candidates.any()) {
        if (!excluded.any()) callback(current);
        return;
    }
    int pivot = (int)(candidates | excluded)._Find_first();
    CliqueMask branch = candidates & ~adjacency[pivot];
    for (int vertex = 0; vertex < (int)adjacency.size();
    ++vertex)
        if (branch[vertex]) {
            current[vertex] = true;
            maximalCliqueDfs(adjacency, callback,
                candidates & adjacency[vertex],
                excluded & adjacency[vertex], current);
            current[vertex] = false;
            candidates[vertex] = false;
            excluded[vertex] = true;
        }
}

// Calls callback once for every maximal clique. n <= 128,
O(3^(n/3)).
template<class F>
void enumerateMaximalCliques(const vector<CliqueMask>&
adjacency, F callback) {
    assert(adjacency.size() <= 128);
    CliqueMask candidates;
    for (int i = 0; i < (int)adjacency.size(); ++i)
    candidates[i] = true;
    maximalCliqueDfs(adjacency, callback, candidates, {}, {});
}
```

5.23 Maximum Clique / Independent Set

Usage: Supports at most 200 vertices. **Time Complexity:** Exponential, fast in practice

```
// Branch-and-bound with greedy coloring. adjacency is
symmetric; n <= 200.
struct MaximumClique {
    using Mask = bitset<200>;
    vector<Mask> adjacency;
    vector<int> best, current;

    MaximumClique(vector<Mask> graph) : adjacency(move(graph))
    {
        assert(adjacency.size() <= 200);
    }

    void colorSort(const vector<int>& candidates,
            vector<int>& order, vector<int>& bound) {
        vector<Mask> colorClass;
        for (int vertex : candidates) {
            int color = 0;
            while (color < (int)colorClass.size()
                    && (colorClass[color] &
                    adjacency[vertex]).any())
                ++color;
            if (color == (int)colorClass.size())
            colorClass.push_back({});
            colorClass[color][vertex] = true;
        }
        for (int color = 0; color < (int)colorClass.size();
        ++color)
            for (int vertex : candidates)
                if (colorClass[color][vertex]) {
                    order.push_back(vertex);
                    bound.push_back(color + 1);
                }
    }

    void expand(const vector<int>& candidates) {
        vector<int> order, bound;
        colorSort(candidates, order, bound);
        for (int i = (int)order.size() - 1; i >= 0; --i) {
            if ((int)current.size() + bound[i] <=
            (int)best.size()) return;
            int vertex = order[i];
            current.push_back(vertex);
            vector<int> next;
            for (int j = 0; j < i; ++j)
                if (adjacency[vertex][order[j]])
                next.push_back(order[j]);
            if (next.empty()) {
                if (current.size() > best.size()) best =
                current;
            } else {
                expand(next);
            }
            current.pop_back();
        }
    }

    vector<int> solve() {
        vector<int> vertices(adjacency.size());
        iota(vertices.begin(), vertices.end(), 0);
        sort(vertices.begin(), vertices.end(), [&](int a, int
        b) {
            return adjacency[a].count() >
            adjacency[b].count();
        });
        expand(vertices);
        return best;
    }
};

vector<int> maximumIndependentSet(vector<bitset<200>>
adjacency) {
    int n = (int)adjacency.size();
    for (int i = 0; i < n; ++i) {
        adjacency[i].flip();
        adjacency[i][i] = false;
        for (int j = n; j < 200; ++j) adjacency[i][j] = false;
    }
    return MaximumClique(move(adjacency)).solve();
}
```

5.24 Binary Lifting

Time Complexity: $O(N \log N)$ build, $O(\log N)$ query

```
vector<vector<int>> buildJumpTable(const vector<int>& parent)
{
    int levels = 1;
    while ((1LL << levels) <= (int)parent.size()) ++levels;
    vector<vector<int>> jump(levels, parent);
    for (int level = 1; level < levels; ++level)
        for (int vertex = 0; vertex < (int)parent.size();
        ++vertex)
            jump[level][vertex] = jump[level - 1][jump[level -
            1][vertex]];
    return jump;
}

int jumpUp(const vector<vector<int>>& jump, int vertex, int
distance) {
    for (int bit = 0; bit < (int)jump.size(); ++bit)
        if (distance & (1LL << bit)) vertex =
        jump[bit][vertex];
    return vertex;
}

int liftingLca(const vector<vector<int>>& jump,
        const vector<int>& depth, int a, int b) {
    if (depth[a] < depth[b]) swap(a, b);
    a = jumpUp(jump, a, depth[a] - depth[b]);
    if (a == b) return a;
    for (int level = (int)jump.size() - 1; level >= 0;
    --level)
        if (jump[level][a] != jump[level][b])
            a = jump[level][a], b = jump[level][b];
    return jump[0][a];
}
```

5.25 Lowest Common Ancestor

Time Complexity: $O(N \log N)$ build, $O(\log N)$ query

```
struct LCA {
    int timer = 0;
    vector<int> parent, depth, tin, tout;
    vector<vector<int>> jump;

    LCA(const vector<vector<int>>& tree, int root = 0)
        : parent(tree.size(), root), depth(tree.size()),
          tin(tree.size()), tout(tree.size()) {
        int n = (int)tree.size();
        vector<int> iterator(n), stack = {root};
        parent[root] = root;
        while (!stack.empty()) {
            int vertex = stack.back();
            if (!iterator[vertex]) tin[vertex] = timer++;
            if (iterator[vertex] == (int)tree[vertex].size())
            {
                tout[vertex] = timer;
                stack.pop_back();
                continue;
            }
            int to = tree[vertex][iterator[vertex]++];
            if (to == parent[vertex]) continue;
            parent[to] = vertex;
            depth[to] = depth[vertex] + 1;
            stack.push_back(to);
        }
        jump = buildJumpTable(parent);
    }

    bool ancestor(int a, int b) const {
        return tin[a] <= tin[b] && tout[b] <= tout[a];
    }

    int lca(int a, int b) const {
        if (ancestor(a, b)) return a;
        if (ancestor(b, a)) return b;
        for (int level = (int)jump.size() - 1; level >= 0;
        --level)
            if (!ancestor(jump[level][a], b)) a =
            jump[level][a];
        return parent[a];
    }

    int distance(int a, int b) const {
        int c = lca(a, b);
        return depth[a] + depth[b] - 2 * depth[c];
    }
};
```

5.26 Virtual Tree

Time Complexity: $O(K \log K)$

```
// Returns {(parent index in result, original vertex)}; root
points to itself.
// Requires LCA. Empty subset returns empty. O(k log k).
vector<pair<int, int>> compressTree(const LCA& lca,
vector<int> vertices) {
    if (vertices.empty()) return {};
    auto byTin = [&](int a, int b) { return lca.tin[a] <
    lca.tin[b]; };
    sort(vertices.begin(), vertices.end(), byTin);
    int originalSize = (int)vertices.size();
    for (int i = 0; i + 1 < originalSize; ++i)
        vertices.push_back(lca.lca(vertices[i], vertices[i +
        1]));
    sort(vertices.begin(), vertices.end(), byTin);
    vertices.erase(unique(vertices.begin(), vertices.end()),
    vertices.end());
    vector<pair<int, int>> result;
    vector<int> stack, index(lca.parent.size(), -1);
    for (int vertex : vertices) {
        while (!stack.empty() && !lca.ancestor(stack.back(),
        vertex))
            stack.pop_back();
        int parentIndex = stack.empty() ? (int)result.size()
                                        : index[stack.back()];
        index[vertex] = (int)result.size();
        result.push_back({parentIndex, vertex});
        stack.push_back(vertex);
    }
    return result;
}
```

5.27 Heavy-Light Decomposition

Time Complexity: $O(\log N)$ intervals per path

```
// Heavy-light decomposition. processPath emits half-open
position intervals.
struct HLD {
    int timer = 0;
    vector<vector<int>> tree;
    vector<int> parent, depth, size, head, position;

    HLD(vector<vector<int>> graph, int root = 0)
        : tree(move(graph)), parent(tree.size(), -1),
        depth(tree.size()),
          size(tree.size(), 1), head(tree.size()),
          position(tree.size()) {
        dfsSize(root, -1);
        head[root] = root;
        dfsHld(root, -1);
    }

    void dfsSize(int vertex, int previous) {
        parent[vertex] = previous;
        int heavy = -1;
        for (int to : tree[vertex]) if (to != previous) {
            depth[to] = depth[vertex] + 1;
            dfsSize(to, vertex);
            size[vertex] += size[to];
            if (heavy == -1 || size[to] > size[heavy]) heavy =
            to;
        }
        if (heavy != -1) {
            auto it = find(tree[vertex].begin(),
            tree[vertex].end(), heavy);
            swap(tree[vertex][0], *it);
        }
    }

    void dfsHld(int vertex, int previous) {
        position[vertex] = timer++;
        bool firstChild = true;
        for (int to : tree[vertex]) if (to != previous) {
            head[to] = firstChild ? head[vertex] : to;
            firstChild = false;
            dfsHld(to, vertex);
        }
```

```
    }

    template<class F>
    void processPath(int a, int b, F operation, bool
    valuesOnEdges = false) const {
        while (head[a] != head[b]) {
            if (depth[head[a]] < depth[head[b]]) swap(a, b);
            operation(position[head[a]], position[a] + 1);
            a = parent[head[a]];
        }
        if (depth[a] > depth[b]) swap(a, b);
        operation(position[a] + valuesOnEdges, position[b] +
        1);
    }

    pair<int, int> subtreeInterval(int vertex, bool
    valuesOnEdges = false) const {
        return {position[vertex] + valuesOnEdges,
                position[vertex] + size[vertex]};
    }
};
```

5.28 Link-Cut Tree

Time Complexity: Amortized $O(\log N)$

```
struct LinkCutNode {
    LinkCutNode *parent = nullptr, *pathParent = nullptr,
    *child[2];
    bool flipped = false;

    LinkCutNode() { child[0] = child[1] = nullptr; fix(); }

    void fix() {
        if (child[0]) child[0]->parent = this;
        if (child[1]) child[1]->parent = this;
        // Update path aggregates here if needed.
    }

    void push() {
        if (!flipped) return;
        flipped = false;
        swap(child[0], child[1]);
        if (child[0]) child[0]->flipped ^= true;
        if (child[1]) child[1]->flipped ^= true;
    }

    int direction() const {
        return parent ? (parent->child[1] == this) : -1;
    }

    void rotate(int side, int type) {
        int diagonal = side ^ type;
        LinkCutNode* x = child[side];
        LinkCutNode* y = (type == 2 ? x : x->child[diagonal]);
        LinkCutNode* z = type ? y : x;
        y->parent = parent;
        if (parent) parent->child[direction()] = y;
        child[side] = z->child[side ^ 1];
        if (type < 2) {
            x->child[diagonal] = y->child[diagonal ^ 1];
            y->child[diagonal ^ 1] = x;
        }
        z->child[side ^ 1] = this;
        fix(); x->fix(); y->fix();
        if (parent) parent->fix();
        swap(pathParent, y->pathParent);
    }

    void splay() {
        push();
        while (parent) {
            if (parent->parent) parent->parent->push();
            parent->push();
            push();
            int first = direction(), second =
            parent->direction();
            if (second == -1) parent->rotate(first, 2);
            else parent->parent->rotate(second, first !=
            second);
        }
    }
    LinkCutNode* first() {
        push();
        return child[0] ? child[0]->first() : (splay(), this);
    }
};

struct LinkCutTree {
    vector<LinkCutNode> node;
    LinkCutTree(int n) : node(n) {}

    LinkCutNode* access(LinkCutNode* vertex) {
        vertex->splay();
        while (LinkCutNode* upper = vertex->pathParent) {
            upper->splay();
            vertex->pathParent = nullptr;
            if (upper->child[1]) {
                upper->child[1]->parent = nullptr;
                upper->child[1]->pathParent = upper;
            }
            upper->child[1] = vertex;
            upper->fix();
            vertex = upper;
        }
        return vertex;
    }

    void makeRoot(LinkCutNode* vertex) {
        access(vertex);
        vertex->splay();
        if (vertex->child[0]) {
            vertex->child[0]->parent = nullptr;
            vertex->child[0]->flipped ^= true;
            vertex->child[0]->pathParent = vertex;
            vertex->child[0] = nullptr;
            vertex->fix();
        }
    }

    bool connected(int a, int b) {
        if (a == b) return true;
        LinkCutNode* rootA = access(&node[a])->first();
        LinkCutNode* rootB = access(&node[b])->first();
        return rootA == rootB;
    }

    void link(int a, int b) {
        assert(!connected(a, b));
        makeRoot(&node[a]);
        node[a].pathParent = &node[b];
    }

    void cut(int a, int b) {
        LinkCutNode* first = &node[a];
        LinkCutNode* second = &node[b];
        makeRoot(second);
        first->splay();
        LinkCutNode* upper = first->pathParent
            ? first->pathParent : first->child[0];
        assert(upper == second);
        if (first->pathParent) {
            first->pathParent = nullptr;
        } else {
            first->child[0] = nullptr;
            second->parent = nullptr;
            first->fix();
        }
    }
};
```

5.29 Directed MST

Time Complexity: $O(E \log V)$

```
struct DMSTEdge { int from, to; ll weight; };

struct RollbackDSU {
    vector<int> data;
    vector<pair<int, int>> history;
    RollbackDSU(int n) : data(n, -1) {}
    int find(int x) { return data[x] < 0 ? x : find(data[x]);
    }
    int time() const { return (int)history.size(); }
    void rollback(int time) {
```

```
        while ((int)history.size() > time) {
            auto change = history.back();
            history.pop_back();
            data[change.first] = change.second;
        }
    }
    bool join(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (data[a] > data[b]) swap(a, b);
        history.push_back({a, data[a]});
        history.push_back({b, data[b]});
        data[a] += data[b];
        data[b] = a;
        return true;
    }
};

struct DMSTHeap {
    DMSTEdge edge;
    DMSTHeap *left = nullptr, *right = nullptr;
    ll lazy = 0;
    void push() {
        edge.weight += lazy;
        if (left) left->lazy += lazy;
        if (right) right->lazy += lazy;
        lazy = 0;
    }
};

DMSTHeap* mergeDMST(DMSTHeap* a, DMSTHeap* b) {
    if (!a || !b) return a ? a : b;
    a->push(); b->push();
    if (a->edge.weight > b->edge.weight) swap(a, b);
    a->right = mergeDMST(b, a->right);
    swap(a->left, a->right);
    return a;
}

void popDMST(DMSTHeap*& heap) {
    heap->push();
    heap = mergeDMST(heap->left, heap->right);
}

// Minimum rooted arborescence. Returns {-1,{}} if some node
is unreachable.
// parent[root] is -1. O(E log V).
pair<ll, vector<int>> directedMST(
        int n, int root, const vector<DMSTEdge>& edges) {
    RollbackDSU dsu(n);
    vector<DMSTHeap*> heap(n);
    for (auto edge : edges)
        if (edge.from != edge.to)
            heap[edge.to] = mergeDMST(heap[edge.to],
                new DMSTHeap{edge, nullptr, nullptr, 0});
    ll answer = 0;
    vector<int> seen(n, -1), path(n), parent(n, -1);
    seen[root] = root;
    vector<DMSTEdge> queue(n), incoming(n, {-1, -1, 0});
    struct Cycle {
        int representative, rollbackTime;
        vector<DMSTEdge> edges;
    };
    deque<Cycle> cycles;
    for (int start = 0; start < n; ++start) {
        int vertex = start, queueSize = 0;
        while (seen[vertex] < 0) {
            while (heap[vertex]
                    && dsu.find(heap[vertex]->edge.from) ==
                    vertex)
                popDMST(heap[vertex]);
            if (!heap[vertex]) return {-1, {}};
            heap[vertex]->push();
            DMSTEdge edge = heap[vertex]->edge;
            heap[vertex]->lazy -= edge.weight;
            popDMST(heap[vertex]);
            queue[queueSize] = edge;
            path[queueSize++] = vertex;
            seen[vertex] = start;
            answer += edge.weight;
            vertex = dsu.find(edge.from);
            if (seen[vertex] == start) {
                DMSTHeap* cycleHeap = nullptr;
                int end = queueSize, rollbackTime =
                dsu.time(), member;
                do {
                    member = path[--queueSize];
                    cycleHeap = mergeDMST(cycleHeap,
                    heap[member]);
                } while (dsu.join(vertex, member));
                vertex = dsu.find(vertex);
                heap[vertex] = cycleHeap;
                seen[vertex] = -1;
                cycles.push_front({vertex, rollbackTime,
                    vector<DMSTEdge>(queue.begin() +
                    queueSize,
                                     queue.begin() + end)});
            }
        }
        for (int i = 0; i < queueSize; ++i)
            incoming[dsu.find(queue[i].to)] = queue[i];
    }
    for (const Cycle& cycle : cycles) {
        dsu.rollback(cycle.rollbackTime);
        DMSTEdge entering = incoming[cycle.representative];
        for (auto edge : cycle.edges)
            incoming[dsu.find(edge.to)] = edge;
        incoming[dsu.find(entering.to)] = entering;
    }
    for (int vertex = 0; vertex < n; ++vertex)
        if (vertex != root) parent[vertex] =
        incoming[vertex].from;
    return {answer, parent};
}
```

5.30 Dominator Tree

Time Complexity: $O((V+E)\log V)$

```
struct DominatorTree {
    int n, id;
    vector <vector <int>> g, rg, bucket;
    vector <int> sdom, dom, vis, rev, pa, rt, mn, res;
    // dom[s] = s, dom[v] = -1 if s -> v not exists
    int query(int v, int x) {
        if (rt[v] == v) return x ? -1 : v;
        int p = query(rt[v], 1);
        if (p == -1) return x ? rt[v] : mn[v];
        if (sdom[mn[v]] > sdom[mn[rt[v]]])
            mn[v] = mn[rt[v]];
        rt[v] = p;
        return x ? p : mn[v];
    }
    void dfs(int v) {
        vis[v] = id, rev[id] = v;
        rt[id] = mn[id] = sdom[id] = id, id++;
        for (int u : g[v]) {
            if (vis[u] == -1) dfs(u), pa[vis[u]] = vis[v];
            rg[vis[u]].push_back(vis[v]);
        }
    }
    void build(int s) {
        dfs(s);
        for (int i = id - 1; ~i; --i) {
            for (int u : rg[i])
                sdom[i] = min(sdom[i], sdom[query(u, 0)]);
            if (i) bucket[sdom[i]].push_back(i);
            for (int u : bucket[i]) {
                int p = query(u, 0);
                dom[u] = sdom[p] == i ? i : p;
            }
            if (i) rt[i] = pa[i];
        }
        fill(res.begin(), res.end(), -1);
        for (int i = 1; i < id; ++i) {
            if (dom[i] != sdom[i]) dom[i] = dom[dom[i]];
        }
        for (int i = 1; i < id; ++i)
            res[rev[i]] = rev[dom[i]];
        res[s] = s;
        for (int i = 0; i < n; ++i) dom[i] = res[i];
    }
    void addEdge(int u, int v) { g[u].push_back(v); }
    DominatorTree(int _n) : n(_n), id(0), g(n), rg(n),
```

```
        bucket(n), sdom(n), dom(n, -1), vis(n, -1),
        rev(n), pa(n), rt(n), mn(n), res(n) {
    }
};
```

5.31 Flow Model

- Maximum/Minimum flow with lower bound / Circulation problem
  1. Construct super source S and sink T.
  2. For each edge (x, y, l, u), connect $x \to y$ with capacity $u - l$.
  3. For each vertex v, denote by $in(v)$ the difference between the sum of **incoming** lower bounds and the sum of **outgoing** lower bounds.
  4. If $in(v) > 0$, connect $S \to v$ with capacity $in(v)$, otherwise, connect $v \to T$ with capacity $-in(v)$.
     - To maximize, connect $t \to s$ with capacity ∞ (skip this in circulation problem), and let f be the maximum flow from S to T. If $f \neq \sum_{v \in V, in(v)>0} in(v)$, there's no solution. Otherwise, the maximum flow from s to t is the answer.
     - To minimize, let f be the maximum flow from S to T. Connect $t \to s$ with capacity ∞ and let the flow from S to T be f'. If $f + f' \neq \sum_{v \in V, in(v)>0} in(v)$, there's no solution. Otherwise, f' is the answer.
  5. The solution of each edge e is $l_e + f_e$, where f_e corresponds to the flow of edge e on the graph.
- Construct minimum vertex cover from maximum matching M on bipartite graph (X, Y)
  1. Redirect every edge: $y \to x$ if $(x, y) \in M$, $x \to y$ otherwise.
  2. DFS from unmatched vertices in X.
  3. $x \in X$ is chosen iff x is unvisited.
  4. $y \in Y$ is chosen iff y is visited.
- Minimum cost cyclic flow
  1. Consruct super source S and sink T
  2. For each edge (x, y, c), connect $x \to y$ with $(cost, cap) = (c, 1)$ if $c > 0$, otherwise connect $y \to x$ with $(cost, cap) = (-c, 1)$
  3. For each edge with $c < 0$, sum these cost as K, then increase $d(y)$ by 1, decrease $d(x)$ by 1
  4. For each vertex v with $d(v) > 0$, connect $S \to v$ with $(cost, cap) = (0, d(v))$
  5. For each vertex v with $d(v) < 0$, connect $v \to T$ with $(cost, cap) = (0, -d(v))$
  6. Flow from S to T, the answer is the cost of the flow $C + K$
- Maximum density induced subgraph
  1. Binary search on answer, suppose we're checking answer T
  2. Construct a max flow model, let K be the sum of all weights
  3. Connect source $s \to v$, $v \in G$ with capacity K
  4. For each edge (u, v, w) in G, connect $u \to v$ and $v \to u$ with capacity w
  5. For $v \in G$, connect it with sink $v \to t$ with capacity $K + 2T - (\sum_{e \in E(v)} w(e)) - 2w(v)$
  6. T is a valid answer if the maximum flow $f < K|V|$
- Minimum weight edge cover
  1. Change the weight of each edge to $\mu(u) + \mu(v) - w(u, v)$, where $\mu(v)$ is the cost of the cheapest edge incident to v.
  2. Let the maximum weight matching of the graph be x, the answer will be $\sum \mu(v) - x$.

5.32 Graph Theorems

Matrix-tree theorem. Build the Laplacian L: for every edge $a \to b$, do $L_{ab} -= 1$, $L_{bb} += 1$ (also the reverse for an undirected edge). Delete row and column r; its determinant is the number of directed spanning trees rooted at r.

Erdős–Gallai. A non-increasing sequence $d_1, \ldots, d_n$ is the degree sequence of a simple graph iff its sum is even and, for every $1 \leq k \leq n$,

$$\sum_{i=1}^{k} d_i \leq k(k-1) + \sum_{i=k+1}^{n} \min(d_i, k).$$

6 String

6.1 Aho Corasick

```
// Multiple pattern matching over ['a', 'z']. Empty patterns
are unsupported.
struct AhoCorasick {
    struct Node {
        array<int, 26> next;
        int link = 0;
        vector<int> out;
        Node() { next.fill(-1); }
    };
    vector<Node> trie{1};

    int add(const string& s, int id) {
        assert(!s.empty());
        int v = 0;
        for (char ch : s) {
            int c = ch - 'a';
            assert(0 <= c && c < 26);
            if (trie[v].next[c] == -1) {
                trie[v].next[c] = (int)trie.size();
                trie.emplace_back();
            }
            v = trie[v].next[c];
        }
        trie[v].out.push_back(id);
        return v;
    }

    void build() {
        queue<int> q;
        for (int c = 0; c < 26; ++c) {
            int u = trie[0].next[c];
            if (u == -1) trie[0].next[c] = 0;
            else q.push(u);
        }
        while (!q.empty()) {
            int v = q.front(); q.pop();
            int f = trie[v].link;
            trie[v].out.insert(trie[v].out.end(),
            trie[f].out.begin(), trie[f].out.end());
            for (int c = 0; c < 26; ++c) {
                int u = trie[v].next[c];
                if (u == -1) trie[v].next[c] =
                trie[f].next[c];
                else {
                    trie[u].link = trie[f].next[c];
                    q.push(u);
                }
            }
        }
    }

    vector<pair<int, int>> match(const string& text) const {
        vector<pair<int, int>> ans;
        int v = 0;
        for (int i = 0; i < (int)text.size(); ++i) {
            int c = text[i] - 'a';
            assert(0 <= c && c < 26);
            v = trie[v].next[c];
            for (int id : trie[v].out) ans.push_back({i, id});
        }
        return ans;
    }
};
```

6.2 KMP

```
// Prefix function and all occurrences of pattern in text. O(n
+ m).
vector<int> prefixFunction(const string& s) {
    vector<int> pi(s.size());
    for (int i = 1; i < (int)s.size(); ++i) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) j = pi[j - 1];
        if (s[i] == s[j]) ++j;
        pi[i] = j;
    }
    return pi;
}
```

```
vector<int> kmpMatch(const string& text, const string&
pattern) {
    if (pattern.empty()) {
        vector<int> ans(text.size() + 1);
        iota(ans.begin(), ans.end(), 0);
        return ans;
    }
    vector<int> pi = prefixFunction(pattern), ans;
    for (int i = 0, j = 0; i < (int)text.size(); ++i) {
        while (j > 0 && text[i] != pattern[j]) j = pi[j - 1];
        if (text[i] == pattern[j]) ++j;
        if (j == (int)pattern.size()) {
            ans.push_back(i - j + 1);
            j = pi[j - 1];
        }
    }
    return ans;
}
```

6.3 Manacher

Time Complexity: $O(n)$

```
// even[i]: radius around gap before i; odd[i]: radius
excluding center i.
array<vector<int>, 2> manacher(const string& s) {
    int n = (int)s.size();
    array<vector<int>, 2> p = {vector<int>(n + 1),
    vector<int>(n)};
    for (int z = 0; z < 2; ++z) {
        for (int i = 0, l = 0, r = 0; i < n; ++i) {
            int t = r - i + !z;
            if (i < r) p[z][i] = min(t, p[z][l + t]);
            int L = i - p[z][i], R = i + p[z][i] - !z;
            while (L >= 1 && R + 1 < n && s[L - 1] == s[R +
            1])
                ++p[z][i], --L, ++R;
            if (R > r) l = L, r = R;
        }
    }
    return p;
}
```

6.4 Minimum Rotation

Time Complexity: $O(n)$

```
// Index of the lexicographically smallest cyclic rotation.
O(n).
int minRotation(const string& s) {
    int n = (int)s.size();
    if (n == 0) return 0;
    string t = s + s;
    int i = 0, j = 1, k = 0;
    while (i < n && j < n && k < n) {
        if (t[i + k] == t[j + k]) { ++k; continue; }
        if (t[i + k] > t[j + k]) i += k + 1;
        else j += k + 1;
        if (i == j) ++j;
        k = 0;
    }
    return min(i, j);
}
```

6.5 String Hashing

Usage: Arithmetic modulo $2^{64}-1$. **Time Complexity:** $O(n)$ build, $O(1)$ query

```
// Arithmetic modulo 2^64-1; robust against structured
anti-hash strings.
using ull = uint64_t;
struct Hash64 {
    ull x;
    Hash64(ull x = 0) : x(x) {}
    Hash64 operator+(Hash64 o) const { ull y = x + o.x; return
    y + (y < x); }
    Hash64 operator-(Hash64 o) const { return *this +
    Hash64(~o.x); }
    Hash64 operator*(Hash64 o) const {
        __uint128_t m = (__uint128_t)x * o.x;
        return Hash64((ull)m) + Hash64((ull)(m >> 64));
    }
    ull get() const { return x + (x == ULLONG_MAX); }
    bool operator==(Hash64 o) const { return get() == o.get();
    }
    bool operator<(Hash64 o) const { return get() < o.get(); }
};
const Hash64 HASH_BASE = 100000000003ULL;

struct StringHash {
    vector<Hash64> pref, power;
    explicit StringHash(const string& s) : pref(s.size() + 1),
    power(s.size() + 1) {
        power[0] = 1;
        for (int i = 0; i < (int)s.size(); ++i) {
            pref[i + 1] = pref[i] * HASH_BASE + (unsigned
            char)s[i] + 1;
            power[i + 1] = power[i] * HASH_BASE;
        }
    }
    Hash64 get(int l, int r) const {
        assert(0 <= l && l <= r && r < (int)pref.size());
        return pref[r] - pref[l] * power[r - l];
    }
};
```

6.6 Palindrome Tree

```
/* THIS SHIT RUN IN O(N) TOO */

struct PalindromeTree{
    const static int K = 26;
    struct Node{
        long child[K], layer, pi;
        Node(long _layer = 0){
            memset(child, -1, sizeof child);
            layer = _layer;
            pi = -1;
        }
    };

    vector<Node> a;

    PalindromeTree(){
        a.push_back(Node(-1)); a.push_back(Node(0));
        a[1].pi = a[0].pi = 0;
    }

    void add_child(long &x, long layer){
        x = a.size();
        a.push_back(Node(layer));
    }

    void build(string s){
        long id = 0;
        s = "#" + s;
        for(long i = 1; i<s.size(); ++i){
            long digit = s[i] - '0';
            long j = id;
            while(j >= 0){
                if (s[i] == s[i - a[j].layer - 1]) break;
                j = a[j].pi;
            }

            if (a[j].child[digit] != -1) {
                id = a[j].child[digit];
                continue;
            }

            add_child(a[j].child[digit], a[j].layer + 2);
            long v = a[j].child[digit];

            if (a[v].layer == 1) j = 1;
            else if (a[v].layer == 2) j = a[0].child[digit];
            else{
                j = a[j].pi;
                while(j >= 0){
                    if (j == 0 || (s[i] == s[i - a[j].layer -
                    1])) break;
                    j = a[j].pi;
                }
                j = a[j].child[digit];
            }
            a[v].pi = j;
```

```
            id = v;
        }
        cout << a.size() - 2 << "\n";
    }
};
```

6.7 Suffix Array

```
// sa includes empty suffix n at sa[0]. lcp[i] = LCP(sa[i-1],
sa[i]).
struct SuffixArray {
    vector<int> sa, lcp;
    explicit SuffixArray(string s, int lim = 256) {
        s.push_back('\0');
        int n = (int)s.size(), k = 0;
        vector<int> x(s.begin(), s.end()), y(n), ws(max(n,
        lim));
        sa.resize(n); lcp.assign(n, 0);
        iota(sa.begin(), sa.end(), 0);
        for (int j = 0, p = 0; p < n; j = max(1, 2 * j), lim =
        p) {
            p = j;
            iota(y.begin(), y.begin() + p, n - j);
            for (int i = 0; i < n; ++i) if (sa[i] >= j) y[p++]
            = sa[i] - j;
            fill(ws.begin(), ws.end(), 0);
            for (int i = 0; i < n; ++i) ++ws[x[i]];
            for (int i = 1; i < lim; ++i) ws[i] += ws[i - 1];
            for (int i = n - 1; i >= 0; --i) sa[--ws[x[y[i]]]]
            = y[i];
            swap(x, y); p = 1; x[sa[0]] = 0;
            for (int i = 1; i < n; ++i) {
                int a = sa[i - 1], b = sa[i];
                x[b] = y[a] == y[b] && y[a + j] == y[b + j] ?
                p - 1 : p++;
            }
        }
        for (int i = 0; i < n - 1; ++i) {
            int rank = x[i];
            if (rank == 0) { k = 0; continue; }
            if (k) --k;
            int j = sa[rank - 1];
            while (s[i + k] == s[j + k]) ++k;
            lcp[rank] = k;
        }
    }
};
```

6.8 Suffix Automaton

```
/* IT IS THE SHORTEST VER I CAN FIND */
// Time complexity : O(N)

struct SuffixAutomaton {
    vector<vector<int>> next;
    vector<int> len, link;
    int last, size;

    SuffixAutomaton(const string &s) {
        size = 1; last = 0;
        len.push_back(0); link.push_back(-1);
        next.push_back(vector<int>(26, -1));

        for (char c : s) {
            int cur = size++;
            len.push_back(len[last] + 1);
            next.push_back(vector<int>(26, -1));
            link.push_back(-1);

            int p = last;
            while (p != -1 && next[p][c - 'a'] == -1) {
                next[p][c - 'a'] = cur;
                p = link[p];
            }

            if (p == -1) link[cur] = 0;
            else {
                int q = next[p][c - 'a'];
                if (len[p] + 1 == len[q]) link[cur] = q;
                else {
                    int clone = size++;
                    len.push_back(len[p] + 1);
                    next.push_back(next[q]);
                    link.push_back(link[q]);
                    while (p != -1 && next[p][c - 'a'] == q) {
                        next[p][c - 'a'] = clone;
                        p = link[p];
                    }
                    link[q] = link[cur] = clone;
                }
            }
            last = cur;
        }
    }
};
```

6.9 Suffix Tree

Usage: Append a unique sentinel to expose every suffix. **Time Complexity:** $O(n \log |\Sigma|)$

```
// Online suffix tree (Ukkonen). Append a unique sentinel to
expose every suffix.
// Edges store half-open intervals [l, r) in s. O(n log
alphabet).
struct SuffixTree {
    struct Node {
        int l, r, parent, link = -1;
        map<char, int> next;
        Node(int l = 0, int r = 0, int p = -1) : l(l), r(r),
        parent(p) {}
        int len() const { return r - l; }
    };
    struct State { int v, pos; };

    string s;
    vector<Node> tree;
    State active{0, 0};

    explicit SuffixTree(string str) : s(move(str)) {
        tree.reserve(2 * s.size() + 1);
        tree.emplace_back(0, 0, -1);
        tree[0].link = 0;
        for (int i = 0; i < (int)s.size(); ++i) extend(i);
    }

    int child(int v, char c) const {
        auto it = tree[v].next.find(c);
        return it == tree[v].next.end() ? -1 : it->second;
    }
    State go(State st, int l, int r) const {
        while (l < r) {
            if (st.pos == tree[st.v].len()) {
                st = {child(st.v, s[l]), 0};
                if (st.v == -1) return st;
            } else {
                if (s[tree[st.v].l + st.pos] != s[l]) return
                {-1, -1};
                int take = min(r - l, tree[st.v].len() -
                st.pos);
                st.pos += take; l += take;
            }
        }
        return st;
    }
    int split(State st) {
        if (st.pos == tree[st.v].len()) return st.v;
        if (st.pos == 0) return tree[st.v].parent;
        Node edge = tree[st.v];
        int mid = (int)tree.size();
        tree.emplace_back(edge.l, edge.l + st.pos,
        edge.parent);
        tree[edge.parent].next[s[edge.l]] = mid;
        tree[mid].next[s[edge.l + st.pos]] = st.v;
        tree[st.v].parent = mid;
        tree[st.v].l += st.pos;
        return mid;
    }
    int getLink(int v) {
        if (tree[v].link != -1) return tree[v].link;
        if (tree[v].parent == -1) return 0;
        int to = getLink(tree[v].parent);
        int skip = tree[v].parent == 0;
        return tree[v].link = split(go({to, tree[to].len()},
```

```
                                    tree[v].l + skip,
                                    tree[v].r));
    }
    void extend(int pos) {
        while (true) {
            State next = go(active, pos, pos + 1);
            if (next.v != -1) { active = next; return; }
            int mid = split(active);
            int leaf = (int)tree.size();
            tree.emplace_back(pos, (int)s.size(), mid);
            tree[mid].next[s[pos]] = leaf;
            active.v = getLink(mid);
            active.pos = tree[active.v].len();
            if (mid == 0) break;
        }
    }

    bool contains(const string& pattern) const {
        if (pattern.empty()) return true;
        if (s.empty()) return false;
        State st = go({0, 0}, 0, 0);
        int v = 0, at = 0, i = 0;
        while (i < (int)pattern.size()) {
            if (at == tree[v].len()) {
                v = child(v, pattern[i]); at = 0;
                if (v == -1) return false;
            }
            int take = min((int)pattern.size() - i,
            tree[v].len() - at);
            for (int k = 0; k < take; ++k)
                if (pattern[i + k] != s[tree[v].l + at + k])
                return false;
            i += take; at += take;
        }
        return true;
    }
};
```

6.10 Z Function

```
/* YEP! IT IS JUST A Z FUNCTION */
//    Time complexity : O(N)

vector<int> ZFunction(const string &S) {
    int n = S.length();
    vector<int> Z(n);
    for (int L = 0, R = 0, i = 1; i < n; ++i) {
        if (i <= R) Z[i] = min(R - i + 1, Z[i - L]);
        while (i + Z[i] < n && S[Z[i]] == S[i + Z[i]]) Z[i]++;
        if (i + Z[i] - 1 > R) L = i, R = i + Z[i] - 1;
    }
    Z[0] = n;
    return Z;
}
```

7 Arithmetic

7.1 Number Theory

For $d = \gcd(a, b)$, $ax + by = c$ is solvable iff $d \mid c$. Given one solution (x_0, y_0), all solutions are

$$x = x_0 + k\frac{b}{d}, \qquad y = y_0 - k\frac{a}{d}, \qquad k \in \mathbb{Z}.$$

$$\varphi(n) = n\prod_{p\mid n}\left(1 - \frac{1}{p}\right), \qquad \sum_{d\mid n}\varphi(d) = n,$$

$$a^{\varphi(n)} \equiv 1 \pmod{n} \qquad (\gcd(a, n) = 1).$$

$$\mu(n) = \begin{cases} 0 & n \text{ is not square-free,} \\ (-1)^k & n \text{ is a product of } k \text{ distinct primes,} \end{cases} \qquad g(n) = \sum_{d\mid n} f(d)$$

$$\iff \qquad f(n) = \sum_{d\mid n}\mu(d)g(n/d).$$

Primitive roots exist modulo $2, 4, p^k, 2p^k$ for odd prime p. For prime p, $a^{p-1} \equiv 1 \pmod{p}$ when $p \nmid a$.

7.2 Modular Arithmetic

Time Complexity: $O(\log e)$ for power

```
// Safe for 0 <= a,b < mod < 2^64 (requires GCC/Clang
__int128).
using u64 = uint64_t;
using u128 = __uint128_t;

u64 modMul64(u64 a, u64 b, u64 mod) {
    return (u128)a * b % mod;
}

u64 modPow64(u64 a, u64 e, u64 mod) {
    u64 result = 1 % mod;
    while (e) {
        if (e & 1) result = modMul64(result, a, mod);
        a = modMul64(a, a, mod);
        e >>= 1;
    }
    return result;
}

// Returns gcd(a,b), and x,y such that ax+by=gcd(a,b).
ll extGcd(ll a, ll b, ll& x, ll& y) {
    if (!b) {
        x = (a >= 0 ? 1 : -1);
        y = 0;
        return abs(a);
    }
    ll x1, y1, g = extGcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - (a / b) * y1;
    return g;
}

// Inverse modulo m, or -1 when gcd(a,m) != 1.
ll modInverse(ll a, ll m) {
    ll x, y;
    if (extGcd(a, m, x, y) != 1) return -1;
    return (x % m + m) % m;
}

// All inverses modulo a prime; requires 1 < n <= mod.
vector<ll> inverseTable(int n, ll mod) {
    vector<ll> inv(n);
    if (n > 1) inv[1] = 1;
    for (int i = 2; i < n; ++i)
        inv[i] = mod - mod / i * inv[mod % i] % mod;
    return inv;
}
```

7.3 Fast Modulo

Usage: Barrett reduction for a fixed runtime modulus. **Time Complexity:** $O(1)$

```
// Barrett reduction for a fixed runtime modulus b. reduce(a)
is in [0, 2b).
// Use reduce(a) % b only when a canonical remainder is
required.
using ull = unsigned long long;
struct FastMod {
    ull b, m;
    explicit FastMod(ull mod) : b(mod), m(mod ? ULLONG_MAX /
    mod : 0) { assert(mod > 0); }
    ull reduce(ull a) const {
        return a - (ull)(((__uint128_t)m * a) >> 64) * b;
    }
};
```

7.4 Discrete Logarithm

Time Complexity: $O(\sqrt{m})$ expected

```
// Smallest x > 0 with a^x == b (mod m), or -1. Works when
gcd(a,m) > 1.
// O(sqrt(m)) expected time and memory.
ll discreteLog(ll a, ll b, ll m) {
    if (m == 1) return 1;
    a %= m;
    b = (b % m + m) % m;
    ll n = (ll)sqrt((long double)m) + 1;
    ll e = 1 % m, step = 1 % m, j = 1;
    unordered_map<ll, ll> baby;
    while (j <= n && (e = step = (__int128)e * a % m) != b)
        baby[(__int128)e * b % m] = j++;
    if (e == b) return j;
```

```
    if (gcd(m, e) == gcd(m, b)) {
        for (ll i = 2; i <= n + 1; ++i) {
            e = (__int128)e * step % m;
            auto it = baby.find(e);
            if (it != baby.end()) return n * i - it->second;
        }
    }
    return -1;
}
```

7.5 Sums Modulo

Time Complexity: $O(\log m)$

```
// sum_{i=0}^{to-1} floor((k*i+c)/m), for non-negative
arguments.
using u64 = uint64_t;

u64 arithmeticSum(u64 n) {
    return n / 2 * ((n - 1) | 1); // avoids overflow in
    n*(n-1)/2
}

u64 divSum(u64 to, u64 c, u64 k, u64 m) {
    u64 result = k / m * arithmeticSum(to) + c / m * to;
    k %= m;
    c %= m;
    if (!k) return result;
    u64 rows = (to * k + c) / m;
    return result + (to - 1) * rows - divSum(rows, m - 1 - c,
    m, k);
}

// sum_{i=0}^{to-1} ((k*i+c) mod m). Result must fit in signed
64-bit.
ll modSum(u64 to, ll c, ll k, ll m) {
    c = (c % m + m) % m;
    k = (k % m + m) % m;
    return (ll)(to * c + k * arithmeticSum(to)
              - m * divSum(to, c, k, m));
}
```

7.6 Linear Sieve, Phi and Mobius

Time Complexity: $O(n)$

```
// leastPrime[x], phi[x], mobius[x] for 1 <= x <= n in O(n).
struct LinearSieve {
    vector<int> primes, leastPrime, phi, mobius;

    LinearSieve(int n) : leastPrime(n + 1), phi(n + 1),
    mobius(n + 1) {
        if (n >= 1) phi[1] = mobius[1] = 1;
        for (int x = 2; x <= n; ++x) {
            if (!leastPrime[x]) {
                leastPrime[x] = x;
                phi[x] = x - 1;
                mobius[x] = -1;
                primes.push_back(x);
            }
            for (int p : primes) {
                if (p > leastPrime[x] || (ll)x * p > n) break;
                leastPrime[x * p] = p;
                if (x % p == 0) {
                    phi[x * p] = phi[x] * p;
                    mobius[x * p] = 0;
                } else {
                    phi[x * p] = phi[x] * (p - 1);
                    mobius[x * p] = -mobius[x];
                }
            }
        }
    }
};
```

7.7 Continued Fractions

Time Complexity: $O(\log N)$

```
// Closest p/q to x with 0 <= p,q <= limit. O(log limit).
pair<ll, ll> rationalApprox(long double x, ll limit) {
    ll lp = 0, lq = 1, p = 1, q = 0;
    long double y = x;
    while (true) {
        ll inf = LLONG_MAX;
        ll boundP = p ? (limit - lp) / p : inf;
        ll boundQ = q ? (limit - lq) / q : inf;
        ll a = (ll)floor(y), b = min(a, min(boundP, boundQ));
        ll np = b * p + lp, nq = b * q + lq;
        if (a > b) {
            long double first = nq ? fabsl(x - (long double)np
            / nq) : 1e100L;
            long double second = q ? fabsl(x - (long double)p
            / q) : 1e100L;
            return first < second ? make_pair(np, nq) :
            make_pair(p, q);
        }
        long double rem = y - a;
        if (fabsl(rem) < 1e-30L) return {np, nq};
        y = 1 / rem;
        if (fabsl(y) > 3 * (long double)limit) return {np,
        nq};
        lp = p; p = np;
        lq = q; q = nq;
    }
}
```

7.8 Fraction Binary Search

Time Complexity: $O(\log N)$ predicate calls

```
struct Fraction { ll p, q; };

// Smallest p/q in [0,1] satisfying monotone predicate f, with
p,q <= limit.
template<class F>
Fraction fractionBinarySearch(F f, ll limit) {
    bool direction = true, active = true, previousActive =
    true;
    Fraction low{0, 1}, high{1, 1};
    if (f(low)) return low;
    assert(f(high));
    while (active || previousActive) {
        ll advance = 0, step = 1;
        for (int stop = 0; step; (step *= 2) >>= stop) {
            advance += step;
            Fraction middle{
                low.p * advance + high.p,
                low.q * advance + high.q
            };
            if (abs(middle.p) > limit || middle.q > limit
                    || direction == !f(middle)) {
                advance -= step;
                stop = 2;
            }
        }
        high.p += low.p * advance;
        high.q += low.q * advance;
        direction = !direction;
        swap(low, high);
        active = previousActive;
        previousActive = (advance != 0);
    }
    return direction ? high : low;
}
```

7.9 Big Num Long

```
/* BIGNUM AGAIN!! BUT TESTED BY I_LOVE_HOANG_YEN */

// BigInt {{{
const int BASE_DIGITS = 9;
const int BASE = 1000000000;

struct BigInt {
    int sign;
    vector<int> a;

    // -------------------- Constructors --------------------
    // Default constructor.
    BigInt() : sign(1) {}

    // Constructor from long long.
    BigInt(long long v) {
        *this = v;
    }
    BigInt& operator = (long long v) {
```

```
        sign = 1;
        if (v < 0) {
            sign = -1;
            v = -v;
        }
        a.clear();
        for (; v > 0; v = v / BASE)
            a.push_back(v % BASE);
        return *this;
    }

    // Initialize from string.
    BigInt(const string& s) {
        read(s);
    }

    // -------------------- Input / Output
    --------------------
    void read(const string& s) {
        sign = 1;
        a.clear();
        int pos = 0;
        while (pos < (int) s.size() && (s[pos] == '-' ||
        s[pos] == '+')) {
            if (s[pos] == '-')
                sign = -sign;
            ++pos;
        }
        for (int i = s.size() - 1; i >= pos; i -= BASE_DIGITS)
        {
            int x = 0;
            for (int j = max(pos, i - BASE_DIGITS + 1); j <=
            i; j++)
                x = x * 10 + s[j] - '0';
            a.push_back(x);
        }
        trim();
    }
    friend istream& operator>>(istream &stream, BigInt &v) {
        string s;
        stream >> s;
        v.read(s);
        return stream;
    }

    friend ostream& operator<<(ostream &stream, const BigInt
    &v) {
        if (v.sign == -1 && !v.isZero())
            stream << '-';
        stream << (v.a.empty() ? 0 : v.a.back());
        for (int i = (int) v.a.size() - 2; i >= 0; --i)
            stream << setw(BASE_DIGITS) << setfill('0') <<
            v.a[i];
        return stream;
    }

    // -------------------- Comparison --------------------
    bool operator<(const BigInt &v) const {
        if (sign != v.sign)
            return sign < v.sign;
        if (a.size() != v.a.size())
            return a.size() * sign < v.a.size() * v.sign;
        for (int i = ((int) a.size()) - 1; i >= 0; i--)
            if (a[i] != v.a[i])
                return a[i] * sign < v.a[i] * sign;
        return false;
    }

    bool operator>(const BigInt &v) const {
        return v < *this;
    }
    bool operator<=(const BigInt &v) const {
        return !(v < *this);
    }
    bool operator>=(const BigInt &v) const {
        return !(*this < v);
    }
    bool operator==(const BigInt &v) const {
        return !(*this < v) && !(v < *this);
    }
    bool operator!=(const BigInt &v) const {
        return *this < v || v < *this;
    }

    // Returns:
    // 0 if |x| == |y|
    // -1 if |x| < |y|
    // 1 if |x| > |y|
    friend int __compare_abs(const BigInt& x, const BigInt& y)
    {
        if (x.a.size() != y.a.size()) {
            return x.a.size() < y.a.size() ? -1 : 1;
        }

        for (int i = ((int) x.a.size()) - 1; i >= 0; --i) {
            if (x.a[i] != y.a[i]) {
                return x.a[i] < y.a[i] ? -1 : 1;
            }
        }
        return 0;
    }

    // -------------------- Unary operator - and operators +-
    --------------------
    BigInt operator-() const {
        BigInt res = *this;
        if (isZero()) return res;

        res.sign = -sign;
        return res;
    }

    // Note: sign ignored.
    void __internal_add(const BigInt& v) {
        if (a.size() < v.a.size()) {
            a.resize(v.a.size(), 0);
        }
        for (int i = 0, carry = 0; i < (int) max(a.size(),
        v.a.size()) || carry; ++i) {
            if (i == (int) a.size()) a.push_back(0);

            a[i] += carry + (i < (int) v.a.size() ? v.a[i] :
            0);
            carry = a[i] >= BASE;
            if (carry) a[i] -= BASE;
        }
    }

    // Note: sign ignored.
    void __internal_sub(const BigInt& v) {
        for (int i = 0, carry = 0; i < (int) v.a.size() ||
        carry; ++i) {
            a[i] -= carry + (i < (int) v.a.size() ? v.a[i] :
            0);
            carry = a[i] < 0;
            if (carry) a[i] += BASE;
        }
        this->trim();
    }

    BigInt operator += (const BigInt& v) {
        if (sign == v.sign) {
            __internal_add(v);
        } else {
            if (__compare_abs(*this, v) >= 0) {
                __internal_sub(v);
            } else {
                BigInt vv = v;
                swap(*this, vv);
                __internal_sub(vv);
            }
        }
        return *this;
    }

    BigInt operator -= (const BigInt& v) {
        if (sign == v.sign) {
            if (__compare_abs(*this, v) >= 0) {
                __internal_sub(v);
            } else {
                BigInt vv = v;
                swap(*this, vv);
```

```
            __internal_sub(vv);
            this->sign = -this->sign;
        }
    } else {
        __internal_add(v);
    }
    return *this;
}

template< typename L, typename R >
    typename std::enable_if<
        std::is_convertible<L, BigInt>::value &&
        std::is_convertible<R, BigInt>::value &&
        std::is_lvalue_reference<R&&>::value,
        BigInt>::type friend operator + (L&& l, R&& r) {
    BigInt result(std::forward<L>(l));
    result += r;
    return result;
}
template< typename L, typename R >
    typename std::enable_if<
        std::is_convertible<L, BigInt>::value &&
        std::is_convertible<R, BigInt>::value &&
        std::is_rvalue_reference<R&&>::value,
        BigInt>::type friend operator + (L&& l, R&& r) {
    BigInt result(std::move(r));
    result += l;
    return result;
}

template< typename L, typename R >
    typename std::enable_if<
        std::is_convertible<L, BigInt>::value &&
        std::is_convertible<R, BigInt>::value,
        BigInt>::type friend operator - (L&& l, R&& r) {
    BigInt result(std::forward<L>(l));
    result -= r;
    return result;
}

// -------------------- Operators * / %
--------------------
friend pair<BigInt, BigInt> divmod(const BigInt& a1, const
BigInt& b1) {
    assert(b1 > 0);  // divmod not well-defined for b < 0.

    long long norm = BASE / (b1.a.back() + 1);
    BigInt a = a1.abs() * norm;
    BigInt b = b1.abs() * norm;
    BigInt q = 0, r = 0;
    q.a.resize(a.a.size());

    for (int i = a.a.size() - 1; i >= 0; i--) {
        r *= BASE;
        r += a.a[i];
        long long s1 = r.a.size() <= b.a.size() ? 0 :
        r.a[b.a.size()];
        long long s2 = r.a.size() <= b.a.size() - 1 ? 0 :
        r.a[b.a.size() - 1];
        long long d = ((long long) BASE * s1 + s2) /
        b.a.back();
        r -= b * d;
        while (r < 0) {
            r += b, --d;
        }
        q.a[i] = d;
    }

    q.sign = a1.sign * b1.sign;
    r.sign = a1.sign;
    q.trim();
    r.trim();
    auto res = make_pair(q, r / norm);
    if (res.second < 0) res.second += b1;
    return res;
}
BigInt operator/(const BigInt &v) const {
    if (v < 0) return divmod(-*this, -v).first;
    return divmod(*this, v).first;
}

BigInt operator%(const BigInt &v) const {
    return divmod(*this, v).second;
}

void operator/=(int v) {
    assert(v > 0);  // operator / not well-defined for v
    <= 0.
    if (llabs(v) >= BASE) {
        *this /= BigInt(v);
        return ;
    }
    if (v < 0)
        sign = -sign, v = -v;
    for (int i = (int) a.size() - 1, rem = 0; i >= 0; --i)
    {
        long long cur = a[i] + rem * (long long) BASE;
        a[i] = (int) (cur / v);
        rem = (int) (cur % v);
    }
    trim();
}

BigInt operator/(int v) const {
    assert(v > 0);  // operator / not well-defined for v
    <= 0.

    if (llabs(v) >= BASE) {
        return *this / BigInt(v);
    }
    BigInt res = *this;
    res /= v;
    return res;
}
void operator/=(const BigInt &v) {
    *this = *this / v;
}

long long operator%(long long v) const {
    assert(v > 0);  // operator / not well-defined for v
    <= 0.
    assert(v < BASE);
    int m = 0;
    for (int i = a.size() - 1; i >= 0; --i)
        m = (a[i] + m * (long long) BASE) % v;
    return m * sign;
}

void operator*=(int v) {
    if (llabs(v) >= BASE) {
        *this *= BigInt(v);
        return ;
    }
    if (v < 0)
        sign = -sign, v = -v;
    for (int i = 0, carry = 0; i < (int) a.size() ||
    carry; ++i) {
        if (i == (int) a.size())
            a.push_back(0);
        long long cur = a[i] * (long long) v + carry;
        carry = (int) (cur / BASE);
        a[i] = (int) (cur % BASE);
    }
    trim();
}

BigInt operator*(int v) const {
    if (llabs(v) >= BASE) {
        return *this * BigInt(v);
    }
    BigInt res = *this;
    res *= v;
    return res;
}

// Convert BASE 10^old --> 10^new.
static vector<int> convert_base(const vector<int> &a, int
old_digits, int new_digits) {
    vector<long long> p(max(old_digits, new_digits) + 1);
    p[0] = 1;
    for (int i = 1; i < (int) p.size(); i++)
        p[i] = p[i - 1] * 10;
    vector<int> res;
```

```
        long long cur = 0;
        int cur_digits = 0;
        for (int i = 0; i < (int) a.size(); i++) {
            cur += a[i] * p[cur_digits];
            cur_digits += old_digits;
            while (cur_digits >= new_digits) {
                res.push_back((long long)(cur %
                p[new_digits]));
                cur /= p[new_digits];
                cur_digits -= new_digits;
            }
        }
        res.push_back((int) cur);
        while (!res.empty() && !res.back())
            res.pop_back();
        return res;
    }

    void fft(vector<complex<double> > &x, bool invert) const {
        int n = (int) x.size();

        for (int i = 1, j = 0; i < n; ++i) {
            int bit = n >> 1;
            for (; j >= bit; bit >>= 1)
                j -= bit;
            j += bit;
            if (i < j)
                swap(x[i], x[j]);
        }

        for (int len = 2; len <= n; len <<= 1) {
            double ang = 2 * 3.14159265358979323846 / len *
            (invert ? -1 : 1);
            complex<double> wlen(cos(ang), sin(ang));
            for (int i = 0; i < n; i += len) {
                complex<double> w(1);
                for (int j = 0; j < len / 2; ++j) {
                    complex<double> u = x[i + j];
                    complex<double> v = x[i + j + len / 2] *
                    w;
                    x[i + j] = u + v;
                    x[i + j + len / 2] = u - v;
                    w *= wlen;
                }
            }
        }
        if (invert)
            for (int i = 0; i < n; ++i)
                x[i] /= n;
    }

    void multiply_fft(const vector<int> &x, const vector<int>
    &y, vector<int> &res) const {
        vector<complex<double> > fa(x.begin(), x.end());
        vector<complex<double> > fb(y.begin(), y.end());
        int n = 1;
        while (n < (int) max(x.size(), y.size()))
            n <<= 1;
        n <<= 1;
        fa.resize(n);
        fb.resize(n);

        fft(fa, false);
        fft(fb, false);
        for (int i = 0; i < n; ++i)
            fa[i] *= fb[i];
        fft(fa, true);

        res.resize(n);
        long long carry = 0;
        for (int i = 0; i < n; ++i) {
            long long t = (long long) (fa[i].real() + 0.5) +
            carry;
            carry = t / 1000;
            res[i] = t % 1000;
        }
    }

    BigInt mul_simple(const BigInt &v) const {
        BigInt res;
        res.sign = sign * v.sign;
        res.a.resize(a.size() + v.a.size());
        for (int i = 0; i < (int) a.size(); ++i)
            if (a[i])
                for (int j = 0, carry = 0; j < (int)
                v.a.size() || carry; ++j) {
                    long long cur = res.a[i + j] + (long long)
                    a[i] * (j < (int) v.a.size() ? v.a[j] : 0)
                    + carry;
                    carry = (int) (cur / BASE);
                    res.a[i + j] = (int) (cur % BASE);
                }
        res.trim();
        return res;
    }

    typedef vector<long long> vll;

    static vll karatsubaMultiply(const vll &a, const vll &b) {
        int n = a.size();
        vll res(n + n);
        if (n <= 32) {
            for (int i = 0; i < n; i++)
                for (int j = 0; j < n; j++)
                    res[i + j] += a[i] * b[j];
            return res;
        }

        int k = n >> 1;
        vll a1(a.begin(), a.begin() + k);
        vll a2(a.begin() + k, a.end());
        vll b1(b.begin(), b.begin() + k);
        vll b2(b.begin() + k, b.end());

        vll a1b1 = karatsubaMultiply(a1, b1);
        vll a2b2 = karatsubaMultiply(a2, b2);

        for (int i = 0; i < k; i++)
            a2[i] += a1[i];
        for (int i = 0; i < k; i++)
            b2[i] += b1[i];

        vll r = karatsubaMultiply(a2, b2);
        for (int i = 0; i < (int) a1b1.size(); i++)
            r[i] -= a1b1[i];
        for (int i = 0; i < (int) a2b2.size(); i++)
            r[i] -= a2b2[i];

        for (int i = 0; i < (int) r.size(); i++)
            res[i + k] += r[i];
        for (int i = 0; i < (int) a1b1.size(); i++)
            res[i] += a1b1[i];
        for (int i = 0; i < (int) a2b2.size(); i++)
            res[i + n] += a2b2[i];
        return res;
    }

    BigInt mul_karatsuba(const BigInt &v) const {
        vector<int> x6 = convert_base(this->a, BASE_DIGITS,
        6);
        vector<int> y6 = convert_base(v.a, BASE_DIGITS, 6);
        vll x(x6.begin(), x6.end());
        vll y(y6.begin(), y6.end());
        while (x.size() < y.size())
            x.push_back(0);
        while (y.size() < x.size())
            y.push_back(0);
        while (x.size() & (x.size() - 1))
            x.push_back(0), y.push_back(0);
        vll c = karatsubaMultiply(x, y);
        BigInt res;
        res.sign = sign * v.sign;
        long long carry = 0;
        for (int i = 0; i < (int) c.size(); i++) {
            long long cur = c[i] + carry;
            res.a.push_back((int) (cur % 1000000));
            carry = cur / 1000000;
        }
        res.a = convert_base(res.a, 6, BASE_DIGITS);
        res.trim();
        return res;
    }

    void operator*=(const BigInt &v) {
```

```
    *this = *this * v;
}
BigInt operator*(const BigInt &v) const {
    if (a.size() * v.a.size() <= 1000111) return
    mul_simple(v);
    if (a.size() > 500111 || v.a.size() > 500111) return
    mul_fft(v);
    return mul_karatsuba(v);
}

BigInt mul_fft(const BigInt& v) const {
    BigInt res;
    res.sign = sign * v.sign;
    multiply_fft(convert_base(a, BASE_DIGITS, 3),
    convert_base(v.a, BASE_DIGITS, 3), res.a);
    res.a = convert_base(res.a, 3, BASE_DIGITS);
    res.trim();
    return res;
}

// -------------------- Misc --------------------
BigInt abs() const {
    BigInt res = *this;
    res.sign *= res.sign;
    return res;
}
void trim() {
    while (!a.empty() && !a.back())
        a.pop_back();
    if (a.empty())
        sign = 1;
}

bool isZero() const {
    return a.empty() || (a.size() == 1 && !a[0]);
}

friend BigInt gcd(const BigInt &x, const BigInt &y) {
    return y.isZero() ? x : gcd(y, x % y);
}
friend BigInt lcm(const BigInt &x, const BigInt &y) {
    return x / gcd(x, y) * y;
}

friend BigInt sqrt(const BigInt &a1) {
    BigInt a = a1;
    while (a.a.empty() || a.a.size() % 2 == 1)
        a.a.push_back(0);

    int n = a.a.size();

    int firstDigit = (int) sqrt((double) a.a[n - 1] * BASE
    + a.a[n - 2]);
    int norm = BASE / (firstDigit + 1);
    a *= norm;
    a *= norm;
    while (a.a.empty() || a.a.size() % 2 == 1)
        a.a.push_back(0);

    BigInt r = (long long) a.a[n - 1] * BASE + a.a[n - 2];
    firstDigit = (int) sqrt((double) a.a[n - 1] * BASE +
    a.a[n - 2]);
    int q = firstDigit;
    BigInt res;

    for(int j = n / 2 - 1; j >= 0; j--) {
        for(; ; --q) {
            BigInt r1 = (r - (res * 2 * BigInt(BASE) + q)
            * q) * BigInt(BASE) * BigInt(BASE) + (j > 0 ?
            (long long) a.a[2 * j - 1] * BASE + a.a[2 * j
            - 2] : 0);
            if (r1 >= 0) {
                r = r1;
                break;
            }
        }
        res *= BASE;
        res += q;

        if (j > 0) {
            int d1 = res.a.size() + 2 < r.a.size() ?
            r.a[res.a.size() + 2] : 0;
            int d2 = res.a.size() + 1 < r.a.size() ?
            r.a[res.a.size() + 1] : 0;
            int d3 = res.a.size() < r.a.size() ?
            r.a[res.a.size()] : 0;
            q = ((long long) d1 * BASE * BASE + (long
            long) d2 * BASE + d3) / (firstDigit * 2);
        }
    }

    res.trim();
    return res / norm;
}
};
// }}}
```

7.10 Chinese Remainder Theorem

```
tuple<int64_t, int64_t, int64_t> extended_gcd(int64_t a,
int64_t b) {
    if (b == 0) {
        return {a, 1, 0};
    }
    int64_t gcd, x1, y1;
    tie(gcd, x1, y1) = extended_gcd(b, a % b);
    return {gcd, y1, x1 - (a / b) * y1};
}

pair<int64_t, int64_t> chinese_remainder_theorem(int64_t a1,
int64_t m1, int64_t a2, int64_t m2) {
    int64_t gcd, x, y;
    std::tie(gcd, x, y) = extended_gcd(m1, m2);

    if ((a2 - a1) % gcd != 0) {
        return {0, -1};
    }

    int64_t lcm = (m1 / gcd) * m2;
    int64_t combined_solution = a1 + ((a2 - a1) / gcd * x %
    (m2 / gcd)) * m1;
    combined_solution = (combined_solution % lcm + lcm) % lcm;

    return {combined_solution, lcm};
}
```

7.11 Diophantine

```
int gcd(int a, int b, int& x, int& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }
    int x1, y1;
    int d = gcd(b, a % b, x1, y1);
    x = y1;
    y = x1 - y1 * (a / b);
    return d;
}

bool find_any_solution(int a, int b, int c, int &x0, int &y0,
int &g) {
    g = gcd(abs(a), abs(b), x0, y0);
    if (c % g) {
        return false;
    }

    x0 *= c / g;
    y0 *= c / g;
    if (a < 0) x0 = -x0;
    if (b < 0) y0 = -y0;
    return true;
}



void shift_solution(int & x, int & y, int a, int b, int cnt) {
    x += cnt * b;
    y -= cnt * a;
}

int find_all_solutions(int a, int b, int c, int minx, int
maxx, int miny, int maxy) {
```

```
    int x, y, g;
    if (!find_any_solution(a, b, c, x, y, g))
        return 0;
    a /= g;
    b /= g;

    int sign_a = a > 0 ? +1 : -1;
    int sign_b = b > 0 ? +1 : -1;

    shift_solution(x, y, a, b, (minx - x) / b);
    if (x < minx)
        shift_solution(x, y, a, b, sign_b);
    if (x > maxx)
        return 0;
    int lx1 = x;

    shift_solution(x, y, a, b, (maxx - x) / b);
    if (x > maxx)
        shift_solution(x, y, a, b, -sign_b);
    int rx1 = x;

    shift_solution(x, y, a, b, -(miny - y) / a);
    if (y < miny)
        shift_solution(x, y, a, b, -sign_a);
    if (y > maxy)
        return 0;
    int lx2 = x;

    shift_solution(x, y, a, b, -(maxy - y) / a);
    if (y > maxy)
        shift_solution(x, y, a, b, sign_a);
    int rx2 = x;

    if (lx2 > rx2)
        swap(lx2, rx2);
    int lx = max(lx1, lx2);
    int rx = min(rx1, rx2);

    if (lx > rx)
        return 0;
    return (rx - lx) / abs(b) + 1;
}
```

7.12 Dirichlet Convolution

```
// Recovers prefix sum of f from prefix sums of g and f*g
modulo a prime.
struct DirichletSummatory {
    using Function = ll(*)(ll);
    ll limit = 0, inverse = 0, mod;
    Function prefixF, prefixG, prefixConvolution;
    unordered_map<ll, ll> memo;

    DirichletSummatory(Function prefixF, Function prefixG,
            Function prefixConvolution, ll mod)
        : mod(mod), prefixF(prefixF), prefixG(prefixG),
          prefixConvolution(prefixConvolution) {}

    ll power(ll base, ll exponent) const {
        ll result = 1;
        while (exponent) {
            if (exponent & 1) result = (__int128)result * base
            % mod;
            base = (__int128)base * base % mod;
            exponent >>= 1;
        }
        return result;
    }

    ll calculate(ll x) {
        if (x <= limit) return (prefixF(x) % mod + mod) % mod;
        auto found = memo.find(x);
        if (found != memo.end()) return found->second;
        ll subtracted = 0;
        for (ll left = 2, right; left <= x; left = right + 1)
        {
            right = x / (x / left);
            ll sumG = (prefixG(right) - prefixG(left - 1)) %
            mod;
            if (sumG < 0) sumG += mod;
            subtracted = (subtracted
                + (__int128)sumG * calculate(x / left)) % mod;
        }
        ll answer = (prefixConvolution(x) - subtracted) % mod;
        if (answer < 0) answer += mod;
        return memo[x] = (__int128)answer * inverse % mod;
    }

    // prefixF(x) must be directly available for x <=
    directLimit.
    ll solve(ll n, ll directLimit) {
        limit = directLimit;
        memo.clear();
        ll first = (prefixG(1) % mod + mod) % mod;
        assert(first != 0);
        inverse = power(first, mod - 2);
        return calculate(n);
    }
};
```

7.13 Extended GCD

```
int extendedGCD(int a, int b, int &x, int &y){
    x = 1, y= 0;
    int _x = 0, _y = 1;
    while(b > 0){
        int q = a / b;
        tie(x, _x) = make_tuple(_x, x - q * _x);
        tie(y, _y) = make_tuple(_y, y - q * _y);
        tie(a, b) = make_tuple(b, a - q * b);
    }
    return a;
}
```

7.14 Gaussian Elimination (Float)

```
// Solves A*x=b over reals. Returns rank, or -1 if
inconsistent.
// If rank < number of variables, one solution is returned
(free variables = 0).
int solveLinear(vector<vector<double>>& A, vector<double>& b,
vector<double>& x) {
    const double EPS = 1e-12;
    int n = (int)A.size(), m = n ? (int)A[0].size() :
    (int)x.size(), rank = 0;
    vector<int> where(m, -1);
    for (int col = 0; col < m && rank < n; ++col) {
        int pivot = rank;
        for (int r = rank + 1; r < n; ++r)
            if (abs(A[r][col]) > abs(A[pivot][col])) pivot =
            r;
        if (abs(A[pivot][col]) <= EPS) continue;
        swap(A[pivot], A[rank]); swap(b[pivot], b[rank]);
        where[col] = rank;
        for (int r = 0; r < n; ++r) if (r != rank) {
            double factor = A[r][col] / A[rank][col];
            b[r] -= factor * b[rank];
            for (int c = col; c < m; ++c) A[r][c] -= factor *
            A[rank][c];
        }
        ++rank;
    }
    for (int r = rank; r < n; ++r) if (abs(b[r]) > EPS) return
    -1;
    x.assign(m, 0);
    for (int col = 0; col < m; ++col)
        if (where[col] != -1) x[col] = b[where[col]] /
        A[where[col]][col];
    return rank;
}
```

7.15 Gaussian Elimination (Integer)

```
// Gaussian elimination modulo prime MOD_GAUSS. Returns rank
or -1 if inconsistent.
const long long MOD_GAUSS = 1000000009;
long long gaussPower(long long a, long long e) {
    long long r = 1;
    while (e) {
        if (e & 1) r = r * a % MOD_GAUSS;
        a = a * a % MOD_GAUSS; e >>= 1;
    }
    return r;
```

```
}
int solveLinearMod(vector<vector<long long>>& A, vector<long
long>& b,
                   vector<long long>& x) {
    int n = (int)A.size(), m = n ? (int)A[0].size() :
    (int)x.size(), rank = 0;
    auto norm = [](long long v) { v %= MOD_GAUSS; return v < 0
    ? v + MOD_GAUSS : v; };
    vector<int> where(m, -1);
    for (auto& row : A) for (long long& v : row) v = norm(v);
    for (long long& v : b) v = norm(v);
    for (int col = 0; col < m && rank < n; ++col) {
        int pivot = rank;
        while (pivot < n && A[pivot][col] == 0) ++pivot;
        if (pivot == n) continue;
        swap(A[pivot], A[rank]); swap(b[pivot], b[rank]);
        where[col] = rank;
        long long inv = gaussPower(A[rank][col], MOD_GAUSS -
        2);
        for (int c = col; c < m; ++c) A[rank][c] = A[rank][c]
        * inv % MOD_GAUSS;
        b[rank] = b[rank] * inv % MOD_GAUSS;
        for (int r = 0; r < n; ++r) if (r != rank &&
        A[r][col]) {
            long long factor = A[r][col];
            for (int c = col; c < m; ++c)
                A[r][c] = norm(A[r][c] - factor * A[rank][c] %
                MOD_GAUSS);
            b[r] = norm(b[r] - factor * b[rank] % MOD_GAUSS);
        }
        ++rank;
    }
    for (int r = rank; r < n; ++r) if (b[r]) return -1;
    x.assign(m, 0);
    for (int col = 0; col < m; ++col) if (where[col] != -1)
    x[col] = b[where[col]];
    return rank;
}
```

7.16 Solve Linear Binary

Usage: Solves $Ax = b$ over $\mathbb{F}_2$; returns rank or -1 if inconsistent.
Time Complexity: $O(n^2 m/w)$

```
// Solves A * x = b over F_2. Returns the rank, or -1 if no
// solution exists. A and b are destroyed; one solution is
stored in x.
// Multiple solutions exist iff rank < m. Complexity: O(n^2 *
m / word_size).

const int MAX_VARS = 1000;
using BS = bitset<MAX_VARS>;

int solveLinearBinary(vector<BS>& A, vector<int>& b, BS& x,
int m) {
    int n = (int)A.size(), rank = 0;
    assert((int)b.size() == n && m <= MAX_VARS);

    vector<int> col(m);
    iota(col.begin(), col.end(), 0);

    for (int i = 0; i < n && i < m; ++i) {
        int br = -1, bc = -1;
        for (int r = i; r < n && br == -1; ++r) {
            for (int c = i; c < m; ++c) {
                if (A[r][c]) {
                    br = r;
                    bc = c;
                    break;
                }
            }
        }
        if (br == -1) {
            for (int j = i; j < n; ++j)
                if (b[j]) return -1;
            break;
        }

        swap(A[i], A[br]);
        swap(b[i], b[br]);
        swap(col[i], col[bc]);
        for (int j = 0; j < n; ++j) {
            if (A[j][i] != A[j][bc]) {
                A[j].flip(i);
                A[j].flip(bc);
            }
        }

        for (int j = i + 1; j < n; ++j) {
            if (A[j][i]) {
                A[j] ^= A[i];
                b[j] ^= b[i];
            }
        }
        ++rank;
    }

    for (int i = rank; i < n; ++i)
        if (b[i]) return -1;

    x.reset();
    for (int i = rank - 1; i >= 0; --i) {
        if (!b[i]) continue;
        x[col[i]] = 1;
        for (int j = 0; j < i; ++j) b[j] ^= A[j][i];
    }
    return rank;
}
```

7.17 Lagrange Interpolation

```
// Coefficients of the unique degree < n polynomial through
(x[i],y[i]).
// mod must be prime and x values must be distinct modulo mod.
O(n^3).
vector<ll> lagrangePolynomial(
        const vector<ll>& x, const vector<ll>& y, ll mod) {
    int n = (int)x.size();
    assert((int)y.size() == n);
    auto power = [&](ll base, ll exponent) {
        ll result = 1;
        while (exponent) {
            if (exponent & 1) result = (__int128)result * base
            % mod;
            base = (__int128)base * base % mod;
            exponent >>= 1;
        }
        return result;
    };
    vector<ll> answer(n);
    for (int chosen = 0; chosen < n; ++chosen) {
        vector<ll> basis(1, 1);
        ll denominator = 1;
        for (int other = 0; other < n; ++other) if (other !=
        chosen) {
            vector<ll> next(basis.size() + 1);
            ll root = (x[other] % mod + mod) % mod;
            for (int degree = 0; degree < (int)basis.size();
            ++degree) {
                next[degree + 1] =
                    (next[degree + 1] + basis[degree]) % mod;
                next[degree] =
                    (next[degree] - (__int128)root *
                    basis[degree]) % mod;
                if (next[degree] < 0) next[degree] += mod;
            }
            basis.swap(next);
            ll difference = (x[chosen] - x[other]) % mod;
            if (difference < 0) difference += mod;
            denominator = (__int128)denominator * difference %
            mod;
        }
        assert(denominator != 0);
        ll scale = (__int128)((y[chosen] % mod + mod) % mod)
                * power(denominator, mod - 2) % mod;
        for (int degree = 0; degree < n; ++degree)
            answer[degree] =
                (answer[degree] + (__int128)basis[degree] *
                scale) % mod;
    }
    return answer;
}
```

7.18 Miller Rabin

```
namespace MillerRabin {
```

```
using u64 = uint64_t;
using u128 = __uint128_t;

u64 multiply(u64 a, u64 b, u64 mod) {
    return (u128)a * b % mod;
}

u64 power(u64 base, u64 exponent, u64 mod) {
    u64 result = 1;
    while (exponent) {
        if (exponent & 1) result = multiply(result, base,
        mod);
        base = multiply(base, base, mod);
        exponent >>= 1;
    }
    return result;
}

// Deterministic for every unsigned 64-bit integer.
bool isPrime64(u64 n) {
    if (n < 2) return false;
    for (u64 prime : {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL,
    17ULL,
                      19ULL, 23ULL, 29ULL, 31ULL, 37ULL})
        if (n % prime == 0) return n == prime;
    int shifts = 0;
    u64 odd = n - 1;
    while (!(odd & 1)) odd >>= 1, ++shifts;
    for (u64 base : {2ULL, 325ULL, 9375ULL, 28178ULL,
                     450775ULL, 9780504ULL, 1795265022ULL}) {
        if (base % n == 0) continue;
        u64 value = power(base % n, odd, n);
        if (value == 1 || value == n - 1) continue;
        bool composite = true;
        for (int round = 1; round < shifts; ++round) {
            value = multiply(value, value, n);
            if (value == n - 1) {
                composite = false;
                break;
            }
        }
        if (composite) return false;
    }
    return true;
}
}
```

7.19 Pollard Rho

```
// Requires MillerRabin above.
namespace PollardRho {
using u64 = uint64_t;

u64 findFactor(u64 n) {
    if (!(n & 1)) return 2;
    if (n % 3 == 0) return 3;
    u64 x = 0, y = 0, iteration = 30, product = 2, constant =
    1;
    auto advance = [&](u64 value) {
        return
        (u64)(((__uint128_t)MillerRabin::multiply(value,
        value, n)
                    + constant) % n);
    };
    while (iteration++ % 40 || gcd(product, n) == 1) {
        if (x == y) x = ++constant, y = advance(x);
        u64 difference = x > y ? x - y : y - x;
        u64 candidate = MillerRabin::multiply(product,
        difference, n);
        if (candidate) product = candidate;
        x = advance(x);
        y = advance(advance(y));
    }
    return gcd(product, n);
}

void factorRec(u64 n, map<u64, int>& factors) {
    if (n == 1) return;
    if (MillerRabin::isPrime64(n)) {
        ++factors[n];
        return;
    }
    u64 divisor = findFactor(n);
    factorRec(divisor, factors);
    factorRec(n / divisor, factors);
}

map<u64, int> factor(u64 n) {
    map<u64, int> result;
    factorRec(n, result);
    return result;
}
}
```

7.20 Pythagorean Triples

All Pythagorean triples are generated by

$$a = k(m^2 - n^2), \qquad b = 2kmn, \qquad c = k(m^2 + n^2),$$

where $m > n > 0$, $k > 0$, $\gcd(m, n) = 1$, and exactly one of m, n is even. The triple is primitive exactly when $k = 1$.

7.21 Sqrt Mod

```
// Tonelli-Shanks: one x with x^2 == a (mod odd prime p), or
-1.
ll modSqrt(ll a, ll p) {
    auto power = [&](ll base, ll exponent) {
        ll result = 1;
        while (exponent) {
            if (exponent & 1) result = (__int128)result * base
            % p;
            base = (__int128)base * base % p;
            exponent >>= 1;
        }
        return result;
    };
    a = (a % p + p) % p;
    if (a == 0 || p == 2) return a;
    if (power(a, (p - 1) / 2) != 1) return -1;
    if (p % 4 == 3) return power(a, (p + 1) / 4);
    ll odd = p - 1;
    int exponent = 0;
    while (!(odd & 1)) odd >>= 1, ++exponent;
    ll nonSquare = 2;
    while (power(nonSquare, (p - 1) / 2) != p - 1)
    ++nonSquare;
    ll x = power(a, (odd + 1) / 2);
    ll value = power(a, odd);
    ll generator = power(nonSquare, odd);
    while (value != 1) {
        ll current = value;
        int index = 0;
        while (current != 1) {
            current = (__int128)current * current % p;
            ++index;
        }
        ll correction = power(generator, 1LL << (exponent -
        index - 1));
        x = (__int128)x * correction % p;
        generator = (__int128)correction * correction % p;
        value = (__int128)value * generator % p;
        exponent = index;
    }
    return x;
}
```

8 Numerical

8.1 Real Polynomial, Roots and Interpolation

Time Complexity: $O(n^2 \log(1/\varepsilon))$

```
struct RealPolynomial {
    vector<double> a; // a[i] * x^i
    double operator()(double x) const {
        double value = 0;
        for (int i = (int)a.size() - 1; i >= 0; --i) value =
        value * x + a[i];
        return value;
    }
    RealPolynomial derivative() const {
        if (a.size() <= 1) return {{0}};
        vector<double> d(a.size() - 1);
```

```
        for (int i = 1; i < (int)a.size(); ++i) d[i - 1] = i *
        a[i];
        return {d};
    }
    void divideByRoot(double root) { // assumes root is a root
        for (int i = (int)a.size() - 2; i >= 0; --i) a[i] +=
        root * a[i + 1];
        a.erase(a.begin());
    }
};

vector<double> realRoots(RealPolynomial p, double lo, double
hi) {
    while (p.a.size() > 1 && abs(p.a.back()) < 1e-14)
    p.a.pop_back();
    if (p.a.size() <= 1) return {};
    if (p.a.size() == 2) {
        double x = -p.a[0] / p.a[1];
        return lo - 1e-10 <= x && x <= hi + 1e-10 ?
        vector<double>{x} : vector<double>{};
    }
    vector<double> critical = realRoots(p.derivative(), lo,
    hi), cuts{lo};
    cuts.insert(cuts.end(), critical.begin(), critical.end());
    cuts.push_back(hi);
    vector<double> roots;
    if (abs(p(lo)) < 1e-8) roots.push_back(lo);
    if (abs(p(hi)) < 1e-8) roots.push_back(hi);
    for (double x : critical) if (abs(p(x)) < 1e-8)
    roots.push_back(x);
    for (int i = 0; i + 1 < (int)cuts.size(); ++i) {
        double l = cuts[i], r = cuts[i + 1], fl = p(l), fr =
        p(r);
        if ((fl < 0) == (fr < 0) || fl == 0 || fr == 0)
        continue;
        for (int it = 0; it < 80; ++it) {
            double m = (l + r) / 2, fm = p(m);
            if ((fl < 0) == (fm < 0)) l = m, fl = fm;
            else r = m;
        }
        roots.push_back((l + r) / 2);
    }
    sort(roots.begin(), roots.end());
    roots.erase(unique(roots.begin(), roots.end(), [](double
    x, double y) {
        return abs(x - y) < 1e-7;
    }), roots.end());
    return roots;
}

// Coefficients of the degree < n polynomial through (x[i],
y[i]). O(n^2).
vector<double> interpolatePolynomial(vector<double> x,
vector<double> y) {
    int n = (int)x.size(); assert((int)y.size() == n);
    vector<double> result(n), basis(n); basis[0] = 1;
    for (int k = 0; k < n; ++k) {
        double denom = 1;
        for (int j = 0; j < k; ++j) denom *= x[k] - x[j];
        assert(abs(denom) > 1e-15);
        double coef = y[k];
        if (k > 0) {
            double known = 0;
            for (int i = n - 1; i >= 0; --i) known = known *
            x[k] + result[i];
            coef = y[k] - known;
        }
        coef /= denom;
        for (int i = 0; i < n; ++i) result[i] += coef *
        basis[i];
        if (k + 1 < n) {
            for (int i = k + 1; i >= 1; --i)
                basis[i] = basis[i - 1] - x[k] * basis[i];
            basis[0] *= -x[k];
        }
    }
    return result;
}
```

8.2 Golden Search and Hill Climbing

```
// Golden-section search: argument minimizing a unimodal f on
[lo, hi].
template<class F>
double goldenSearch(double lo, double hi, F f, double eps =
1e-9) {
    const double ratio = (sqrt(5.0) - 1) / 2;
    double x1 = hi - ratio * (hi - lo), x2 = lo + ratio * (hi
    - lo);
    double f1 = f(x1), f2 = f(x2);
    while (hi - lo > eps) {
        if (f1 < f2) {
            hi = x2; x2 = x1; f2 = f1;
            x1 = hi - ratio * (hi - lo); f1 = f(x1);
        } else {
            lo = x1; x1 = x2; f1 = f2;
            x2 = lo + ratio * (hi - lo); f2 = f(x2);
        }
    }
    return (lo + hi) / 2;
}

using Point2 = array<double, 2>;
template<class F>
pair<double, Point2> hillClimb(Point2 start, F f, double step
= 1e9) {
    pair<double, Point2> best = {f(start), start};
    for (; step > 1e-12; step /= 2) {
        for (int repeat = 0; repeat < 60; ++repeat)
            for (int dx = -1; dx <= 1; ++dx)
                for (int dy = -1; dy <= 1; ++dy) {
                    Point2 p = {best.second[0] + dx * step,
                    best.second[1] + dy * step};
                    best = min(best, make_pair(f(p), p));
                }
    }
    return best;
}
```

8.3 Simpson Integration

```
// Composite and adaptive Simpson integration.
template<class F>
double simpson(double a, double b, F f, int panels = 1000) {
    double h = (b - a) / (2 * panels), sum = f(a) + f(b);
    for (int i = 1; i < 2 * panels; ++i) sum += f(a + i * h) *
    (i & 1 ? 4 : 2);
    return sum * h / 3;
}

template<class F>
double adaptiveSimpsonRec(F& f, double a, double b, double
eps,
                          double whole, int depth) {
    double c = (a + b) / 2;
    auto area = [&](double l, double r) {
        return (f(l) + 4 * f((l + r) / 2) + f(r)) * (r - l) /
        6;
    };
    double left = area(a, c), right = area(c, b), sum = left +
    right;
    if (depth == 0 || abs(sum - whole) <= 15 * eps)
        return sum + (sum - whole) / 15;
    return adaptiveSimpsonRec(f, a, c, eps / 2, left, depth -
    1) +
           adaptiveSimpsonRec(f, c, b, eps / 2, right, depth -
           1);
}

template<class F>
double adaptiveSimpson(double a, double b, F f, double eps =
1e-9) {
    double whole = (f(a) + 4 * f((a + b) / 2) + f(b)) * (b -
    a) / 6;
    return adaptiveSimpsonRec(f, a, b, eps, whole, 30);
}
```

8.4 Simplex

Time Complexity: $O(NM \cdot \text{pivots})$

```
// Maximize c^T x subject to A*x <= b and x >= 0.
// Returns -INF if infeasible, +INF if unbounded; otherwise an
optimum. O(nm*pivots).
struct LPSolver {
    using Vec = vector<double>;
    static double eps() { return 1e-9; }
    static double inf() { return
    numeric_limits<double>::infinity(); }
    int m, n;
    vector<int> nonbasic, basic;
    vector<Vec> tab;

    LPSolver(const vector<Vec>& A, const Vec& b, const Vec& c)
        : m((int)b.size()), n((int)c.size()), nonbasic(n + 1),
        basic(m),
          tab(m + 2, Vec(n + 2)) {
        for (int i = 0; i < m; ++i)
            for (int j = 0; j < n; ++j) tab[i][j] = A[i][j];
        for (int i = 0; i < m; ++i) {
            basic[i] = n + i; tab[i][n] = -1; tab[i][n + 1] =
            b[i];
        }
        for (int j = 0; j < n; ++j) nonbasic[j] = j, tab[m][j]
        = -c[j];
        nonbasic[n] = -1; tab[m + 1][n] = 1;
    }

    void pivot(int r, int s) {
        double inv = 1 / tab[r][s];
        for (int i = 0; i < m + 2; ++i) if (i != r)
            for (int j = 0; j < n + 2; ++j) if (j != s)
                tab[i][j] -= tab[r][j] * tab[i][s] * inv;
        for (int j = 0; j < n + 2; ++j) if (j != s) tab[r][j]
        *= inv;
        for (int i = 0; i < m + 2; ++i) if (i != r) tab[i][s]
        *= -inv;
        tab[r][s] = inv;
        swap(basic[r], nonbasic[s]);
    }

    bool simplex(int phase) {
        int objective = m + phase - 1;
        while (true) {
            int s = -1;
            for (int j = 0; j <= n; ++j) {
                if (nonbasic[j] == -phase) continue;
                if (s == -1 || make_pair(tab[objective][j],
                nonbasic[j]) <
                               make_pair(tab[objective][s],
                               nonbasic[s])) s = j;
            }
            if (tab[objective][s] >= -eps()) return true;
            int r = -1;
            for (int i = 0; i < m; ++i) if (tab[i][s] > eps())
            {
                pair<double, int> cur = {tab[i][n + 1] /
                tab[i][s], basic[i]};
                if (r == -1 || cur < make_pair(tab[r][n + 1] /
                tab[r][s], basic[r])) r = i;
            }
            if (r == -1) return false;
            pivot(r, s);
        }
    }

    double solve(Vec& x) {
        if (m == 0) {
            x.assign(n, 0);
            for (int j = 0; j < n; ++j) if (tab[m][j] <
            -eps()) return inf();
            return 0;
        }
        int r = min_element(tab.begin(), tab.begin() + m,
            [&](const Vec& a, const Vec& b) { return a[n + 1]
            < b[n + 1]; }) - tab.begin();
        if (tab[r][n + 1] < -eps()) {
            pivot(r, n);
            if (!simplex(2) || tab[m + 1][n + 1] < -eps())
            return -inf();
            for (int i = 0; i < m; ++i) if (basic[i] == -1) {
                int s = 0;
                for (int j = 1; j <= n; ++j)
                    if (make_pair(tab[i][j], nonbasic[j]) <
                    make_pair(tab[i][s], nonbasic[s])) s = j;
                pivot(i, s);
            }
        }
        if (!simplex(1)) return inf();
        x.assign(n, 0);
        for (int i = 0; i < m; ++i) if (basic[i] < n)
        x[basic[i]] = tab[i][n + 1];
        return tab[m][n + 1];
    }
};
```

8.5 Determinant

Usage: Floating point and arbitrary-modulus variants. **Time Complexity:** $O(n^3)$

```
double determinant(vector<vector<double>> a) {
    int n = (int)a.size(); double ans = 1;
    for (int i = 0; i < n; ++i) {
        int pivot = i;
        for (int r = i + 1; r < n; ++r)
            if (abs(a[r][i]) > abs(a[pivot][i])) pivot = r;
        if (abs(a[pivot][i]) < 1e-12) return 0;
        if (pivot != i) swap(a[pivot], a[i]), ans = -ans;
        ans *= a[i][i];
        for (int r = i + 1; r < n; ++r) {
            double factor = a[r][i] / a[i][i];
            for (int c = i + 1; c < n; ++c) a[r][c] -= factor
            * a[i][c];
        }
    }
    return ans;
}

// Works for composite moduli too. Inputs and mod must fit
signed 64-bit.
long long determinantModulo(vector<vector<long long>> a, long
long mod) {
    assert(mod > 0);
    int n = (int)a.size(); long long ans = 1 % mod;
    auto norm = [&](long long x) { x %= mod; return x < 0 ? x
    + mod : x; };
    auto mul = [&](long long x, long long y) {
        return (long long)((__int128)norm(x) * norm(y) % mod);
    };
    for (auto& row : a) for (long long& x : row) x = norm(x);
    for (int i = 0; i < n; ++i) {
        for (int r = i + 1; r < n; ++r) {
            while (a[r][i] != 0) {
                long long q = a[i][i] / a[r][i];
                for (int c = i; c < n; ++c)
                    a[i][c] = norm(a[i][c] - mul(q, a[r][c]));
                swap(a[i], a[r]);
                ans = norm(-ans);
            }
        }
        ans = mul(ans, a[i][i]);
        if (ans == 0) return 0;
    }
    return ans;
}
```

8.6 Solve Linear 2

Usage: Returns every uniquely determined variable; others are NaN. **Time Complexity:** $O(nm\min(n,m))$

```
// Solves A*x=b and marks every non-uniquely-determined x[i]
as NaN.
// Returns rank, or -1 if inconsistent. A and b are destroyed.
O(n*m*min(n,m)).
int solveLinearDetermined(vector<vector<double>>& A,
vector<double>& b,
                          vector<double>& x) {
    const double EPS = 1e-12;
    int n = (int)A.size(), m = n ? (int)A[0].size() :
    (int)x.size(), rank = 0;
    vector<int> col(m); iota(col.begin(), col.end(), 0);
    while (rank < n && rank < m) {
        int pr = rank, pc = rank;
        for (int r = rank; r < n; ++r)
```

```
            for (int c = rank; c < m; ++c)
                if (abs(A[r][c]) > abs(A[pr][pc])) pr = r, pc
                = c;
        if (abs(A[pr][pc]) <= EPS) break;
        swap(A[rank], A[pr]); swap(b[rank], b[pr]);
        swap(col[rank], col[pc]);
        for (int r = 0; r < n; ++r) swap(A[r][rank],
        A[r][pc]);
        for (int r = 0; r < n; ++r) if (r != rank) {
            double factor = A[r][rank] / A[rank][rank];
            b[r] -= factor * b[rank];
            for (int c = rank; c < m; ++c) A[r][c] -= factor *
            A[rank][c];
        }
        ++rank;
    }
    for (int r = rank; r < n; ++r) if (abs(b[r]) > EPS) return
    -1;
    x.assign(m, numeric_limits<double>::quiet_NaN());
    for (int i = 0; i < rank; ++i) {
        bool fixed = true;
        for (int c = rank; c < m; ++c) fixed &= abs(A[i][c])
        <= EPS;
        if (fixed) x[col[i]] = b[i] / A[i][i];
    }
    return rank;
}
```

8.7 Matrix Inverse

Time Complexity: $O(n^3)$

```
// Inverts A in place using full pivoting. Returns rank; rank
< n means singular.
int matrixInverse(vector<vector<double>>& A) {
    const double EPS = 1e-12;
    int n = (int)A.size();
    vector<int> col(n); iota(col.begin(), col.end(), 0);
    vector<vector<double>> inv(n, vector<double>(n));
    for (int i = 0; i < n; ++i) inv[i][i] = 1;
    for (int i = 0; i < n; ++i) {
        int pr = i, pc = i;
        for (int r = i; r < n; ++r)
            for (int c = i; c < n; ++c)
                if (abs(A[r][c]) > abs(A[pr][pc])) pr = r, pc
                = c;
        if (abs(A[pr][pc]) < EPS) return i;
        swap(A[i], A[pr]); swap(inv[i], inv[pr]);
        for (int r = 0; r < n; ++r) swap(A[r][i], A[r][pc]);
        swap(col[i], col[pc]);
        double pivot = A[i][i];
        for (int c = 0; c < n; ++c) A[i][c] /= pivot,
        inv[i][c] /= pivot;
        for (int r = 0; r < n; ++r) if (r != i) {
            double factor = A[r][i];
            for (int c = 0; c < n; ++c) {
                A[r][c] -= factor * A[i][c];
                inv[r][c] -= factor * inv[i][c];
            }
        }
    }
    vector<vector<double>> answer(n, vector<double>(n));
    for (int i = 0; i < n; ++i) answer[col[i]] = inv[i];
    A = move(answer);
    return n;
}
```

8.8 Tridiagonal System

Time Complexity: $O(n)$

```
// Solves diag*x + super/sub diagonals = rhs. Stable for
diagonally dominant matrices.
vector<double> solveTridiagonal(vector<double> diag, const
vector<double>& super,
                                const vector<double>& sub,
                                vector<double> rhs) {
    int n = (int)diag.size();
    assert((int)rhs.size() == n && (int)super.size() == max(0,
    n - 1) &&
           (int)sub.size() == max(0, n - 1));
    for (int i = 0; i + 1 < n; ++i) {
        assert(abs(diag[i]) > 1e-15);
        double factor = sub[i] / diag[i];
        diag[i + 1] -= factor * super[i];
        rhs[i + 1] -= factor * rhs[i];
    }
    vector<double> x(n);
    if (n == 0) return x;
    assert(abs(diag[n - 1]) > 1e-15);
    x[n - 1] = rhs[n - 1] / diag[n - 1];
    for (int i = n - 2; i >= 0; --i)
        x[i] = (rhs[i] - super[i] * x[i + 1]) / diag[i];
    return x;
}
```

8.9 FFT (Integer)

```
// NTT convolution modulo 998244353. Inputs must be normalized
modulo MOD.
namespace NTT {
    using ll = long long;
    const ll MOD = 998244353, ROOT = 3;

    ll power(ll a, ll e) {
        ll result = 1;
        while (e) {
            if (e & 1) result = result * a % MOD;
            a = a * a % MOD; e >>= 1;
        }
        return result;
    }
    void transform(vector<ll>& a, bool inverse) {
        int n = (int)a.size();
        for (int i = 1, j = 0; i < n; ++i) {
            int bit = n >> 1;
            while (j & bit) j ^= bit, bit >>= 1;
            j ^= bit;
            if (i < j) swap(a[i], a[j]);
        }
        for (int len = 2; len <= n; len <<= 1) {
            ll root = power(ROOT, (MOD - 1) / len);
            if (inverse) root = power(root, MOD - 2);
            for (int i = 0; i < n; i += len) {
                ll w = 1;
                for (int j = 0; j < len / 2; ++j) {
                    ll u = a[i + j], v = a[i + j + len / 2] *
                    w % MOD;
                    a[i + j] = (u + v) % MOD;
                    a[i + j + len / 2] = (u - v + MOD) % MOD;
                    w = w * root % MOD;
                }
            }
        }
        if (inverse) {
            ll invN = power(n, MOD - 2);
            for (ll& x : a) x = x * invN % MOD;
        }
    }
    vector<ll> convolution(vector<ll> a, vector<ll> b) {
        if (a.empty() || b.empty()) return {};
        int need = (int)a.size() + (int)b.size() - 1, n = 1;
        while (n < need) n <<= 1;
        a.resize(n); b.resize(n);
        transform(a, false); transform(b, false);
        for (int i = 0; i < n; ++i) a[i] = a[i] * b[i] % MOD;
        transform(a, true); a.resize(need);
        return a;
    }
}
```

8.10 FFT (Float)

```
/*
Usage:
- conv: multiply A and B modulo 998244353
*/

namespace PolyMul {
    using namespace std;
    using cd = complex<double>;
    const double PI = acos(-1);

    // Perform FFT or inverse FFT
    void fft(vector<cd>& a, bool invert) {
        int n = a.size();
```

```
        for (int i = 1, j = 0; i < n; ++i) {
            int bit = n >> 1;
            for (; j & bit; bit >>= 1) j ^= bit;
            j ^= bit;
            if (i < j) swap(a[i], a[j]);
        }

        for (int len = 2; len <= n; len <<= 1) {
            double ang = 2 * PI / len * (invert ? -1 : 1);
            cd wlen(cos(ang), sin(ang));
            for (int i = 0; i < n; i += len) {
                cd w(1);
                for (int j = 0; j < len / 2; ++j) {
                    cd u = a[i + j], v = a[i + j + len / 2] *
                    w;
                    a[i + j] = u + v;
                    a[i + j + len / 2] = u - v;
                    w *= wlen;
                }
            }
        }

        if (invert) for (cd& x : a) x /= n;
    }

    // Multiply two polynomials
    vector<double> multiply(const vector<double>& a, const
    vector<double>& b) {
        if (a.empty() || b.empty()) return {};
        int n = 1;
        int total_size = a.size() + b.size() - 1;
        while (n < total_size) n <<= 1;  // Ensure n is a
        power of two

        vector<cd> fa(a.begin(), a.end()), fb(b.begin(),
        b.end());
        fa.resize(n);
        fb.resize(n);

        fft(fa, false);
        fft(fb, false);
        for (int i = 0; i < n; ++i) fa[i] *= fb[i];
        fft(fa, true);

        vector<double> result(total_size);
        for (int i = 0; i < total_size; ++i) result[i] =
        fa[i].real();
        return result;
    }
}
```

8.11 Matrix

```
template<class T>
struct Matrix {
    int rows, cols;
    vector<vector<T>> a;
    Matrix(int rows = 0, int cols = 0, T value = T())
        : rows(rows), cols(cols), a(rows, vector<T>(cols,
        value)) {}
    vector<T>& operator[](int i) { return a[i]; }
    const vector<T>& operator[](int i) const { return a[i]; }

    static Matrix identity(int n) {
        Matrix result(n, n);
        for (int i = 0; i < n; ++i) result[i][i] = T(1);
        return result;
    }
    Matrix operator*(const Matrix& other) const {
        assert(cols == other.rows);
        Matrix result(rows, other.cols);
        for (int i = 0; i < rows; ++i)
            for (int k = 0; k < cols; ++k)
                for (int j = 0; j < other.cols; ++j)
                    result[i][j] = result[i][j] + a[i][k] *
                    other[k][j];
        return result;
    }
    Matrix power(long long exponent) const {
        assert(rows == cols && exponent >= 0);
        Matrix base = *this, result = identity(rows);
        while (exponent) {
            if (exponent & 1) result = result * base;
            base = base * base;
            exponent >>= 1;
        }
        return result;
    }
};
```

8.12 Modular

```
/*
Usage: - use as if it was integers
*/

const int MOD = 998244353;
struct Modular{
    ll x;
    Modular(ll _x = 0){
        x = _x % MOD;
        if (x < 0) x += MOD;
    }
    Modular& operator += (Modular y){
        x += y.x;
        if (x >= MOD) x -= MOD;
        return *this;
    }
    Modular operator + (Modular y) {
        Modular tmp = *this;
        return tmp += y;
    }

    Modular& operator -= (Modular y){
        x -= y.x;
        if (x < 0) x += MOD;
        return *this;
    }
    Modular operator - (Modular y) {
        Modular tmp = *this;
        return tmp -= y;
    }

    Modular& operator *= (Modular y){
        x *= y.x;
        if (x >= MOD) x %= MOD;
        return *this;
    }
    Modular operator * (Modular y) {
        Modular tmp = *this;
        return tmp *= y;
    }

    // use at your own risk
    bool operator == (Modular y){
        return x == y.x;
    }
    bool operator != (Modular y){
        return x != y.x;
    }
};
ostream& operator << (ostream& out, Modular x){
    out << x.x;
    return out;
}
istream& operator >> (istream& in, Modular& x) {
    in >> x.x;
    return in;
}


Modular fast_pow(Modular a, ll n){
    Modular ans = 1;
    while(n > 0){
        if (n & 1) ans *= a;
        a *= a;
        n >>= 1;
    }
    return ans;
}
Modular inverse(Modular a){return fast_pow(a, MOD - 2);}
```

8.13 Berlekamp–Massey + Linear Recurrence

Usage: Finds a shortest recurrence, then evaluates its 0-indexed k-th term. **Time Complexity:** $O(N^2)$ / $O(L^2 \log k)$

```
// All operations are modulo mod; mod must be prime.
// berlekampMassey(s): finds the shortest recurrence
//   s[i] = sum(rec[j] * s[i - j - 1]).
// linearRec(init, rec, k): returns the 0-indexed k-th term.
// Complexity: O(N^2) and O(L^2 log k), respectively.

using ll = long long;
const ll mod = 998244353;

ll modpow(ll a, ll e) {
    ll res = 1;
    while (e > 0) {
        if (e & 1) res = res * a % mod;
        a = a * a % mod;
        e >>= 1;
    }
    return res;
}

vector<ll> berlekampMassey(vector<ll> s) {
    int n = (int)s.size(), L = 0, m = 0;
    if (n == 0) return {};
    vector<ll> C(n), B(n), T;
    C[0] = B[0] = 1;

    ll b = 1;
    for (int i = 0; i < n; ++i) {
        ++m;
        ll d = (s[i] % mod + mod) % mod;
        for (int j = 1; j <= L; ++j)
            d = (d + C[j] * s[i - j]) % mod;
        if (d == 0) continue;

        T = C;
        ll coef = d * modpow(b, mod - 2) % mod;
        for (int j = m; j < n; ++j)
            C[j] = (C[j] - coef * B[j - m]) % mod;
        if (2 * L > i) continue;

        L = i + 1 - L;
        B = T;
        b = d;
        m = 0;
    }

    C.resize(L + 1);
    C.erase(C.begin());
    for (ll& x : C) x = (mod - x) % mod;
    return C;
}

ll linearRec(vector<ll> init, vector<ll> rec, long long k) {
    int n = (int)rec.size();
    if (k < (long long)init.size()) return (init[k] % mod +
    mod) % mod;
    if (n == 0) return 0;
    assert((int)init.size() >= n);

    auto combine = [&](vector<ll> a, vector<ll> b) {
        vector<ll> res(2 * n + 1);
        for (int i = 0; i <= n; ++i)
            for (int j = 0; j <= n; ++j)
                res[i + j] = (res[i + j] + a[i] * b[j]) % mod;

        for (int i = 2 * n; i > n; --i)
            for (int j = 0; j < n; ++j)
                res[i - 1 - j] =
                    (res[i - 1 - j] + res[i] * rec[j]) % mod;
        res.resize(n + 1);
        return res;
    };

    vector<ll> pol(n + 1), e(n + 1);
    pol[0] = e[1] = 1;
    for (++k; k > 0; k >>= 1) {
        if (k & 1) pol = combine(pol, e);
        e = combine(e, e);
    }
    ll ans = 0;
    for (int i = 0; i < n; ++i)
        ans = (ans + pol[i + 1] * init[i]) % mod;
    return ans;
}
```

8.14 FFT + Poly Utilities

```
using ll = long long;
using C = complex<double>;
constexpr ll MOD = 998244353;
static inline ll modpow(ll x, ll e) {
    ll r = 1 % MOD;
    while (e) {
        if (e & 1) r = r * x % MOD;
        x = x * x % MOD;
        e >>= 1;
    }
    return r;

}

static inline ll invmod(ll x) { return modpow(x, MOD - 2); }
struct Poly {
    vector<ll> a;
    Poly(vector<ll> v = vector<ll>(0)) : a(v) {}
    // ---------- FFT (for convolution) ----------

    static void fft(vector<C> &f) {
        int n = (int)f.size(), L = 31 - __builtin_clz(n);
        static vector<complex<long double>> R(2, 1);
        static vector<C> rt(2, 1);
        for (int k = 2; k < n; k <<= 1) {
            R.resize(n); rt.resize(n);
            auto x = polar(1.0L, acosl(-1.0L) / k);
            for (int i = k; i < 2 * k; ++i) rt[i] = R[i] = (i
            & 1) ? R[i/2] * x : R[i/2];
        }
        vector<int> rev(n);
        for (int i = 0; i < n; ++i) rev[i] = (rev[i/2] | (i &
        1) << L) / 2;
        for (int i = 0; i < n; ++i) if (i < rev[i]) swap(f[i],
        f[rev[i]]);
        for (int k = 1; k < n; k <<= 1)
            for (int i = 0; i < n; i += k << 1)
                for (int j = 0; j < k; ++j) {
                    auto x = (double*)&rt[j+k], y =
                    (double*)&f[i+j+k];
                    C z(x[0]*y[0] - x[1]*y[1], x[0]*y[1] +
                    x[1]*y[0]);
                    f[i+j+k] = f[i+j] - z;
                    f[i+j] += z;
                }
    }
    // convolution using FFT-splitting trick (handles large
    MOD)
    static vector<ll> conv(const vector<ll>& A, const
    vector<ll>& B) {
        if (A.empty() || B.empty()) return {};
        int need = (int)A.size() + (int)B.size() - 1;
        int B2 = 32 - __builtin_clz(need), n = 1 << B2;
        int cut = (int)floor(sqrt(MOD));
        vector<C> L(n), R(n);
        for (size_t i = 0; i < A.size(); ++i) L[i] =
        C((ll)(A[i] / cut), (ll)(A[i] % cut));
        for (size_t i = 0; i < B.size(); ++i) R[i] =
        C((ll)(B[i] / cut), (ll)(B[i] % cut));
        fft(L); fft(R);
        vector<C> outl(n), outs(n);
        for (int i = 0; i < n; ++i) {
            int j = (-i) & (n - 1);
            outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
            outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) /
            C(0,1);
        }
        fft(outl); fft(outs);
        vector<ll> res(need);
        for (int i = 0; i < need; ++i) {
            ll av = (ll) (real(outl[i]) + 0.5) % MOD;
            ll cv = (ll) (imag(outs[i]) + 0.5) % MOD;
```

```
            ll bv = ((ll)(imag(outl[i]) + 0.5) +
            (ll)(real(outs[i]) + 0.5)) % MOD;
            res[i] = (( (av * cut + bv) % MOD) * cut + cv) %
            MOD;
        }
        return res;

    }

    // ---------- operators ----------
    Poly operator+(const Poly& o) const {
        vector<ll> r(max(a.size(), o.a.size()), 0);
        for (size_t i = 0; i < r.size(); ++i) {
            ll x = i < a.size() ? a[i] : 0;
            ll y = i < o.a.size() ? o.a[i] : 0;
            r[i] = (x + y) % MOD;
        }
        return Poly(r);
    }
    Poly operator-(const Poly& o) const {
        vector<ll> r(max(a.size(), o.a.size()), 0);
        for (size_t i = 0; i < r.size(); ++i) {
            ll x = i < a.size() ? a[i] : 0;
            ll y = i < o.a.size() ? o.a[i] : 0;
            r[i] = (x - y) % MOD;
            if (r[i] < 0) r[i] += MOD;
        }
        return Poly(r);
    }
    Poly operator*(const Poly& o) const {
        return Poly(conv(a, o.a));
    }

    // ---------- basic utilities ----------
    int deg() const { return (int)a.size() - 1; }
    void mod_xk(int k) { if ((int)a.size() > k) a.resize(k); }

    // ---------- calculus ----------
    Poly deri() const {
        if (a.size() <= 1) return Poly();
        vector<ll> r(a.size()-1);
        for (size_t i = 1; i < a.size(); ++i) r[i-1] = a[i] *
        (ll)i % MOD;
        return Poly(r);
    }
    Poly inte() const {
        vector<ll> r(a.size()+1);
        r[0] = 0;
        for (size_t i = 0; i < a.size(); ++i) r[i+1] = a[i] *
        invmod(i+1) % MOD;
        return Poly(r);
    }

    // ---------- inverse, log, exp ----------
    // inverse assumes a[0] != 0
    Poly inv() const {
        assert(!a.empty() && a[0] != 0);
        Poly R(vector<ll>{invmod(a[0])});

        int n = (int)a.size();
        int d = 1;
        while (d < n) {
            d <<= 1;
            vector<ll> f(d);
            for (int i = 0; i < (int)min(f.size(), a.size());
            ++i) f[i] = a[i];
            Poly F(f);

            Poly FR = F * R;
            FR.mod_xk(d);
            for (auto &x : FR.a)
                x = (MOD - x) % MOD;
            FR.a[0] = (FR.a[0] + 2) % MOD;
            R = R * FR;
            R.mod_xk(d);
        }
        R.mod_xk(n);
        return R;
    }

    // log assumes a[0] == 1
    Poly log(int k = -1) const {
        int need = (k == -1 ? (int)a.size() : k);
        assert(!a.empty() && a[0] == 1);
        Poly res = (deri() * inv()).mod_xk_here(need).inte();
        res.mod_xk(need);
        return res;
    }

    // helper for chaining mod_xk without mutating original
    Poly mod_xk_here(int k) const { Poly r = *this;
    r.mod_xk(k); return r; }

    Poly exp() const {
        assert(!a.empty() && a[0] == 0);
        Poly R({1});
        int n = a.size();
        for (int len = 2; ; len <<= 1) {
            R.a.resize(len, 0);
            Poly Ln = R.log(len);
            Ln.a[0] = 1;
            for (int i = 1; i < len; ++i) {
                ll val = (i < n ? a[i] : 0);
                Ln.a[i] = val - Ln.a[i];
                if (Ln.a[i] < 0) Ln.a[i] += MOD;
            }
            R = R * Ln;
            R.mod_xk(len);

            if (len >= n) break;
        }

        R.mod_xk(n);
        return R;
    }
};
```

8.15 AND / OR / XOR Convolution

```
// AND / OR / XOR subset convolution. Size must be a power of
two.
enum class SubsetOp { AND, OR, XOR };
const long long SUBSET_MOD = 998244353;

long long subsetPower(long long a, long long e) {
    long long r = 1;
    while (e) {
        if (e & 1) r = r * a % SUBSET_MOD;
        a = a * a % SUBSET_MOD; e >>= 1;
    }
    return r;
}
void subsetTransform(vector<long long>& a, SubsetOp op, bool
inverse) {
    int n = (int)a.size(); assert(n && !(n & (n - 1)));
    for (int step = 1; step < n; step <<= 1)
        for (int i = 0; i < n; i += 2 * step)
            for (int j = 0; j < step; ++j) {
                long long& x = a[i + j], &y = a[i + j + step];
                if (op == SubsetOp::XOR) {
                    long long u = x, v = y;
                    x = (u + v) % SUBSET_MOD;
                    y = (u - v + SUBSET_MOD) % SUBSET_MOD;
                } else if (op == SubsetOp::OR) {
                    y = (y + (inverse ? -x : x) + SUBSET_MOD)
                    % SUBSET_MOD;
                } else {
                    x = (x + (inverse ? -y : y) + SUBSET_MOD)
                    % SUBSET_MOD;
                }
            }
    if (inverse && op == SubsetOp::XOR) {
        long long inv = subsetPower(n, SUBSET_MOD - 2);
        for (long long& x : a) x = x * inv % SUBSET_MOD;
    }
}
vector<long long> subsetConvolution(vector<long long> a,
vector<long long> b,
                                    SubsetOp op) {
    assert(a.size() == b.size());
    subsetTransform(a, op, false); subsetTransform(b, op,
    false);
    for (int i = 0; i < (int)a.size(); ++i) a[i] = a[i] * b[i]
    % SUBSET_MOD;
```

```
    subsetTransform(a, op, true);
    return a;
}
```